
Talk is cheap, JUST DO IT

基于**ESPduino**的智慧物联开发宝典

四博智联

序

据美国国家大气监测组织说 2015 年是厄尔尼诺年，主要指海洋洋面温度异常增暖，从而造成地球大气环流异常的现象，这种现象会对地球自然生态带来消极影响。想想也是，一只巴西亚马逊的蝴蝶扇扇翅膀，就可能给大上海带来一场大风暴，何况海洋温度升高这一等一的全球大事？其实，人类社会未尝没有厄尔尼诺，尤其是科技、资本这类颜值很高的东东，给社会带来的改变说销魂蚀骨也不为过。幸运的是，人类自己总要把握自己命运的，是好是坏取决于人们努力的方向。2015 年的互联网+、大众创新、全民创业，以及物联网真正地开始落地，给相关产业的汪洋大海增温不少，所带来的是社会的厄尔尼诺还是新世界的起点，取决于我们的行动——那就是在这片热土中，务实而不虚夸，勤奋而非取巧，把热度变为踏踏实实的质感，形成产业的良性循环，在这个实体企业的寒冬中点亮未来高技术制造的明灯。因此，在我从师兄李洪刚博士那里了解到要做这样一本书时，一开始我就是赞同的。（句式没-有-错！）

李博士做企业多年，其嵌入式和云计算平台的开发技术炉火纯青，更经历了中国科学院的学习与锤炼，在技术视野、市场把握、战略机遇等方面都已达到了更高的境界。他作为四博智联公司的联合创始人，主导了深圳四博智联公司的创立和发展。公司的硬件设计和研发则由尚留记博士负责，他和杨平博士均毕业于中国科学院自动化所，其中尚博以对国家核电控制事业精益求精的精神对待公司产品的设计，而杨博则主导了公司整个产品系列的固件程序开发和嵌入式设计。易称福作为中山大学的博士，同时也是中国科学院的博士后，因此，他需要做更重要的工作——市场拓展。四位博士的共同努力，使四博智联的产品日益壮大，在短时期内就获得国内外业界同行的关注。在这个历程中，苦于人们对物联网落地的困惑以及对技术门槛的畏惧，深感有必要把自身创业实践中的思想、技术、经验、教训拿出来分享，共同培育万众创新的大好局面，同时也成为鞭策自身向着更新、更高方向发展的一股力量。本人忝列公司的战略顾问，却往往问而不顾，甚是惭愧。在得知成书意向后，遂自告奋勇，参与捉刀代笔，希望能为开源社区和智能硬件的发展贡献绵薄之力。

本书由李、杨二位博士草拟了大纲并提供了所用到的大部分源代码，本人对其进行了逐一测试，进一步验证了产品的有效性，并撰写了前两篇内容；尚博提供了杠杠的硬件环境，易博则以他专业的素养指导了很多具体写作的细节；最后又由李博撰写了第三篇内容并审阅了全书。在此，特为本书付梓做出了辛勤努力的同伴致以真挚的感谢——同伴们的精诚团结

与无私合作是这项任务能够按时完成的充要条件，当然也离不开 **Arduino** 和 **github** 开源社区的重要支持，在此一并谢过。

最后，也是最重要的，向在各位背后默默付出，无条件支持我们的家人致以最高的敬意！

何晨光 博士

中国科学院深圳先进技术研究院

2016 年 1 月 15 日于西丽大学城

前言

欢迎来到“JUST DO IT”的世界！—— 我很喜欢这类科技应用书籍中最惯常出现的句式，虽然落了俗套，但言简意赅，同高级编程语言中每次必讲的首课“Hello, World！”异曲同工。是的，本书就是本有关指导开源硬件开发和应用的“Hello, World”。在本书中你将能够接触到当下最流行的开源硬件系统——Arduino——特别适于各种小白级大咖们学习和掌握对硬件的编程与控制方法。基于此，本书立足于四博智联（Doit）自主开发的 ESPDuino 物联网开发环境，将传统的 Arduino 赋予了更加便利和丰富的功能，为当下风起云涌的物联世界又提供了一柄屠龙宝刀，感谢它给我们带来了更多的可能性。

ESPDuino 正如其名称所示，来源于两个词根 ESP 和 DUINO，前者指今年在物联网领域大行其道的 ESP8266 系列 WiFi 芯片，后者则指 Arduino。四博智联的 Doiter 们将其二者有机地结合起来，构造了自主的 SDK 环境，再辅以公司已有的久经考验的成熟云计算平台，在稳定性、开发效率、可扩展性等方面都有了大大的提高；同时，以长期致力于开源社区的分享精神初步构建的 ESPDuino 生态社区，形成了崭新的、独具特色的物联网开发环境。所以，无论你是小白还是大咖，无论是搞硬件还是软件，甚至无论你是哪个行业，只要对物联网感兴趣，都可以借助本书的脉络一试物联技术开发的风采，真正将传说中各种华丽丽的智能控制和物联网应用落在实处——毕竟，踩在大地上的感觉还是踏实许多。

本书主要讲述 ESPDuino 在物联网的一些典型环境中，是如何达到的所需的技术要求的，并一步一步实现给大家看。在这一过程中，理论不会涉及太多，但关键原理会以简明易懂的方式进行阐述。阅读本书也许还需要具备一些初步的 C 语言编程知识，至于对电子硬件的了解，初中物理水平就够了（也许还可以更少）。本书重点是采用亲自实践的办法，用“HOW TO”的方式告诉你应该怎么做。在每一章的结束，你都可以掌握一些最有用的技术，最终，可以独立地开发一些物联网项目，这也是本书的目的。

本书的内容分为主要三个部分：基础篇、进阶篇和产品篇。每一篇下又有若干章节，会独立讲授一些在物联开发中所遇到的问题和解决方案。所以，有一定基础的读者可以按需挑选相关内容来阅读。初学者建议按照章目顺序学习，由简到繁，这样可以形成技术的累积，构建自己的技术框架，以最少时间获得前人多年的技术开发经验，倘如此，真正是开卷有益。

本书内容概述如下：

一、基础篇从缘起讲述了 ESPDuino 的来龙去脉，手把手讲述了相应的开发环境，通过

一个读取服务器时钟的例子告诉我们硬件与外部世界是如何联系的。接着，会学习如何让硬件按照你想要的方式来控制，本篇末尾的“盲连接”直接帮你搞定了物联世界的“最后 500 米”。

二、进阶篇则带你进入到你也许自以为很熟悉的 TCP/IP 网络，包括 Internet 的 Web 服务以及 Web 浏览器，包括很多细节。但本篇会告诉你为什么以及如何在底层协议上实现 ESPDuino 硬件为物联网环境提供服务的机制，这些服务都是“超视距”的，使读者知其然也知其所以然，因此也是最为有用的，不要错过。

三、产品篇综合前两篇的各种技术，帮助你形成项目的基本技术框架。通过物联网的设备接入、和微信控制的有机结合、智能坦克小车的设计一直到 ESPDuino 的智能信息站、初级的智能机器人手臂的产品级开发实践，相信会为你的物联人生开拓更广阔的视野。尤其点赞的是，根据本篇内容，你会做出一模一样的智能产品，甚至更酷更炫——不怕你做不到，只怕你想不到！

在学习过程中，实验所需的开发板、软件兴趣小组的信息以及开源社区的资源，包括本书的简易电子版，都可以在四博智联公司以及本书的官方网站上获得：www.doit.am，点进去又是一个大千世界！

光说不练嘴把式，光练不说傻把式，又说又练才是真把式。西谚有云：Talk is cheap，那么大伙一起 Do 起来吧——Just Do IT！

四博智联 Doiters

2016 年 1 月 15 日

本书抛砖引玉，欢迎吐槽！

目录

序	I
前言	III
目录	V
一、基础篇	1
第 1 章 从 Arduino 谈起	1
1.1 Arduino 的前世今生	1
1.1.1 Arduino 能做什么	1
1.1.2 为什么选择它	2
1.1.3 优点？缺点？	4
1.2 ESP8266 是什么	4
1.2.1 ESP8266 特点	5
1.2.2 为什么要用 ESP8266	6
1.2.3 ESP8266 使用场景	7
1.3 什么是 ESPDuino	7
1.3.1 强强联合就开挂	7
1.3.2 ESPDuino 庐山真面	8
1.3.3 学或者玩，Just Do It	9
第 2 章 如何安装 ESPDuino 开发环境	10
2.1 欲善其事必利其器	10
2.1.1 安装 ESPDuino 开发环境	10
2.1.2 搭建 ESPDuino 环境第二种方法	16
2.1.3 要点及注意事项	18
2.1.4 常见问题	18
2.2 样例程序	19
2.2.1 程序功能	19
2.3 源码分析	22
第 3 章 键控 LED 灯，闪闪告诉你	24
3.1 引言	24
3.2 物料准备	24
3.2.1 硬件	24
3.2.2 软件	25
3.3 三个不同的闪灯示例	25
3.3.1 自动闪烁	25
3.3.2 按键控制	25
3.3.3 中断方式	27
第 4 章 PWM，你也做得到	29
4.1 PWM 的原理	29
4.2 实验效果	29
4.3 源码分析	30
第 5 章 Soft AP—连上我就对了	31
5.1 什么是 Soft AP 模式	31

5.2 实验效果	32
5.3 源码分析	33
第 6 章 STA 连接—我要连.....	35
6.1 什么是 STA 模式.....	35
6.2 连接过程	35
6.2.1 STA 连接 WiFi 路由器.....	35
6.2.2 实验效果.....	36
6.2.3 源码分析.....	37
6.3 AP+STA 共存模式	38
6.3.1 AP+STA 工作原理.....	38
6.3.2 实验效果.....	38
6.3.3 源码分析.....	39
第 7 章 Scan WiFi—众里寻他千百度	41
7.1 我扫故我在	41
7.2 试验效果	41
7.3 源码分析	42
第 8 章 从 SmartConfig 到 AirKiss.....	43
8.1 SmartConfig 的作用.....	43
8.1.1 为什么会有 SmartConfig.....	43
8.1.2 SmartConfig 工作流程:	43
8.1.3 SmartConfig 工作原理.....	44
8.1.4 实现细节.....	45
8.2 SmartConfig ESPDuino	46
8.2.1 工作流程.....	46
8.2.2 源码分析.....	47
8.3 手机设置	48
8.3.1 通过专用 app 设置.....	48
8.3.2 通过微信设置.....	52
二、进阶篇.....	55
第 9 章 TCP Server/TCP Client	55
9.1 TCP Server	55
9.1.1 实验步骤.....	55
9.1.2 源码分析.....	58
9.2 TCP Client	59
9.2.1 实验步骤.....	59
9.2.2 具体方法.....	60
9.2.3 源码分析.....	70
第 10 章 UDP 服务	72
10.1 UDP Server	72
10.2 实验步骤	72
10.3 源码分析	76
第 11 章 WebServer, 远程控制硬件.....	78
11.1 连接人和硬件的桥梁.....	78
11.2 实验步骤	78

11.3 源码分析	81
第 12 章通过 Web OTA 升级固件	85
12.1 什么是 web OTA	85
12.2 实验步骤	85
12.2.1 设置固件环境参数	85
12.2.2 使用 STA 模式	89
12.2.3 使用 AP 模式	91
12.3 源码分析	93
第 13 章一个准产品的例子	95
13.1 产品概述	95
13.2 功能描述	95
13.3 实验步骤	96
13.4 项目结构分析	108
13.4.1 主程序分析	108
13.4.2 HTTP 服务器配置	110
13.4.3 网络任务处理	111
13.4.4 协议解析	112
13.4.5 头文件处理	113
13.5 所用技术小结	115
三、产品篇	116
第 14 章DoIT 智能控制三色灯	116
14.1 产品概述	116
第 15 章微信控制智能小车	116
第 16 章ESPduino MP3	118
第 17 章ESPduino WiFi 屏智能信息站	119
第 18 章ESPduino 机械臂	120
附录-ESP8266 Arduino 核心说明	122
致谢	129

一、基础篇

第 1 章 从 Arduino 谈起

1.1 Arduino 的前世今生

Arduino 是一款便捷灵活、方便上手的开源电子原型平台。包含硬件（各种型号的 Arduino 板）和软件（Arduino IDE）。由一个欧洲开发团队于 2005 年冬季开发。其成员包括 Massimo Banzi、David Cuartielles、Tom Igoe、Gianluca Martino、David Mellis 和 Nicholas Zambetti。Massimo Banzi 之前是意大利 Ivrea 一家高科技设计学校的老师。他的学生们经常抱怨找不到便宜好用的微控制器。2005 年冬天，Massimo Banzi 跟 David Cuartielles 讨论了这个问题。David Cuartielles 是一个西班牙籍晶片工程师，当时在这所学校做访问学者。两人决定设计自己的电路板，并引入了 Banzi 的学生 David Mellis 为电路板设计编程语言。两天以后，David Mellis 就写出了程式码。又过了三天，电路板就完工了。Massimo Banzi 喜欢去一家名叫 di Re Arduino 的酒吧，该酒吧是以 1000 年前意大利国王 Arduin 的名字命名的。为了纪念这个地方，他将这块电路板命名为 Arduino。

随后 Banzi、Cuartielles 和 Mellis 把设计图放到了网上。版权法可以监管开源软件，却很难用在硬件上，为了保持设计的开放源码理念，他们决定采用 Creative Commons(CC)的授权方式公开硬件设计图。在这样的授权下，任何人都可以生产电路板的复制品，甚至还能重新设计和销售原设计的复制品。人们不需要支付任何费用，甚至不用取得 Arduino 团队的许可。然而，如果重新发布了引用设计，就必须声明原始 Arduino 团队的贡献。如果修改了电路板，则最新设计必须使用相同或类似的 Creative Commons(CC)的授权方式，以保证新版本的 Arduino 电路板也会一样是自由和开放的。唯一被保留的只有 Arduino 这个名字，它被注册成了商标，在没有官方授权的情况下不能使用。

1.1.1 Arduino 能做什么

Arduino 构建于开放原始码 simple I/O 介面版，并且具有使用类似 Java、C 语言的 Processing/Wiring 开发环境。主要包含两个主要的部分：硬件部分是可以用来做电路连接的

Arduino 电路板；另外一个则是 Arduino IDE，你的计算机中的程序开发环境。你只要在 IDE 中编写程序代码，将程序上传到 Arduino 电路板后，程序便会告诉 Arduino 电路板要做些什么了。

Arduino 能通过各种各样的传感器来感知环境，通过控制灯光、马达和其他的装置来反馈、影响环境。板子上的微控制器可以通过 Arduino 的编程语言来编写程序，编译成二进制文件，烧录进微控制器。对 Arduino 的编程是利用 Arduino 编程语言 (基于 Wiring) 和 Arduino 开发环境(基于 Processing)来实现的。基于 Arduino 的项目，可以只包含 Arduino，也可以包含 Arduino 和其他一些在 PC 上运行的软件，他们之间进行通信 (比如 Flash, Processing, MaxMSP)来实现。

另外一类很酷的应用就是交互艺术，有很多艺术学院的师生拿它搞出很酷很炫的艺术品出来，其特有的交互性是其它艺术品类型所不具备的。一些具体例子可以参看 <http://www.zhihu.com/question/29441922>。想想就有些小激动呢，搞艺术的都能玩转，我们有什么理由不中呢。

1.1.2 为什么选择它

Arduino 不仅仅是全球最流行的开源硬件，也是一个优秀的硬件开发平台，更是硬件开发的趋势。Arduino 简单的开发方式使得开发者更关注创意与实现，更快的完成自己的项目开发，大大节约了学习的成本，缩短了开发的周期。考虑到 Arduino 主要是为业余爱好者的使用而设计的，所以它被设计成一个小计算机的形式，它使你可以给连接到 Arduino 开发板上的外部输入输出器件编程（见图 1-1）。Arduino 就是所谓的嵌入式计算机平台，这意味着它是一个交互式系统，可以通过硬件和软件与它的环境进行互动。例如，一个简单的 Arduino 应用项目是在按下一个按钮时，点亮一盏小灯并保持一段时间，比如说 30 秒。这个例子中必须有一盏小灯和一个按钮与 Arduino 相连。Arduino 可以一直处于等待按钮被按下的状态，一旦按钮被按下，Arduino 就点亮那盏小灯并开始计时，当它计时到 30 秒时，Arduino 熄灭小灯并且等待下一次按钮被按下，本书后续章节可以看到有关示例。

因为 Arduino 的种种优势，越来越多的专业硬件开发者已经或开始使用 Arduino 来开发他们的项目、产品；越来越多的软件开发者使用 Arduino 进入硬件、物联网等开发领域；大学里，自动化、软件，甚至艺术专业，也纷纷开展了 Arduino 相关课程。一言以蔽之，有了 Arduino，以前搞纯软件的程序员也可以鼓捣硬件了。下图是一款 Arduino Uno 的主板，还有更小规格

的，被称为 Arduino Nano,

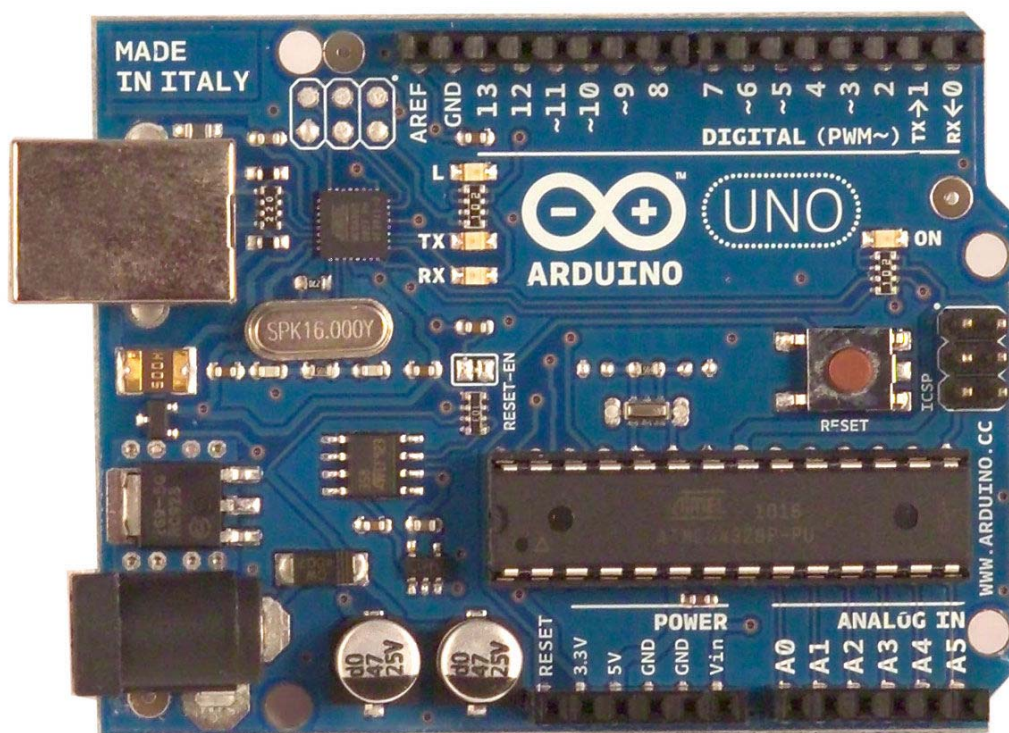


图 1-1 Arduino Uno

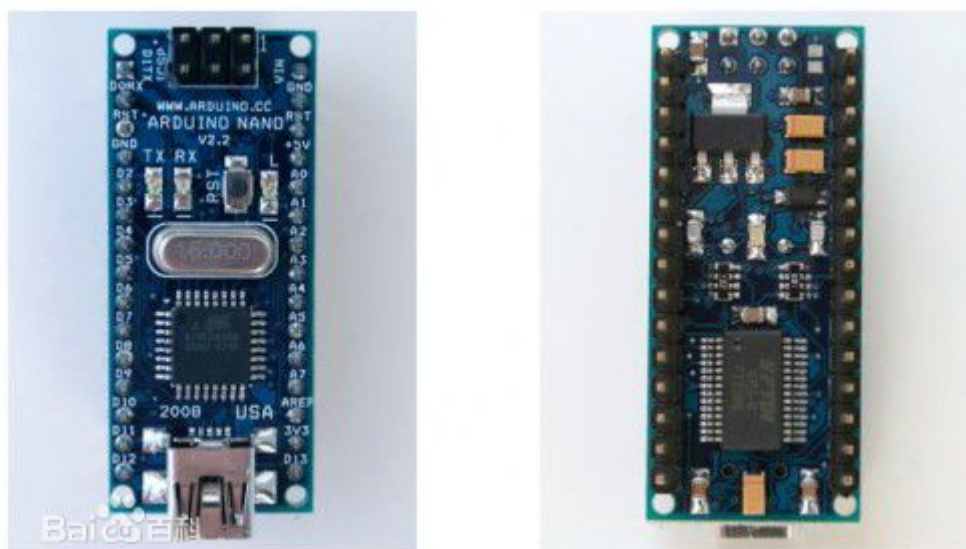


图 1-2 Arduino Nan

为了增强功能，还可以在此基础上增添更多的电子元件，比如各类传感或无线通讯原件，形成各类扩展版，给开发带来了极大的便利。

1.1.3 优点？缺点？

Arduino 最大的优势在于庞大的库资源，可以减少大量的在硬件上花费的精力。Arduino 包括各类单片机的定位就在于介于 PC 和一般的模拟电路之间的处理能力，很适合智能车、飞行器、机器人、智能家电的定位，因此在物联网开发领域是一个快速解决方案。但从另一方面来说，这未尝不是一种缺憾，至少极客们会认为这束缚了他们的想象力。此外，原生 Arduino 硬件资源的贫乏也限制了其应用的范围，如果附加太多的扩展板无疑增大了体积又拉高了成本，这在物联网普遍的受限使用环境和低成本要求的条件下是不可接受的。

1.2 ESP8266 是什么

ESP8266 由 Espressif（乐鑫信息科技）设计，凭借其高集成度、低功耗和易用性，成为 2016 年智能家居物联网里最为著名的一款芯片，获得了业界同行的肯定，并被新兴物联网领域所关注。内建 WiFi 的 ESP8266 微控制器我只谈论极为少数的新电路板，因为它们大部分达不到我的标准，也就是它们是否能让我完成原本做不到的事。不过偶尔还是会有让我兴致勃勃的新开发板上 市，而 ESP8266 就是其中之一。更有趣的是都会有 Maker 来找我讨论它们。这不是公司行销，而是社区的力量。ESP8266 是一款 UART 转 WiFi 的 SoC，根据 Tensilica Xtensa LX3 处理器设计，放在五花八门的扩展板上。其中最常见的就是安信可生产的 ESP12F，它拥有迷你的尺寸，而且价格可以低于 10 人民币。

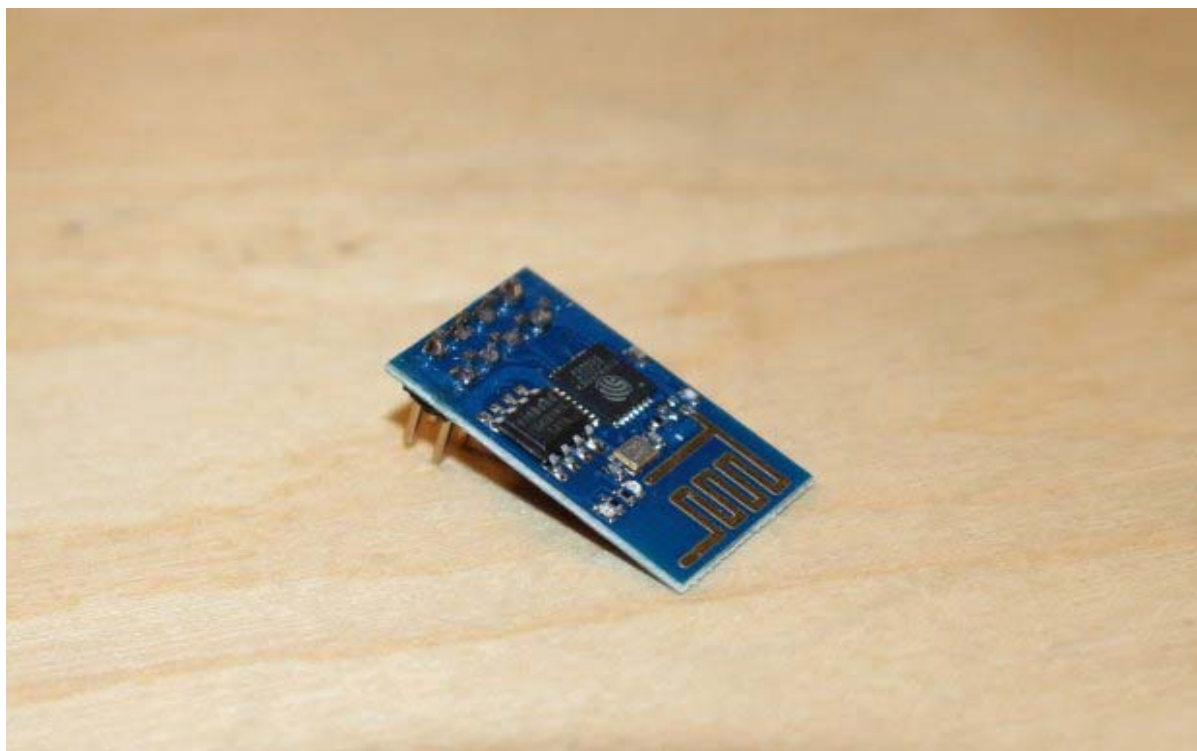


图 1-3 安装于 ESP-01 扩展板上的 ESP8266 SoC^[2]

这些开发板以及做为其基础的 ESP8266 SoC, 最初的市场定位是 Serial 转 WiFi 的转接器, 且拥有支援简单 AT 指令组的固件, 提供无线模块的设定和控制。

1.2.1 ESP8266 特点

之所以采用 ESP8266 做为研发平台, 主要是因为其超低成本, 并且将处理器和 WiFi 芯片集成在一起, 具有 GPIO、PWM、I2C、1-Wire、ADC 等功能, 为开发者提供一套极具性价比的开发平台。ESP8266 是一个完整且自成体系的 Wi-Fi 网络解决方案, 能够搭载软件应用, 也就是说可以将应用直接在 ESP8266 上运行, 或通过另一个应用处理器进行业务逻辑处理, 而仅使用 ESP8266 的 Wi-Fi 网络功能 (通常做为串口透传使用, 目前淘宝上销售的大部分已经写入基于 AT 指令的串口转 wifi 固件的模块, 方便直接连接原有单片机系统)。ESP8266 在搭载应用并作为设备中唯一的应用处理器时, 能够直接从外接闪存中启动。内置的高速缓冲存储器有利于提高系统性能, 并减少内存需求。另外一种情况是, 无线上网接入承担 Wi-Fi 适配器的任务时, 可以将其添加到任何基于微控制器的设计中, 连接简单易行, 只需通过 SPI/SDIO 接或中央处理器 AHB 桥接即可。ESP8266 强大的片上处理和存储能力, 使其可通过 GPIO 集成传感器及其他应用的特定设备, 实现了最低前期的开发和运行中最少地占用系统资源。ESP8266 高度片内集成, 包括天线开关 balun、电源管理转换器, 因此仅需极少的外部电路, 且包括前端模块在内的整个解决方案在设计时将所占 PCB 空间降到最低。装有 ESP8266 的系统表现出来的领先特征有: 节能 VoIP 在睡眠/唤醒模式之间的快速切换、配合低功率操作的自适应无线电偏置、前端信号的处理功能、故障排除和无线电系统共存特性为消除蜂窝 / 蓝牙/DDR/LVDS/LCD 干扰^[1]。其典型特征如下:

产品特性:

1. 802.11 b/g/n
2. Wi-Fi Direct (P2P)、soft-AP
3. 内置 TCP/IP 协议栈
4. 内置 TR 开关、balun、LNA、功率放大器和匹配网络
5. 内置 PLL、稳压器和电源管理组件
6. 802.11b 模式下 19.5dBm 的输出功率
7. 内置温度传感器
8. 支持天线分集

-
9. 断电泄露电流小于 10uA
 10. 内置低功耗 32 位 CPU: 可以兼作应用处理器
 11. SDIO 2.0、SPI、UART
 12. STBC、1x1 MIMO、2x1 MIMO
 13. A-MPDU 、A-MSDU 的聚合和 0.4ms 的保护间隔
 14. 2ms 之内唤醒、连接并传递数据包
 15. 待机状态消耗功率小于 1.0mW (DTIM3)

超低能耗技术专为移动设备、可穿戴电子产品和物联网应用设计，并与其他几项专利技术一起使机器实现最低能耗。这种节能的构造以三种模式运行：激活模式、睡眠模式和深度睡眠模式。使用高端电源管理技术和逻辑系统调低非必需功能的功率，调控睡眠模式与，处于连接状态时，其消耗的功率少于 1.0mW(DTIM=3)或 0.5mW(DTIM=10)。睡眠模式下，只有校准的实时时钟和 watchdog 处于工作状态。可以通过编程使实时时钟在特定的时间内唤醒 ESP8266。通过编程，ESP8266 会在检测到某种特定情况发生的时候自动唤醒。ESP8266 在最短时间内自动唤醒，这一特征可以应用到移动设备的 SOC 上，这样 SOC 在开启 Wi-Fi 之前均处于低功耗待机状态在近距离时可以通过软件编程减少 PA 的输出功率来降低整体功耗，以适应不同的应用方案。

1.2.2 为什么要用 ESP8266

ESP8266 正以迅雷不及掩耳的速度成为物联网的主要平台之一。其价格十分便宜且操作简单、十分容易取得（这类东西都是），这是和先前产品一个很大的不同。从我接触客户来看，大众选择 ESP8266 原因主要如下：

1. 芯片便宜，10 几元人民币（批量不到 10 元）的模块价格，和以前的 WiFi 几十元相比，极具性价比。
2. 除了能当做 WiFi 模块使用外，因为其内置了 32 位的处理器，可以进行二次开发。
3. 基本 ESP8266 的软件生态已经形成，其自身的 SDK 不停的迭代更新、基于 lua 的 NodeMCU 发展的也如火如荼，特别是 Arduino 生态移植，使 ESP8266 发展更加迅猛。
4. WiFi 有三种工作模式：Station 模式，AP 模式和 AP 兼 Station 模式，这对一些场合很重要。
5. 本身有 AT 固件，可以当串口转 WiFi 使用。
6. 由深圳四博智联开发的具有 Web 配置功能的固件，进一步提升了使用便利性，详见 <http://bbs.doit.am>

1.2.3 ESP8266 使用场景

作为一款 WiFi 芯片，只要需要使用无线网络环境的都可以应用它。具体来讲，那些通过 WiFi 控制的各类设备，所谓的智能设备，如插座、开关、家具、家电，还有更多工业上的应用。因此，不怕你做不到，只怕你想不到。注意，这里使用智能一词而不是用智慧，是因为目前市场上所见的自称智慧的产品其实离智能都还差的远。本书给一个蹩脚的区分吧：所谓智能设备，是可以实现自动控制，并可按需更新的软硬件产品。而智慧设备，是具有主动性和主体意识的生态系统，它可以是一个设备，也可以是一组设备的集合，是智能设备的高级阶段。本书所涉内容都是立足于智能设备来讲。ESP8266 的典型应用有：智能电源插头、家庭自动化、网状网络、工业无线控制、婴儿监控器、网络摄像机、传感器网络、可穿戴电子产品、无线位置感知设备安全 ID 标签、线定位系统信号。

1.3 什么是 ESPDuino

ESPDuino 是深圳四博智联科技有限公司公司出品的多功能物联网开发板，它综合了业界流行的 ESP8266 系列芯片的 WiFi 强大功能，采用更加稳定可靠的 ESP13 模块，又具有 Arduino 丰富的开发库，为发烧友和业界厂家提供了更加灵活、丰富的选择。简单地讲，一块 ESPDuino=Arduino UNO+WiFi。ESPDuino 也继承了 ARDUINO 开源硬件平台的主要特征，可以使用 ARDUINO 的相关开发资源，实现快速开发和应用。

1.3.1 强强联合就开挂

原生的 Arduino 是基于 ATMEGA328 芯片设计的，其 SRAM 只有 2K，限制了很多应用，而且除了接口丰富和友好，却没有更多的额外功能。因此 Doit 基于 esp8266 开发的 ESPDuino 兼容 arduino 的开发板，利用 ESP8266 本身自带 WiFi 的特性，相对于传统的 Arduino 开发，省去了买 WiFi 扩展板的费用。最重要的是扩充了内存容量可以达到 4M，而且集成了当下物联网发展不可或缺的 WiFi 功能，成为玩家和业界的新宠。详细资料可以在 Github 地址：<https://github.com/esp8266/Arduino> 上找到。

1.3.2 ESPDuino 庐山真面

1、外观

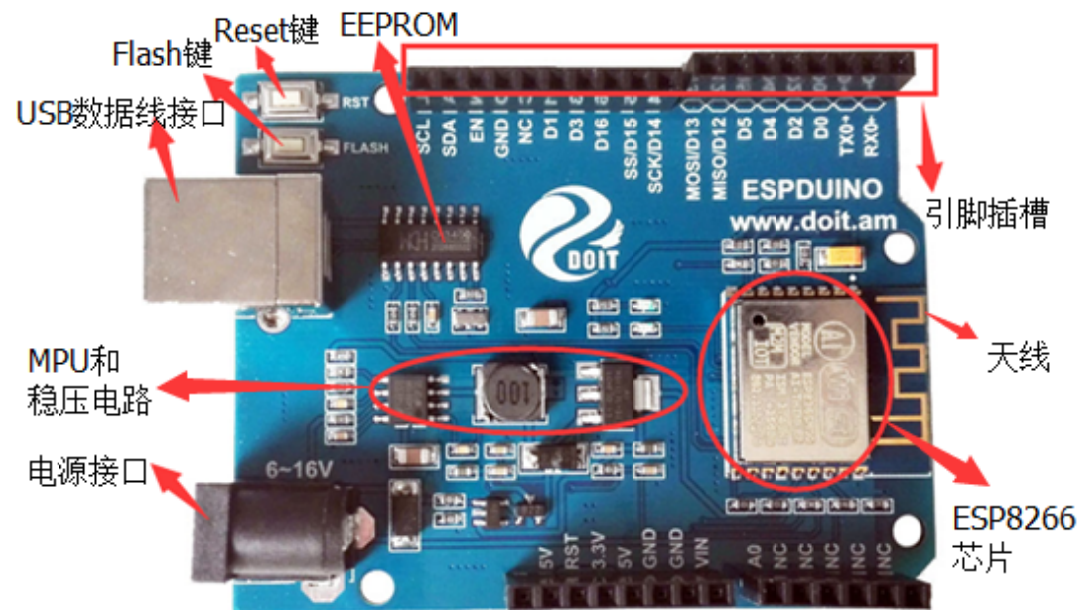
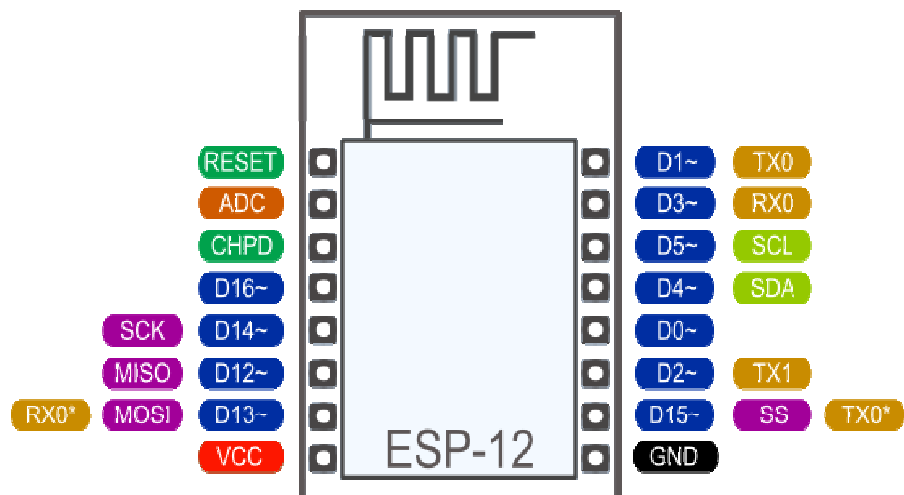


图 1-3 ESPDuino

2、引脚定义



3、电路

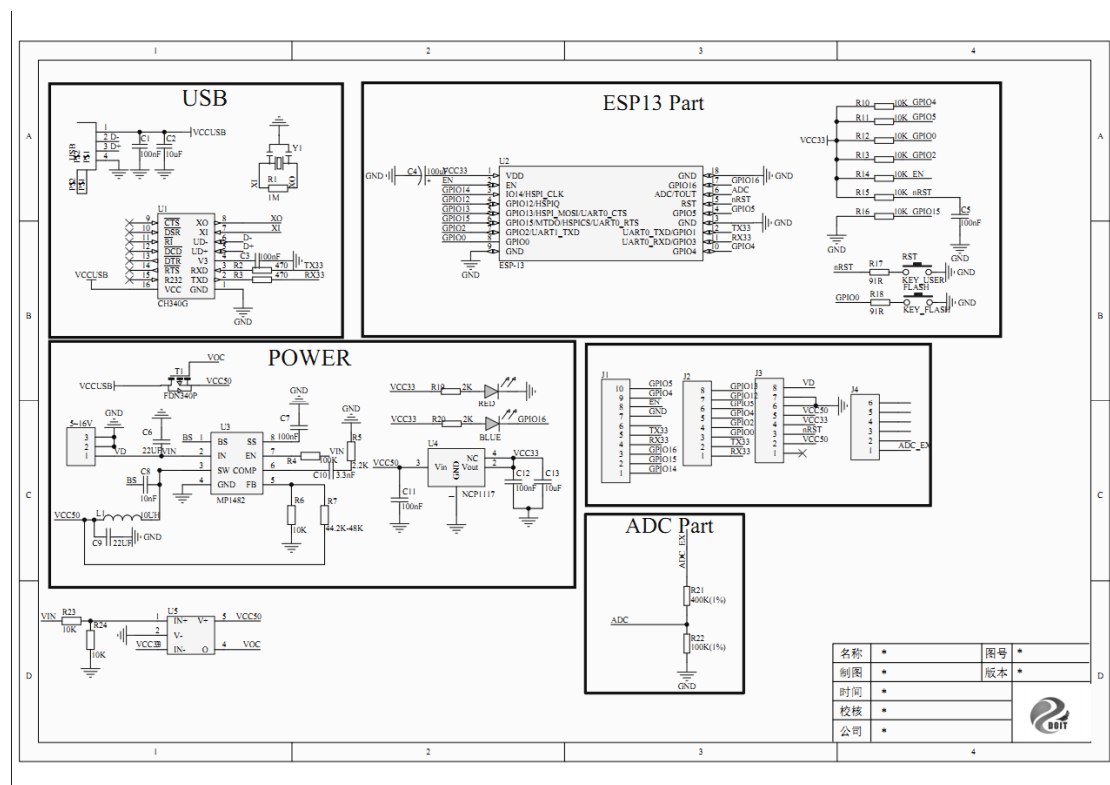


图 1-4 电路结构

1.3.3 学或者玩，Just Do It

ESPDuino 可以干什么呢，其实顾名思义，ESP8266 系列 WiFi 芯片和 Arduino 模组能干的事它都能干，诸如物联网数据采集、机器人控制、工业生产监控等。同时，对于一个小白来说，拿一块 ESPDuino 的开发板来从头学习几乎没什么门槛，无论基础如何，只要肯付出努力，就一定有收获。别的不再多说，开始干吧，JUST DO IT!

第 2 章 如何安装 ESPDuino 开发环境

2.1 欲善其事必利其器

你至少需要：

1. 硬件

一块 ESPDuino 的开发板，可以到以下地址购买：

<https://item.taobao.com/item.htm?id=525057040876>

2. 软件

Arduino IDE 开发环境；

2.1.1 安装 ESPDuino 开发环境

为了使大家快速上手，我们录制了相关的视频，建议你先看视频有个整体的了解，再看下面的文档，视频地址：

http://v.youku.com/v_show/id_XMTQ0MzgWODUwOA==.html?from=s1.8-1-1.2

下面的视频内容的文字描述：

1. Arduino 1.68+2.1 整包下载：

在下面链接中下载 Arduino 软件(实时更新最新版本)

<http://ESPDuino.doit.am/>，下载完成解压，**注意不要解压到中文目录**，建议解压下 D 盘根目录下。解压完成，直接打开 Arduino.exe，出现如下图所示界面：



图 2-1 Arduino 启动画面

2. 运行 Arduino IDE

安装上述链接的安装文件后，默认的安装环境是 ESPDuino。主要参数如下

- 开发板默认选择是 ESPDuino（ESP-13 Module）；
- Upload Using: “Serial”；
- CPU Frequency: “80MHZ”；
- Flash size: “4M（3M SPIFFS）”；
- 端口 “COM7”。

温馨提示

在上述提供的链接中下载的 Arduino IDE 安装包，上面这些参数设置是固定的，除了端口。端口是根据个人电脑不同，端口也不一样。具体查找端口的方法如下。选择右键选择“计算机”→“属性”，选择“设备管理器”，可以查到此台电脑的端口号为“COM7”。如果是下载其他的安装软件。可以参考第 4 节的内容进行安装配置。



图 2-2 串行端口选择

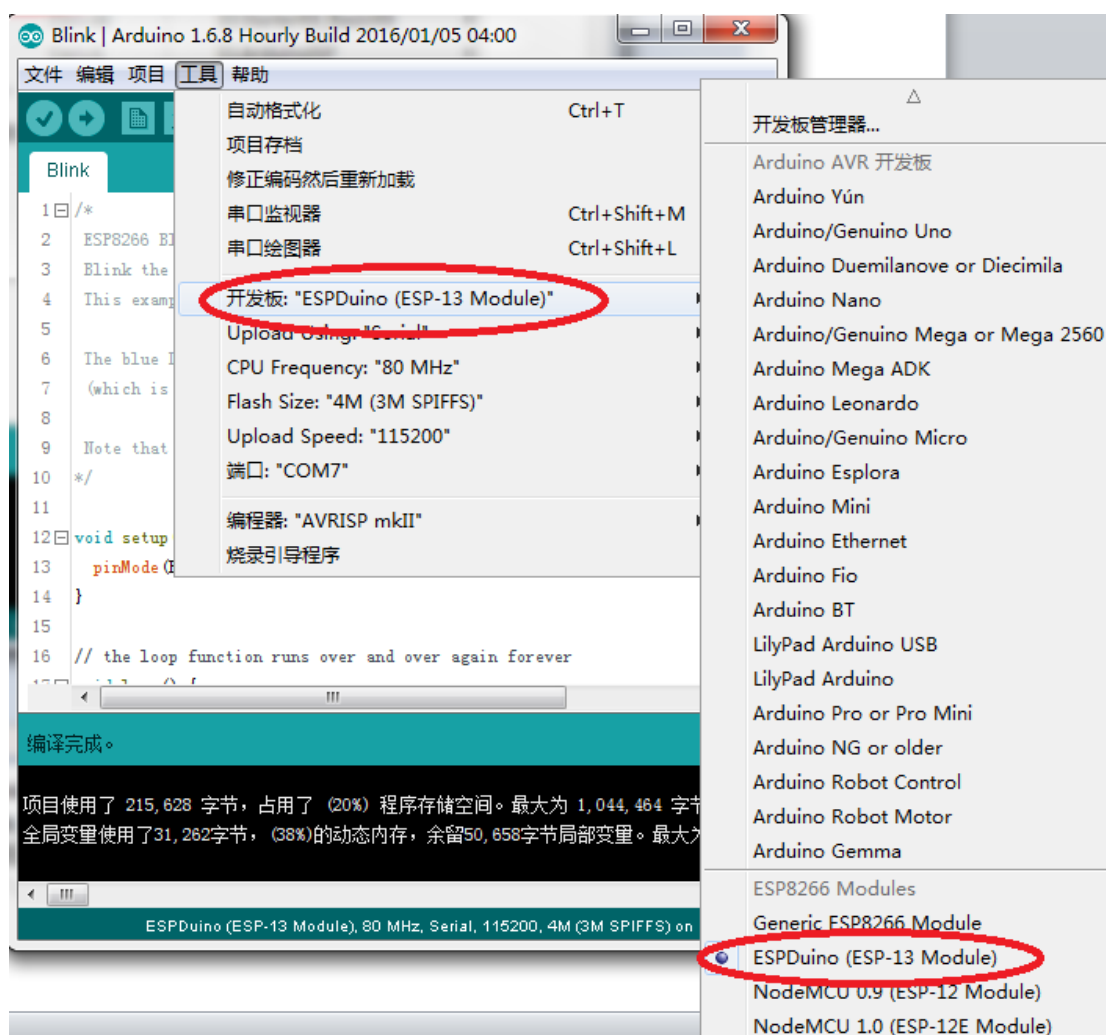


图 2-3 选择对应的硬件开发板

3. 测试 Arduino IDE

在第二节中已经将 ESPduino 的开发环境基本搭建确认好后，接下来需要用 Arduino 自动的实例程序来测试一下 ESPduino 的开发环境。

1) 选用 ESP8266 自带的 Blink 示例

依次点击“文件”→“示例”→“ESP8266”→“Blink”，如下图所示。

注意，不要选择“Basics”里面的“Blink”，因为这个是 Arduino 自带的示例，不是 ESPduino 的自带示例程序。如果一定要选择“Basics”里面的“Blink”，请将其端口号设置为“16”。

ESPduino 默认的 LED 灯端口号为“16”。

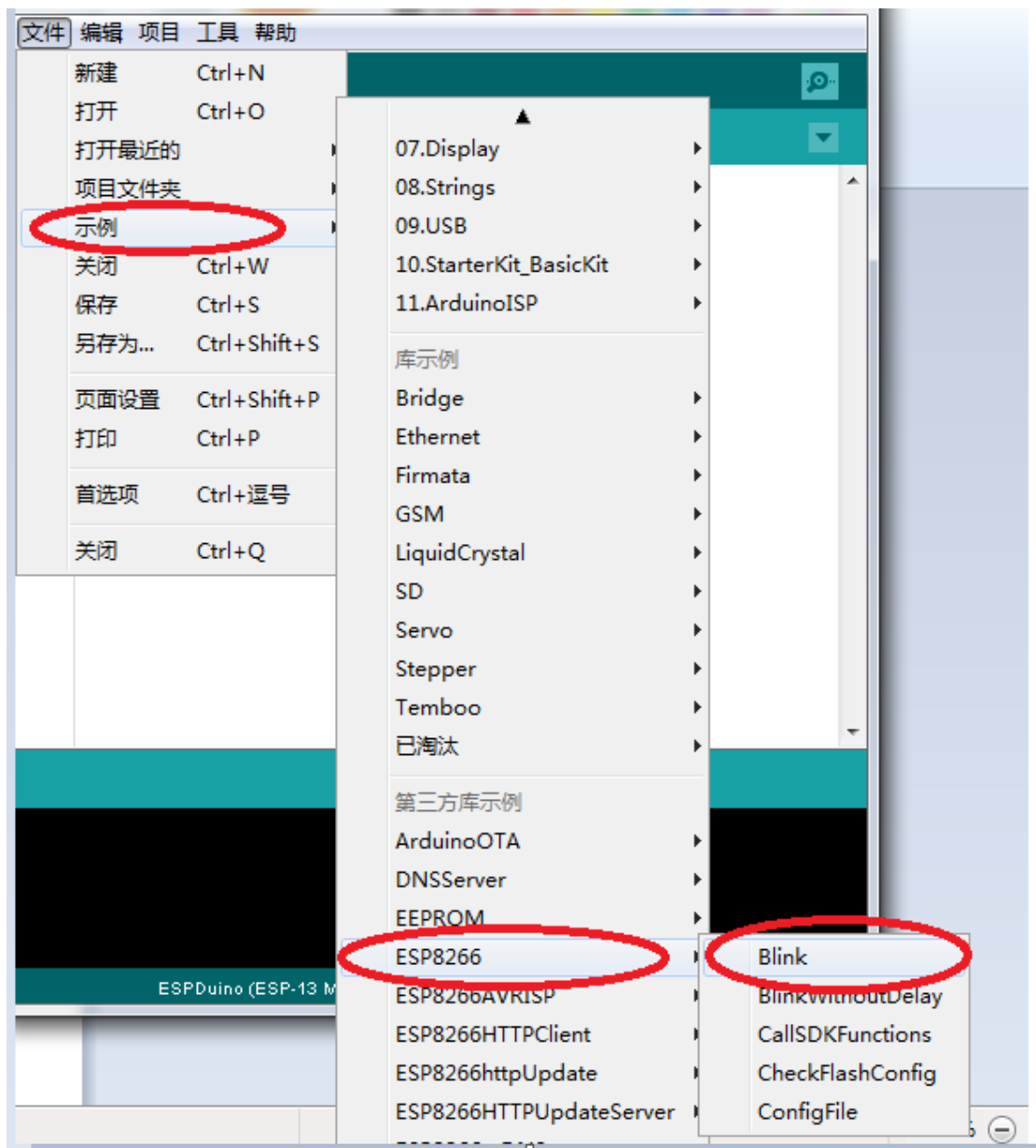


图 2-4 选择示例程序

2) 编译 Blink

打开 ESP8266 自带的 Blink 程序后，先点击编译按钮，如下图所示。如果按照第 2 节中的参数设置成功后，编译一定成功。

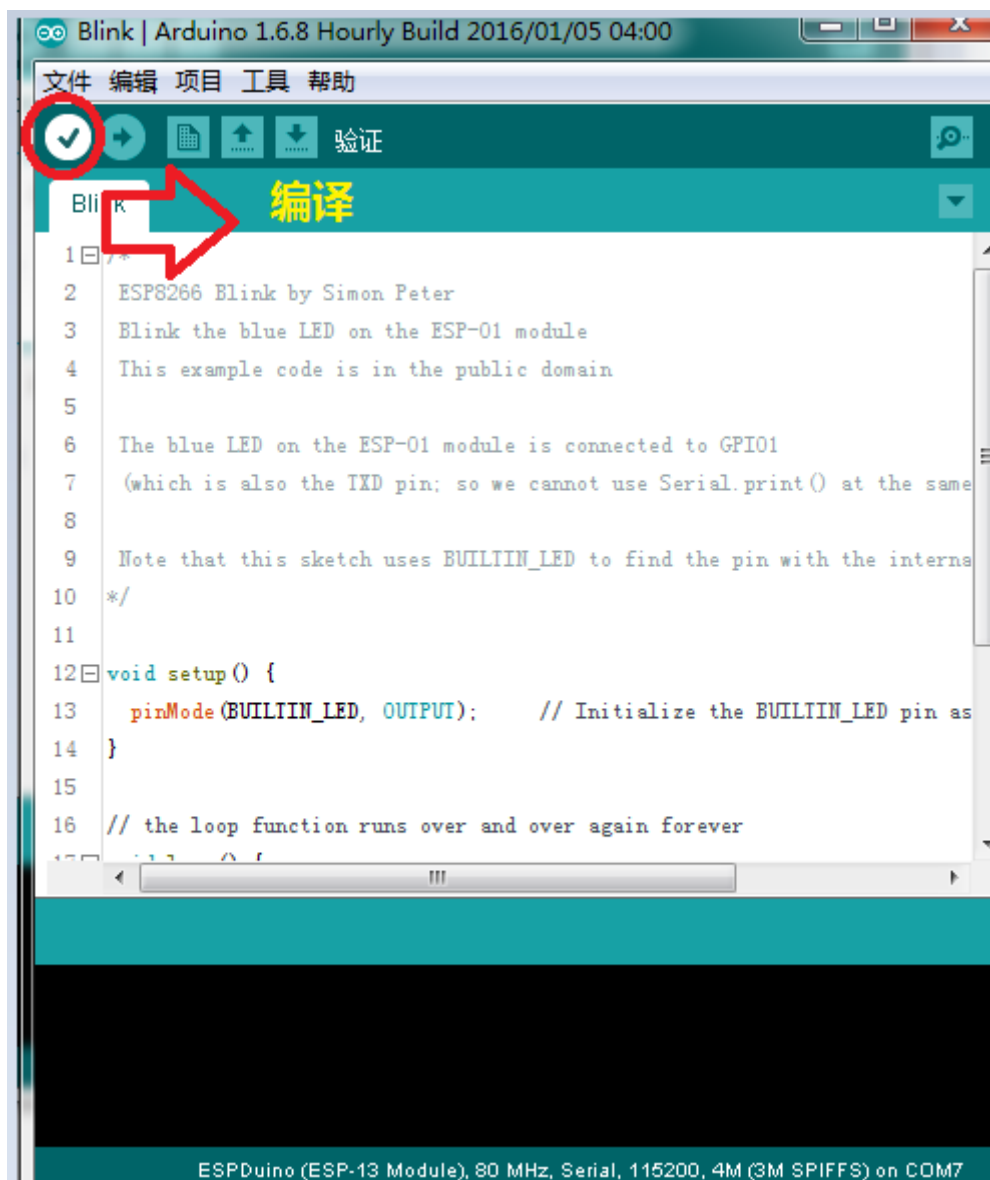


图 2-5 编译固件

3) 烧录程序

在烧录程序前，先认识 ESPduino 开发板上的 2 个按键：Flash 按键和 RST 按键。如图所示。

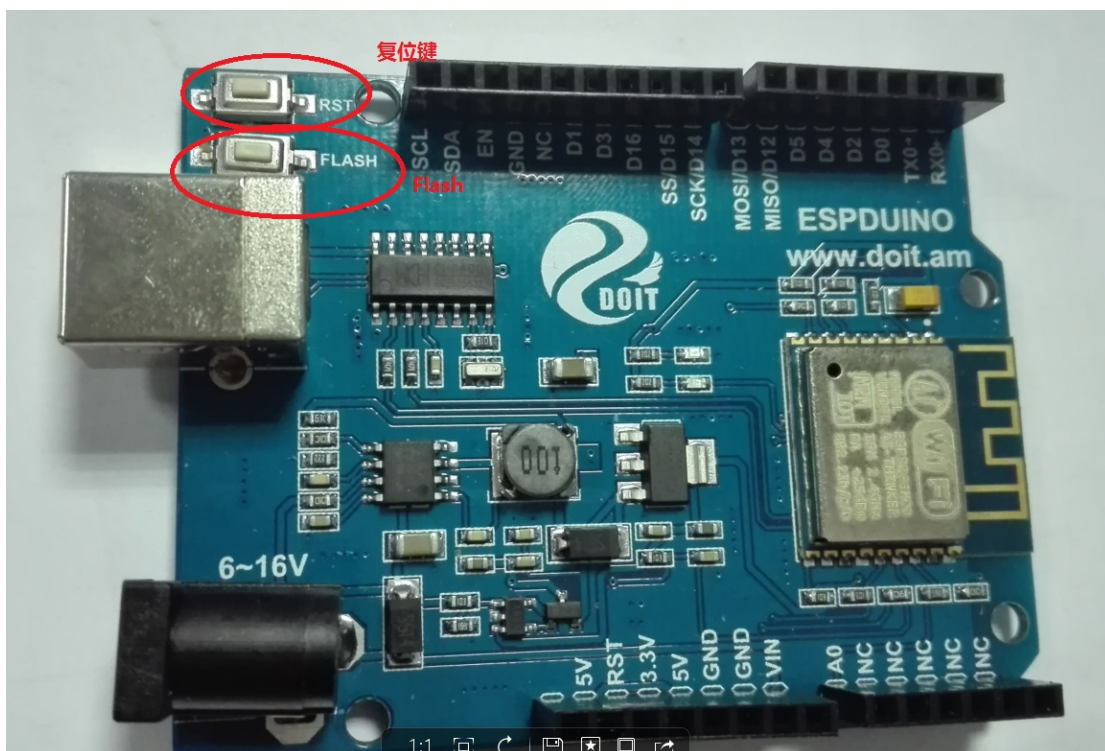


图 2-6 开发板按键位置

在将 Blink 程序烧录到 ESPDuino 时，需要在 ESPDuino 板子上电或复位时，将 Flash 置于低电平，使 ESPDuino 开发板进入烧写模式。所以，在烧录程序时，可以有以下 2 种方法进入程序烧写模式：

- i. 在将 USB 插入 ESPDuino 开发板上电前，按住 ESPDuino 开发板上的 Flash，然后再上电，上电后约 6s 钟，即可使 ESPDuino 开发板进入烧写模式；
- ii. 用 USB 连接电脑和 ESPDuino 开发板正常的情况下，先按住 Flash 键不放手，然后再按住 RST 键，然后松开，也可让 ESPDuino 进入烧写模式。（推荐使用）

实际上，上述两种方式原理都一样。都是使得 ESPDuino 开发板在上电时进入烧写模式，其原理如下图所示。

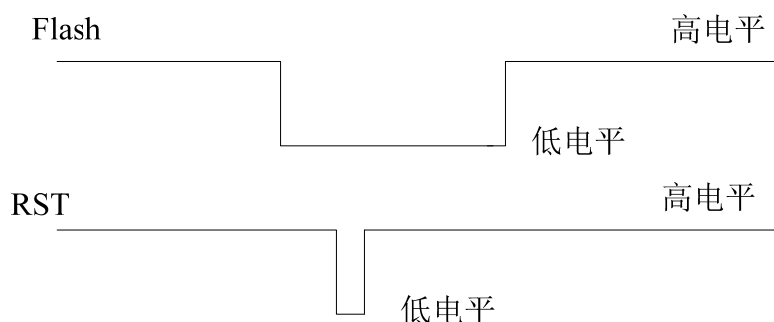


图 2-7 烧写模式原理

4) 烧录程序成功后, ESPDuino 开发板上的 LED 灯将每隔 1s 闪烁, 说明 ESPDuino IDE 搭建成功。

2.1.2 搭建 ESPDuino 环境第二种方法

1. 下载 Arduino:

官方版下载地址: <http://www.arduino.cc/en/main/software>

2. 安装运行 Arduino

1) 打开“文件”-》“首选项”

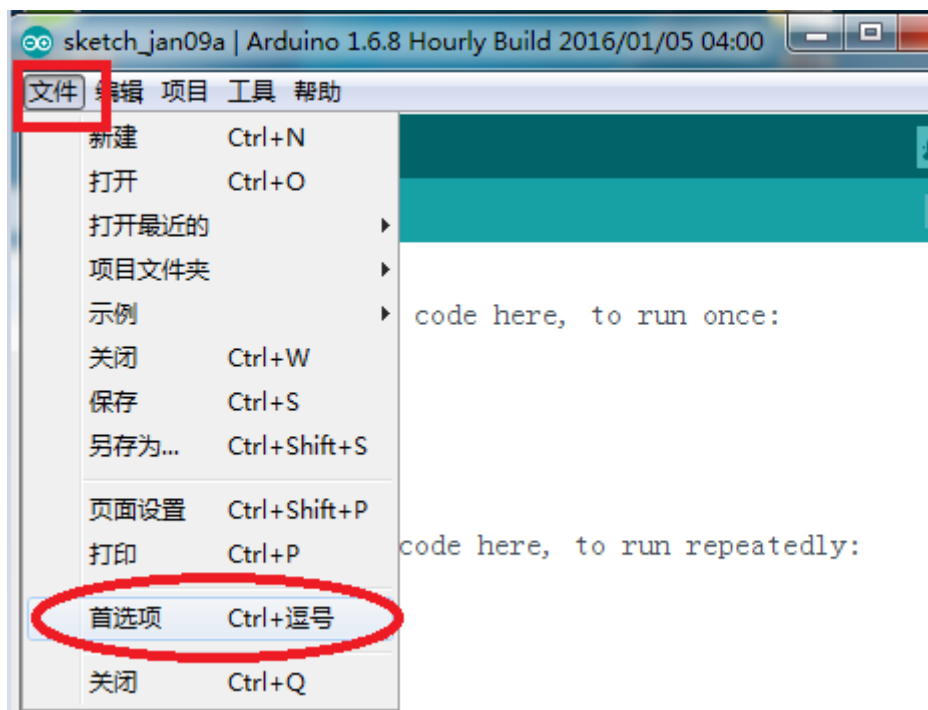


图 2-8 配置运行环境菜单

2) 在弹出的界面中输入地址: http://ESPDuino.doit.am/package_esp8266com_index.json, 这是 Doit 专为 ESPDuino 配置的 ESP8266 开发资源:



图 2-9 首选项参数配置

当然您也可以用官网的源安装，官网的详细步骤如下见 <https://github.com/esp8266/Arduino>。

注意，在安装过程，有可能不能访问，这时请设置一个能访问的代理。

3) 点击确认后，转入第 4 步，工具->开发板

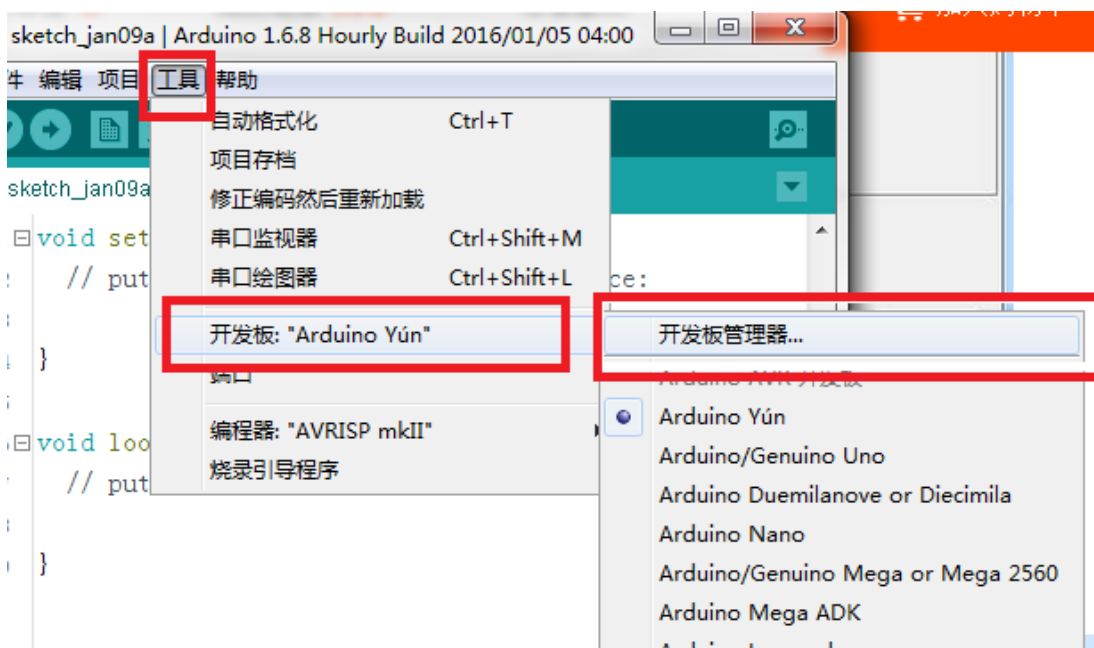


图 2-10 开发板管理菜单

- 4) 搜索 ESP8266, 随便点击空白处, 即可弹出“安装”按钮。点击“安装”按钮即可完成 ESPDuino 的安装。



图 2-11 开发包在线安装

到此, 就完成了支持 ESP8266 的 Arduino 开发环境的安装。

2.1.3 要点及注意事项

- 安装指定链接下载的 Arduino 软件, 可以直接使用, 选择示例时, 请选择 ESP8266 中的示例;
- 在下载代码到 ESPDuino 开发板时, 需要先将 ESPDuino 开发板置于烧写模式: 先按住 ESPDuino 板子上的 FLASH 键不放手, 然后再按住 RST 键, 约 6s 后松手, 即可进入烧写模式;
- 在已有 Arduino 系统上使用 ESPDuino 开发板, 需要将网址 http://ESPDuino.doit.am/package_esp8266com_index.json 输入初始的首选项环境后再安装;
- 以上所有的安装方法适用于基于 ESP8266 的 Arduino 开发板。

2.1.4 常见问题

- 问: 下载, 解压完毕后, 编译出错
答: 请不要解压到中文目录。
- 问: 在步骤 1 完成后, 仍然出错
答: 在 配置 菜单, 选 显示详细把“编译”选中, 具体分析问题原因。
- 问: 如果在设备管理里面, 显示黄色!,
答: 请下载安装如下驱动: <http://en.doit.am/CH341SER.zip>

更多可见：<http://bbs.doit.am/forum.php?mod=viewthread&tid=234>

2.2 样例程序

例子代码可以从如下地址下载：

<https://github.com/SmartArduino/ESPDuino>

2.2.1 程序功能

本示例程序的作用是将 ESPDuino 通过 WiFi 连接到路由器，并获取时间，步骤如下：

1. 在安装好环境之后，板子选择 “ESPDuino(ESP13-module)”

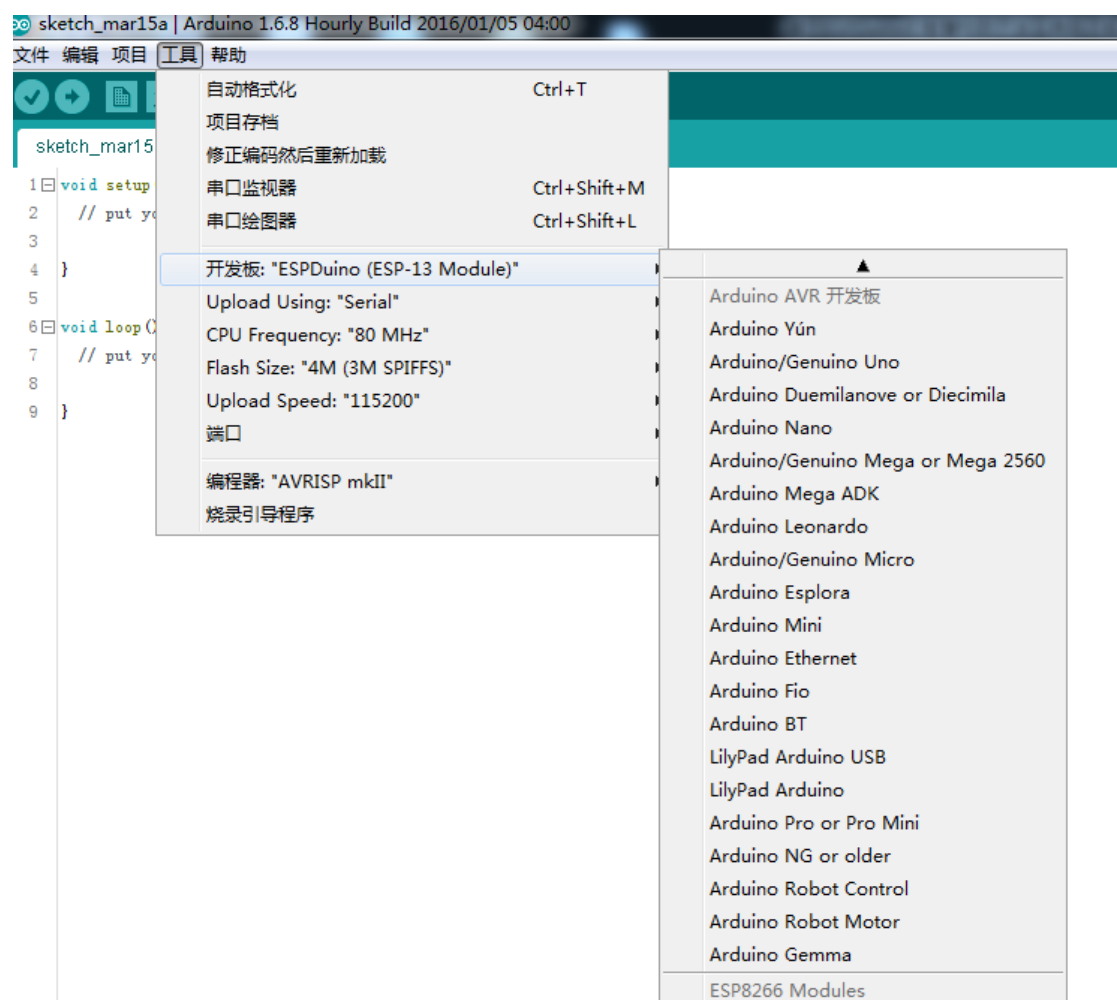


图 2-12 选择开发板

2. 编写示例代码，可以从

<http://bbs.doit.am/> 论坛 ESPDuino 讨论区下载，或是从 GitHub 直接下载，GitHub 下

载地址：<https://github.com/SmartArduino/ESPDuino>。

3. 通常，在嵌入式开发中，有上位机和下位机的概念，比如，PC 机串口连接了一块开发板，那么 PC 机就是相对开发板的下位机，开发板就是相对 PC 的上位机。因此，在 PC 上调试好固件程序上传到开发板的过程我们称之为上传（Upload），从固件中读取固件代码到 PC 称为下载（Download）。由于上传需要固件具备可写状态，因此，往往通过特殊物理操作，将硬件从正常工作模式转换为上传模式。对于 ESPDuino 来说，我们可以先按住"Flash"键，然后按"Rest"键，然后松开"Reset",最后松开 "Flash",此时板子就进入了下载模式。

4. 点“项目”->“上传”，即可更新固件。

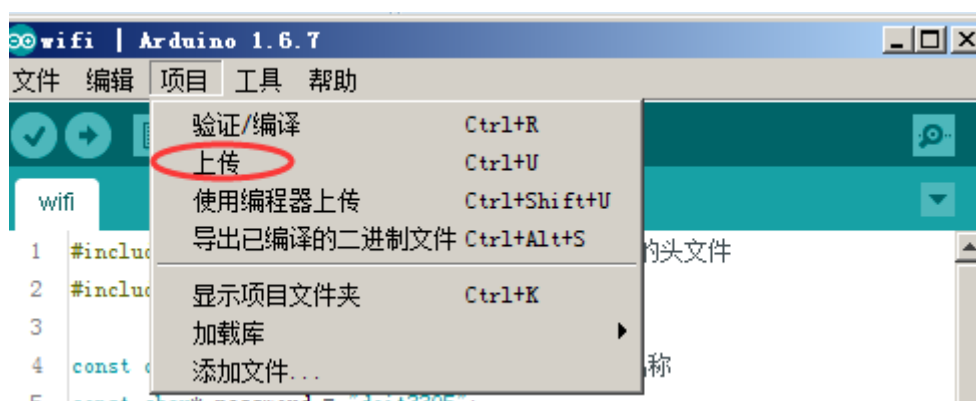


图 2-13 菜单上传

或者直接在快捷栏里点上传的快捷键图标：



图 2-14 快捷键上传

5. 点击上传后，会先自动编译：



图 2-15 编译

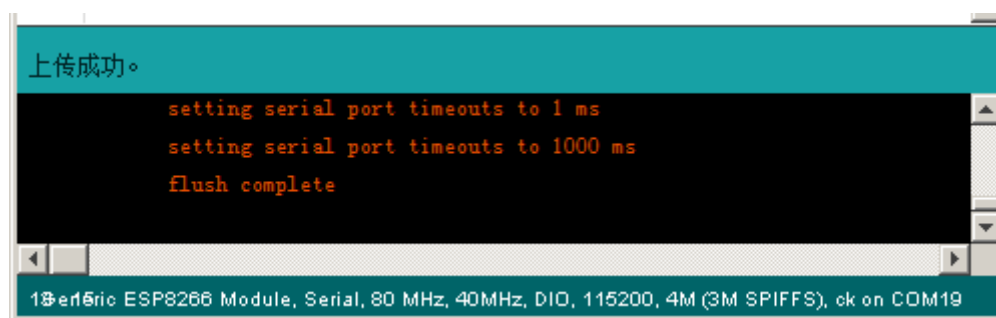


图 2-16 上传成功

温馨提示

如果编译时出现了错误，一般都是语法和未定义之类的错误，这时要再审查一下代码保证没有语法错误，未定义的情况主要是一些函数的使用，需要在程序开头包含相应的头文件，这是和通常的 C/C++ 编译环境相同。上传时，如果出现了下面这种错误：

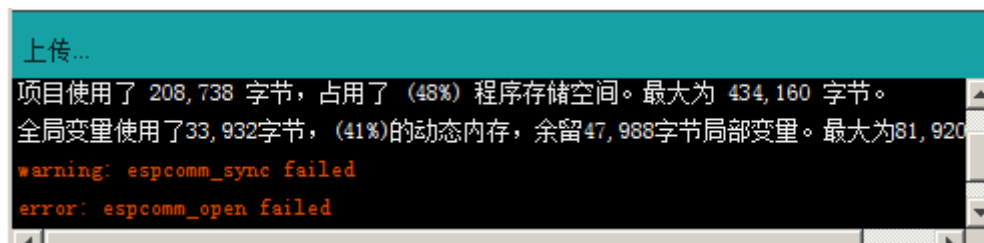
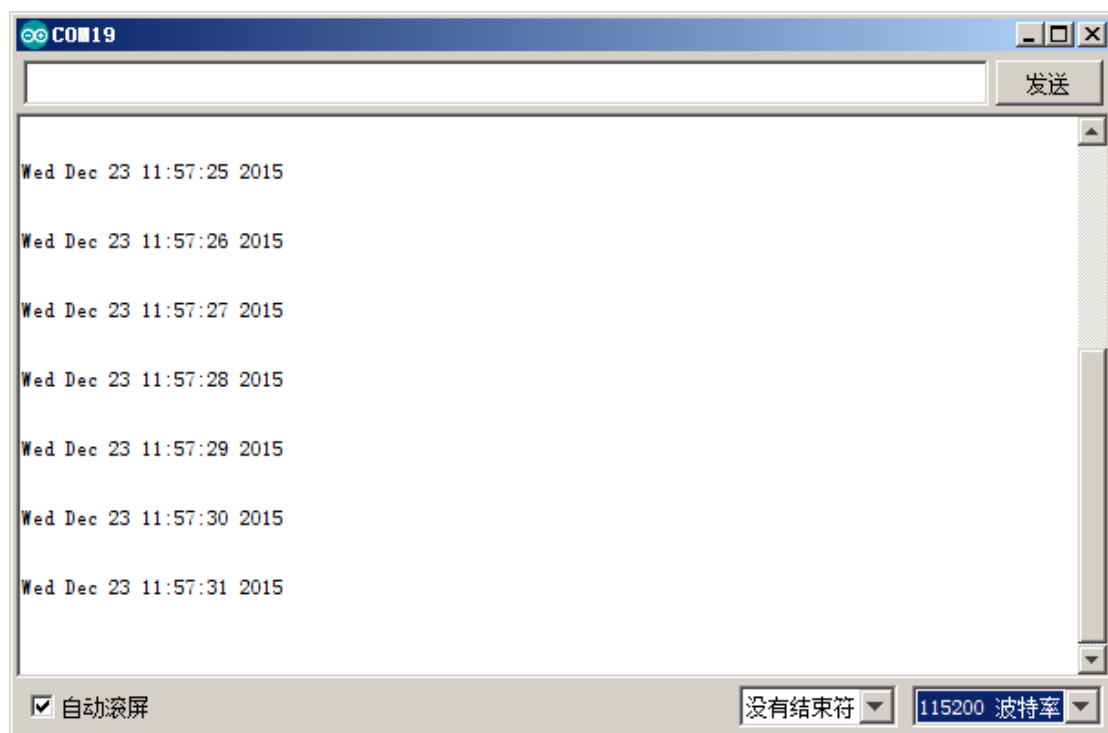


图 2-17 编译错误

一般是没有正确地把开发板置为烧写模式，最好重新设定一下。或者看看连接线是否松掉，要不然就换个电脑上的 USB 接口试试，注意串口（COM）号的重新选择。

运行后，从串口监视器可以看出，服务器的时间每秒钟响应一次，表示硬件工作正常。



2.3 源码分析

ESPduino 兼容 Arduino 的程序，也就是类 C/C++，它还支持面向对象的编程。首先要注意最重要的 `setup()`、`loop()` 两个函数，这是 ESPduino 初始化和运行所需的关键函数，任何 Arduino 项目的程序中都要有且仅有一组 `setup()` 和 `loop()`，作为程序的唯一入口点，是不是有点像 C/C++ 的 `main`？同普通的 C/C++ 语言一样，在程序开头，可以使用 `include`，`define` 等关键字来包含各种库文件、头文件以及自定义一些变量、函数等功能模块。2.2 中的例子程序很简单，主要作用就是从时间服务器取时间，可以参见代码的注释说明，在以后的章节中，相关重复功能的代码说明就不再一一介绍了。

```
#include <ESP8266WiFi.h> //这是支持Wifi基本功能的头文件
#include <time.h> //时间处理功能的头文件
const char* ssid = "Doit"; //所接入的无线网络名称
const char* password = "doit3305";
//访问密码，这两个信息在后面章节会频繁用到
int timezone = 3;
int dst = 0;
//ESPDUINO 开发版的初始函数
void setup() {
    Serial.begin(115200); //设定串口速率为 115200B/s
    Serial.setDebugOutput(true); //将串口设为调试输出模式
```

```

WiFi.mode(WIFI_STA); //将此 Wifi 芯片设为站点模式
WiFi.begin(ssid, password); //接入到无线环境热点
Serial.println("\nConnecting to WiFi");

//如果 Wifi 状态不是 WL_CONNECTED, 则表示连接失败
while (WiFi.status() != WL_CONNECTED) {
    Serial.print("."); //在串口窗口输出., 表示连接进度信息
    delay(1000); //延时 1 秒
}
//设置时间格式以及时间服务器的 url 网址
configTime(3 * 3600, 0, "pool.ntp.org", "time.nist.gov");
//串口信息输出
Serial.println("\nWaiting for time");
while (!time(nullptr)) {
    Serial.print(".");
    delay(1000);
}
Serial.println("");
}
//进入事件主循环
void loop() {
    time_t now = time(nullptr); //获取当前时间
    Serial.println(ctime(&now)); //打印并换行
    delay(1000); //延迟一秒 (间隔)
}

```

常见问题:

- 问: 时间和中国区的不对。

答: 中国在东 8 区, 代码中的时间服务器是记录的格林威治时间, 可自行换算; 当然也可以自己写一个时间服务器进行处理。

第3章 键控 LED 灯，闪闪告诉你

3.1 引言

前一章的例子对初学者来说可能有点 Hold 不住，没关系，本章开始就使用更为简单的例子循序渐进地带你进入 ESPDuino 的世界。在很多需要自动控制的场合或者物联网使用环境，都需要设备或操作的对象通过各种媒介反馈出设备运行的状态，比如声、光、行为、甚至是不同的气味，下面，就介绍通过编程控制 ESPDuino 版载 LED 灯状态的方法，前方低能，因为程序确实很简单。

3.2 物料准备

3.2.1 硬件



图 3-1 所需硬件一览

巧妇也难为无米之炊，因此以下基本资源还是需要的：ESPDuino 开发板：usb 线，PC 或笔记本电脑，这也是以后章节会最常使用的三个装备，若有其他硬件会另外说明。

3.2.2 软件

不用说，当然是前一章介绍过的 Arduino IDE 开发环境，本书中使用的是 1.6.8 版，不同的版本使用方法大体相当。

3.3 三个不同的闪灯示例

3.3.1 自动闪烁

1. ESPDuino 启动后，除了电源指示灯（红色）常亮外，旁边的蓝色 LED 会周期性（间隔半秒）自动闪烁，
2. 源码分析(标签名称 blinklight 右边的\$，在点击保存后会消失)

```
#define PIN_LED 16 //定义 LED 灯：PIN_LED 所对应的引脚为 16 号

void setup() {
    Serial.begin(115200);
    Serial.setDebugOutput(true);
    pinMode(PIN_LED, OUTPUT); //初始化 PIN_LED 引脚模式为输出
    pinMode(KEY_FLASH, INPUT); //初始化 Flash 按键模式为输入
    digitalWrite(PIN_LED, HIGH); //初始 LED 灯置为低电平，表示点亮
    delay(1000); //延时 1 秒
}

void loop() {
    digitalWrite(PIN_LED, LOW);
    delay(500);
    digitalWrite(PIN_LED, HIGH);
}
```

3.3.2 按键控制

那么，能否手工控制 LED 灯呢，当然可以，继续修改 3.3.1 的源码即可。

1. 手工按下 flash 键，板载 led 灯亮，松开，灭。

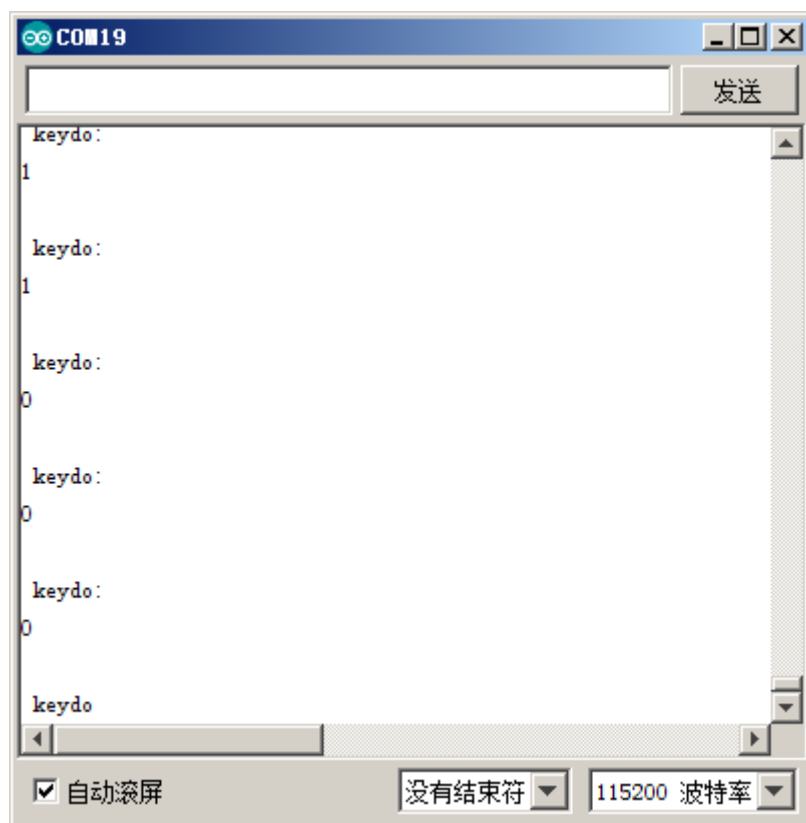
2. 源码分析

```
#define PIN_LED 16 //定义 LED 灯: PIN_LED 所对应的引脚为 16 号
#define KEY_FLASH 0 //定义 KEY_FLASH 对应 0 号引脚, 其实就是 FLASH 按键位置

void setup() {
    Serial.begin(115200);
    Serial.setDebugOutput(true);
    pinMode(PIN_LED, OUTPUT); //初始化 PIN_LED 引脚模式为输出
    pinMode(KEY_FLASH, INPUT); //初始化 Flash 按键模式为输入
    digitalWrite(PIN_LED, HIGH); //初始 LED 灯置为高电平, 表示点亮
    delay(1000); //延时 1 秒
}
//自定义按键函数
void keydo(){
    //读出当前按键状态, 0 表示抬起断开, 1 为按下接通
    int k;
    k=digitalRead(KEY_FLASH);
    Serial.println("\r\n keydo:");
    Serial.write(k);
    Serial.println();
    if(k==0)
        digitalWrite(PIN_LED, HIGH); //高电平表示灯灭
    else
        digitalWrite(PIN_LED, LOW);
}

void loop() {
    //digitalWrite(PIN_LED, LOW);
    delay(500);
    keydo();
    //digitalWrite(PIN_LED, HIGH);
    delay(500);
}
```

观察串口监视器, 可以看到没有按键时, keydo 输出是 1, 灯灭, 按下键后返回 0, 灯亮。



3.3.3 中断方式

在先前的例子中，通过自定义的 `keydo()` 函数可以处理按键操作，使硬件响应。在程序模块不多，`delay()` 函数很少的情况下也能基本满足需求。但实际应用需求会比较复杂，`delay()` 函数比比皆是，如果单用函数调用的方式处理按键事件，将不能及时地得到响应，这样，硬件的行为就不可预期，产生一些意外结果。因此，这里介绍中断方式的编程技巧，使得按键事件发生时，硬件可以随时响应。我们把按键这类事件一般叫做外部中断源，与之对应的，MCU 中如果时间片完成、寄存器溢出等来自于 MCU/CPU 的事件叫做内部中断源，内部中断比外部中断的优先级（硬件响应事件的次序）要高。本书只介绍外部中断事件的编程，一般过程如下：

1. `setup()` 函数里调用 `attachInterrupt(外部中断源名称, 中断函数名称, 触发事件名称)`;
2. 自定义中断函数名称() 及函数体;

源码如下：

```
#define PIN_LED 16
#define KEY_FLASH 0
volatile int state = LOW; // 定义默认输入状态
void setup() {
```

```
pinMode(PIN_LED, OUTPUT);
pinMode(KEY_FLASH, INPUT);
digitalWrite(PIN_LED, LOW); //初始 LED 灯置为低电平，表示点亮
// 监视中断输入按键的变化，此函数为中断函数入口句柄
attachInterrupt(KEY_FLASH, stateChange, CHANGE);
}
//中断函数
void stateChange(){
    //LED 灯状态翻转，实现发生外部按键事件时对 LED 灯状态的改变
    state=!state;
    digitalWrite(PIN_LED, state);
}
void loop() {
    //delay(1000);
    //digitalWrite(PIN_LED, HIGH);
    //delay(1000);
    //digitalWrite(PIN_LED, LOW);
}
```

按键可以随时控制 LED 的亮、灭。

第 4 章 PWM，你也做得到

当需要连续控制电压变化，实现灯光的不同亮度或者电机的不同转速时，就要用到脉冲宽度调制（Pulse Width Modulation，PWM）的方式了。PWM 是一种模拟控制方式，是一种对模拟信号电平进行数字编码的方法。其根据相应载荷的变化来调制晶体管基极或 MOS 管栅极的偏置，来实现晶体管或 MOS 管导通时间的改变，从而实现开关稳压电源输出的改变。这种方式能使电源的输出电压在工作条件变化时保持恒定，是利用微处理器的数字信号对模拟电路进行控制的一种非常有效的技术。

4.1 PWM 的原理

PWM 通过高分辨率计数器的使用，方波的占空比被调制用来对一个具体模拟信号的电平进行编码。PWM 信号仍然是数字的，因为在给定的任何时刻，满幅值的直流供电要么完全有(ON)，要么完全无(OFF)。电压或电流源是以一种通(ON)或断(OFF)的重复脉冲序列被加到模拟负载上去的。通的时候即是直流供电被加到负载上的时候，断的时候即是供电被断开的时候。只要带宽足够，任何模拟值都可以使用 PWM 进行编码，一个典型应用就是对舵机转速的控制，后面的产品篇一节可用得上哟。

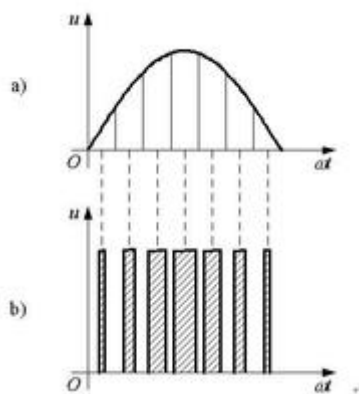


图 4-1 PWM 编码

4.2 实验效果

将 Arduino 程序编译上传后，可以看到 LED 灯有一个明暗的连续变化过程，说明我们已经成功控制了对 LED 灯电压的连续改变。

4.3 源码分析

```
#define PIN_LED 16

void setup() {
    pinMode(PIN_LED, OUTPUT);
    analogWrite(PIN_LED, 0); //模拟信号输出
}

void loop() {
    for (int i = 0; i < 1024; i++)
    { //电平升高, 从明到暗
        analogWrite(PIN_LED, i);
        delay(2);
    }
    for (int i = 1024; i >= 0; i--)
    { //电平降低, 从暗到明
        analogWrite(PIN_LED, i);
        delay(2);
    }
}
```

第 5 章 Soft AP—连上我就对了

5.1 什么是 Soft AP 模式

使用 ESP8266 主要是看中其 WiFi 功能，从本章开始，将进入 WiFi 的世界。在 WiFi 的常用术语当中，先重点说说常用 AP，AP 是 Access Point 简称，也就是访问接入点，是一个网络的创建者，是网络的中心节点。一般家庭或办公室使用的无线路由器就是一个 AP，基于 AP 组建的基础无线网络（Infra）也称为基础网，是由 AP 创建，众多站点（STA）加入所组成的无线网络，这种类型的网络的特点是 AP 是整个网络的中心，网络中所有的通信都通过 AP 来转发完成，如下图所示。

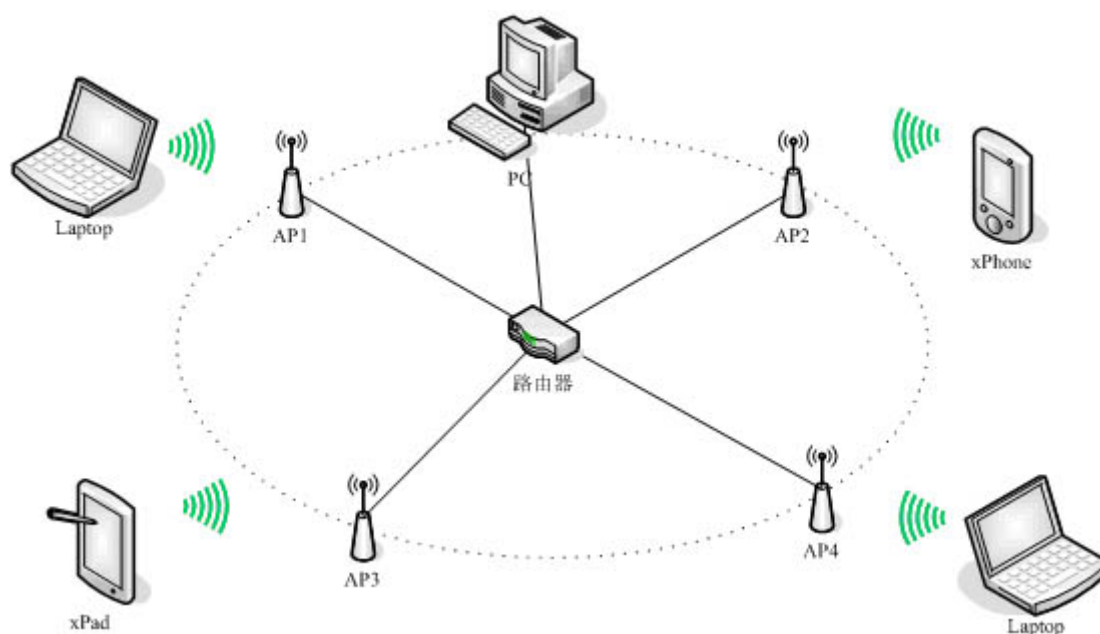


图 5-1 无线 AP 网络

在应用环境中，往往需要 WiFi 芯片提供无线服务接入的能力，主动向外界发送网络信号，以便外部终端可以搜索到此 AP 并请求连接，这时，它就起到了一个简易路由器的功能。此外，当需要 WiFi 芯片搭载到一些硬件设备上，使设备扮演服务器的角色，以便响应终端所发出的指令。

在无线网络中，AP 作为基站起着连接其它无线设备以及与有线骨干网桥接的作用，相当于有线网络中的 HUB 与交换机。软 AP 也称 Soft-AP，它的硬件部分就是一块标准的无线网卡比如 ESP8266，但其通过驱动程序使其提供与 AP 一样的信号转接、路由等功能。与传统 AP 相比，它的成本很低，功能上也能凑合。在基本功能上，Soft AP 与 AP 并没有太大的

差别，不过因为是其用软件的方式实现了 AP 的功能，Soft AP 的接入能力和覆盖范围远不如 AP。

5.2 实验效果

编译程序并上传固件后，通过串口监视器，我们可以看到，这个 WiFi 芯片的工作模式被成功设置为接入点（AP）模式，此 AP 的 IP 地址是 192.168.4.1，并提供了一个简单的 HTTP Server 服务。串口第一行显示的乱码是板子启动时产生的字节码信息，不用管它。

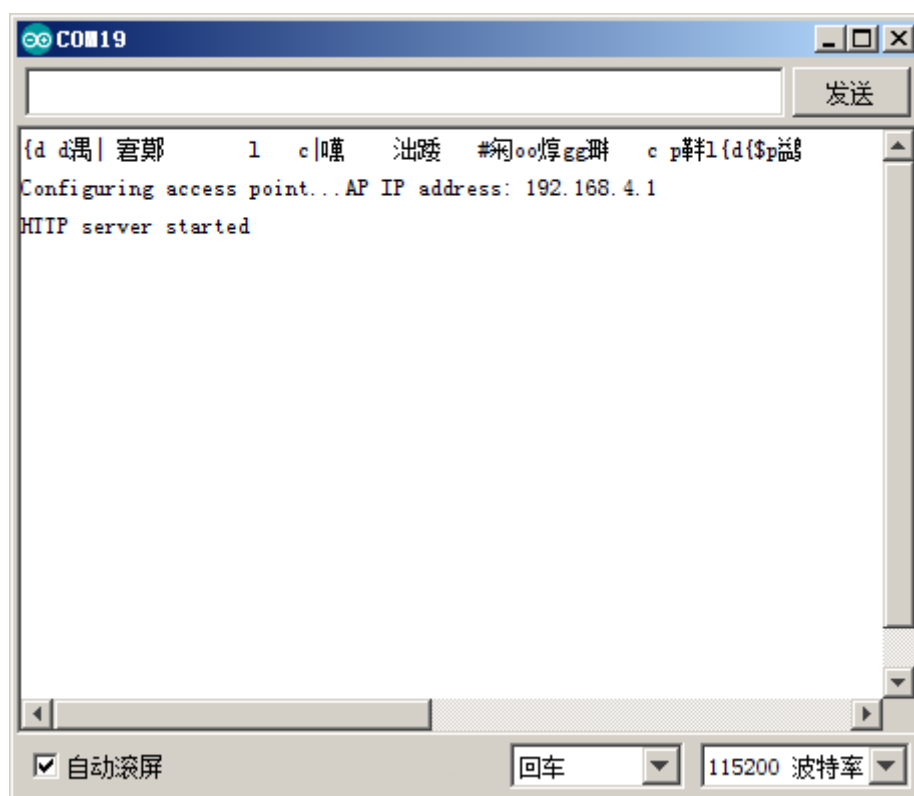


图 5-2 串口状态

从测试笔记本电脑的 Win7 状态栏也能发现有个新的热点 Charlie Testing AP，我们连接以后就可以和这个 AP 建立通信链路了。



图 5-3 连接自设 WiFi

在浏览器地址栏输入 AP 的 IP 地址，返回信息如下，则表示一切正常。

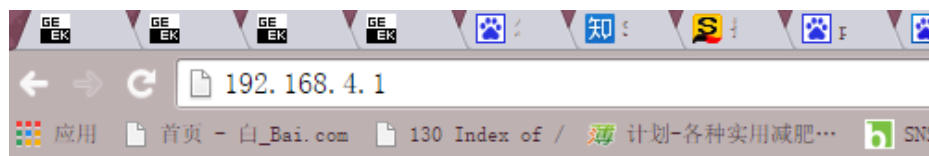


图 5-4 浏览器访问

以上这些信息当然是可以改变的，源码内容参见下一节。

5.3 源码分析

```
#include <ESP8266WiFi.h>
/* 建立 Wifi 热点并设置一个 Web 服务 */
#include <ESP8266WebServer.h>
/*自定义 AP 的 SSID 名称和接入密码 */
const char *ssid = "Charlie Testing AP";
const char *password = "11111111";
//声明 web 服务对象 server 和端口 80
```

```
ESP8266WebServer server(80);

/*在 web 浏览器中输入 http://192.168.4.1 来连接到 AP 的 web 模式，
这个地址是预先在头文件中定义好的，可以在其中修改 */
//设定 web 服务的成功响应信息
void handleRoot() {
    server.send(200, "text/html", "<h1>You are connected</h1>");
}
void setup() {
    delay(1000);
    Serial.begin(115200);
    Serial.println();
    Serial.print("Configuring access point...");
    /*如果使用开放式 AP 模式，可以把下面语句的 password 参数省略 */
    WiFi.softAP(ssid, password);
    //获取 AP 的 IP 地址，存入变量 myIP
    IPAddress myIP = WiFi.softAPIP();
    Serial.print("AP IP address: ");
    Serial.println(myIP);
    //调用 on 方法，给客户端发送响应信息，启动 web 服务，并在串口回显
    server.on("/", handleRoot);
    server.begin();
    Serial.println("HTTP server started");
}
//循环等待处理客户端的连接
void loop() {
    server.handleClient();
}
```

从上面的代码可以看出，Arduino 的网络编程如此简单，仅几行代码就实现了完整的 Web Server,要知道 Web Server 是基于 Http 的，而 Http 是基于 TCP 的，Http 是相对很复杂的文本式协议，其解析的工作量很大。在进阶篇中，我们将进一步对 Http 协议进行讲解。

第 6 章 STA 连接—我要连...

6.1 什么是 STA 模式

上一章介绍了 WiFi 芯片的 Soft AP 的工作模式，与之对应的，它还有一种 STA（站点）的工作模式：Station，类似于无线终端。STA 本身并不接受无线的接入，它可以连接到 AP。STA 是设备作为客户端连接到上级路由的 wifi 信号，客户端可以使用 dhcp client 的方式，由上级路由分配的 ip，或是设置成静态 ip。两种方式各有利弊，如果用 dhcp 的方式，获取的 ip 的是动态的，在一些需要知道设备 ip 才能通讯的场合，就需要其它的技术手段来获取 ip，一个常用的方法是 UDP 广播的方式来获取 ip。静态的方法虽然能固定 ip，但要求连接 AP 设置的网段和静态的要一致，在不能固定 AP 网段的情况下，这种方式不可取。

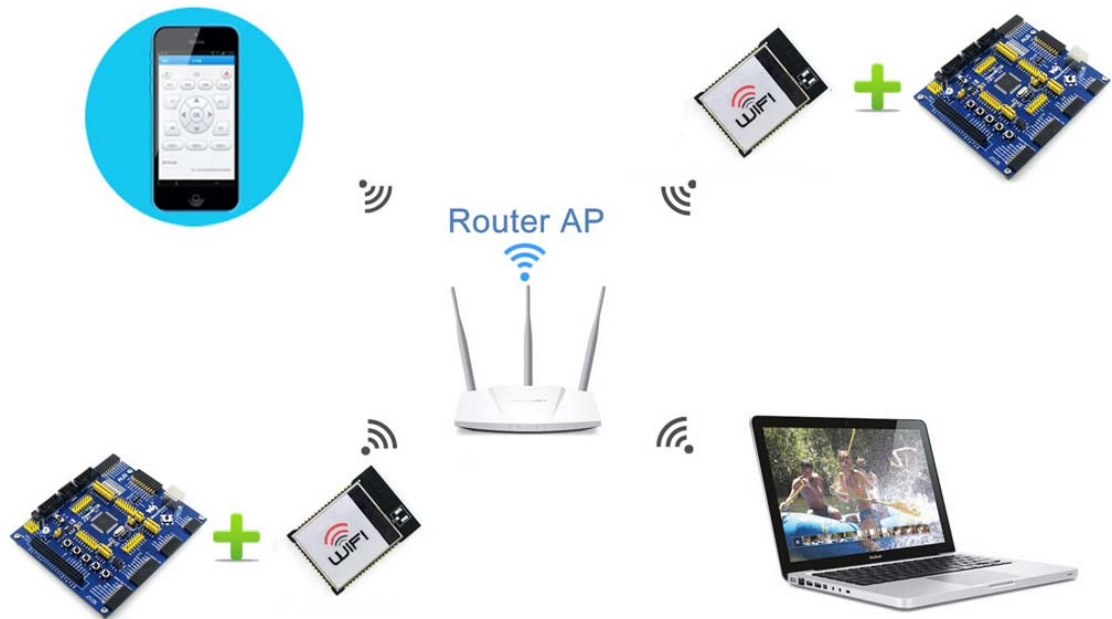


图 6-1 STA 连接

6.2 连接过程

6.2.1 STA 连接 WiFi 路由器

首先 AP 发出信标（beacon）帧，意思就是我在这里，谁来泡我啊，移动工作站也会发出探（probe）帧，意思是有谁我可以泡啊。这个是工作站主动发出来的，每隔一定时间发

出一次。所以，根据这点，可以产生很多有价值的应用。比如你的带 WiFi 功能的手机，即使不连接 wifi 的情况下，只要打开 WiFi 功能，就可以被路由器截获这帧信息，路由器收集之后，你的信息就会被一个审计的东西发到服务器上，你手机号 xxx 上线时间 xxx 下线时间 xxx 都浏览了那些网页，WiFi 建立连接过程都一目了然，你的位置也全都暴露了，这就叫 WiFi 探针。良心企业可以据此开发优质产品，为人民服务。具体流程如下所示：

1. STA -----> Probe Request -----> AP

2. STA <----- Probe Response <----- AP ，这个是由 wifi 返回的应答帧；

接下来是身份验证过程，可以使用诸如 WEP、WEP2、WPA 等加密方式应用到认证请求上；

3. STA -----> Authentication Request -----> AP

认证请求中包含认证 Auth 类型，Open System , Shared Key 等信息，路由器返回认证结果；

4. STA <----- Authentication Response<----- AP

连接请求；

5. STA-----> Association Request -----> AP

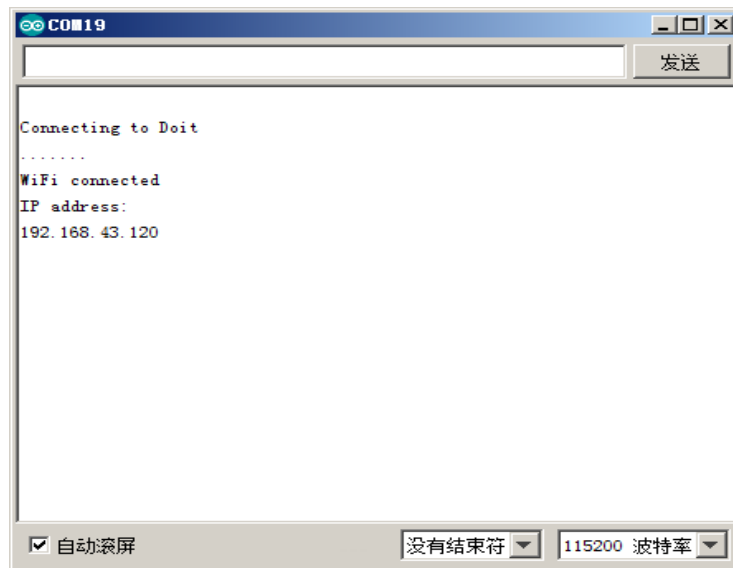
请求与 AP 建立关联，从而可以进行数据交互；

认证过连接请求 OK 返回。

6. STA<----- Association Response<----- AP

6.2.2 实验效果

从串口监视器中可以观察到开发板作为 STA 连接到了 Doit 网络上，其 IP 地址为 192.168.43.120。



6.2.3 源码分析

```
#include <ESP8266WiFi.h>
//连接 AP 的账号
const char* ssid    = "Doit";
//连接用的密码
const char* password = "doit3305";
void setup() {
    Serial.begin(115200);
    delay(10);
    Serial.println();
    Serial.println();
    Serial.print("Connecting to ");
    Serial.println(ssid);
    WiFi.begin(ssid, password);
    while (WiFi.status() != WL_CONNECTED) {
        delay(500);
        Serial.print(".");
    }
    Serial.println("");
    Serial.println("WiFi connected");
    Serial.println("IP address: ");
    Serial.println(WiFi.localIP());
}
void loop() {

}
```

6.3 AP+STA 共存模式

在了解了 WiFi 模块的两种工作模式之后，我们很自然地可以考虑这样一个问题，它能同时工作在两种模式之上么，进而起到中继、转发之类的网桥功能？答案几乎是肯定的，之所以用“几乎”，是因为这还需要 WiFi 芯片本身硬件的支持，幸运的是目前主流的芯片都具有这一功能，ESP8266 也不例外。

6.3.1 AP+STA 工作原理

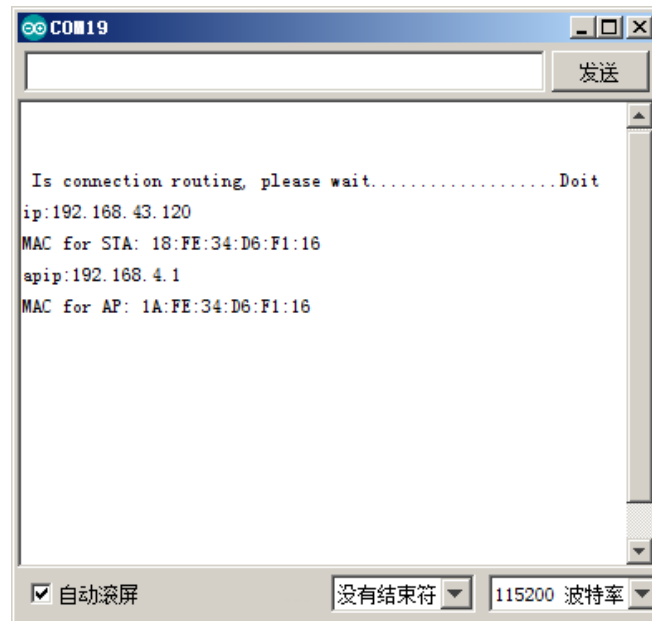
AP+STA 的工作原理就是：当模块作为 AP 时，它同时又可以作为一个 STA 模式存在，例如：WIFI 模块作为 AP，可以让客户的手机或者计算机接入，同时该模块又可以作为一个 STA 接入到路由器或者上位服务器进行数据上传。

需要注意的是，模块在 AP 和模块做 STA 时使用的 MAC 地址是不同的，所以在模块内部看到模块做 AP 时的 MAC 地址与在路由器里面去看到的模块作为 STA 时的 MAC 地址不同。对于 ESPDuino 来说，其搭载的 ESP8266 出厂时就有两个 MAC 地址。

需要指出的是 ESP8266 的 AP+STA 的功能和传统的路由中继功能也不相同，其自身不具备路由功能，需要在应用层开发相应的程序做数据转发。ESP8266 的 AP+STA 一个典型应用就是 wifi mesh 组网。

6.3.2 实验效果

从串口监视器可以看到，模块连接了上位的 Doit 网络，此时它是以 STA 模式，并获得 Doit 为它自动分配的 IP 地址：192.168.43.120，同时输出 STA 的 MAC 地址；而 ap ip 表示它还同时作为 Soft AP 提供热点服务，其 IP 为 192.168.4.1，并输出 AP 的 MAC 地址，可以看出，这两个 MAC 地址是不同的。



6.3.3 源码分析

```
#include <ESP8266WiFi.h>
#include <WiFiUdp.h>
WiFiUDP udp;
IPAddress sip(192, 168, 1, 29); //本地 IP
IPAddress sip1(192, 168, 1, 1); //本地网关
IPAddress sip2(255, 255, 255, 0); //本地子网掩码
IPAddress xip(192, 168, 2, 2); //下位远程 IP
IPAddress lxip(192, 168, 2, 1); //AP 端 IP
IPAddress lxip1(192, 168, 2, 1); //AP 端网关
IPAddress lxip2(255, 255, 255, 0); //AP 端子网掩码
IPAddress Serverip(192, 168, 1, 4); //上位机远程 IP
unsigned int localPort = 9999; //本地端口
unsigned int remoteport = 9999; //远程端口
const char *ssid = "Netcore_wsn"; //连接到远程 AP 的名字
const char *password = "99325408322"; //连接到远程 AP 的密码
const char *ssid1 = "Netcore_wsn1"; //建立 AP 名字
const char *password1 = "99325408322"; //AP 连接密码
char packetBuffer[255]; //收发缓冲区
void setup() {Serial.begin(115200); //初始化串口波特率
delay(5000); //延时 5S
WiFi.mode(WIFI_AP_STA); //设置模式为 AP+STA
WiFi.softAPConfig(lxip, lxip1, lxip2); //设置 AP 网络参数
WiFi.softAP(ssid1, password1, 1); //设置 AP 账号密码
WiFi.begin(ssid, password); //连接指定路由
WiFi.config(sip, sip1, sip2); //设置本地网络参数
Serial.print("Is connection routing, please wait");
```

```

while(WiFi.status()!=WL_CONNECTED)//等待路由连接
{

delay(500);
Serial.print(".");
}
Serial.println(" ");
udp.begin(localPort);//建立 UDP Server, 监听指定端口
Serial.print("ip:");
Serial.println(WiFi.localIP());
Serial.print("apip:");
Serial.println(WiFi.softAPIP());
}
void loop()
{
if(udp.parsePacket())
{
udp.read(packetBuffer, 255);//读取数据
udp.beginPacket(Serverip, remoteport);
udp.write(packetBuffer, 255);
udp.endPacket();
Serial.println(packetBuffer);
udp.beginPacket(xip, remoteport);
udp.write(packetBuffer, 255);
udp.endPacket();
memset(packetBuffer, 0, 255);//清除缓冲器数值
}
}

```

在这个例子当中，我们建立了一个 UDP Server，为了和这个 Server 通讯，我们有两种方式：

1. 连接 ESPDuino 建立的 Netcore_wsn1 热点,这时分得到 192.168.1.网段的 ip, 此时和 ESPDuino 的 AP 模式通讯。
2. 连接 Netcore_wsn 热点, 和 ESPDuino 处于同一个网络当中, 也可以通过 UDP 进行通讯。

在做了上面两个步骤中的一个这后，就可以找一个 UDP 调试工具和 ESPDuino 进行通讯了，实际运行的效果 6.3.2 所示。

第 7 章 Scan WiFi—众里寻他千百度

7.1 我扫故我在

通常，无线网络中提供 WiFi 接入的热点较多，大部分也都开放了 SSID 广播，Scan WiFi 的功能就是可用扫描出所有附近 WiFi 热点的 SSID 信息，这样，一个客户端就可以根据需要进行不同的 SSID 连入对应无线网络中。

7.2 试验效果

我们把串口信息打印如下，可以看出，当前环境下扫描出了 15 个不同的 WiFi 热点（当前屏幕只显示了 13 个），如 Doit (-72)* 这个结果就表示 SSID 名称为 Doit，加密的 WiFi 网络（用*代表，无*的网络表示开放，没有设置密码），其网络连接质量是括号中的数值表示，此处为-72dBm，表示信号接收的强度，一般为负值，绝对值越小,信号强度越大，连接质量也会越高。

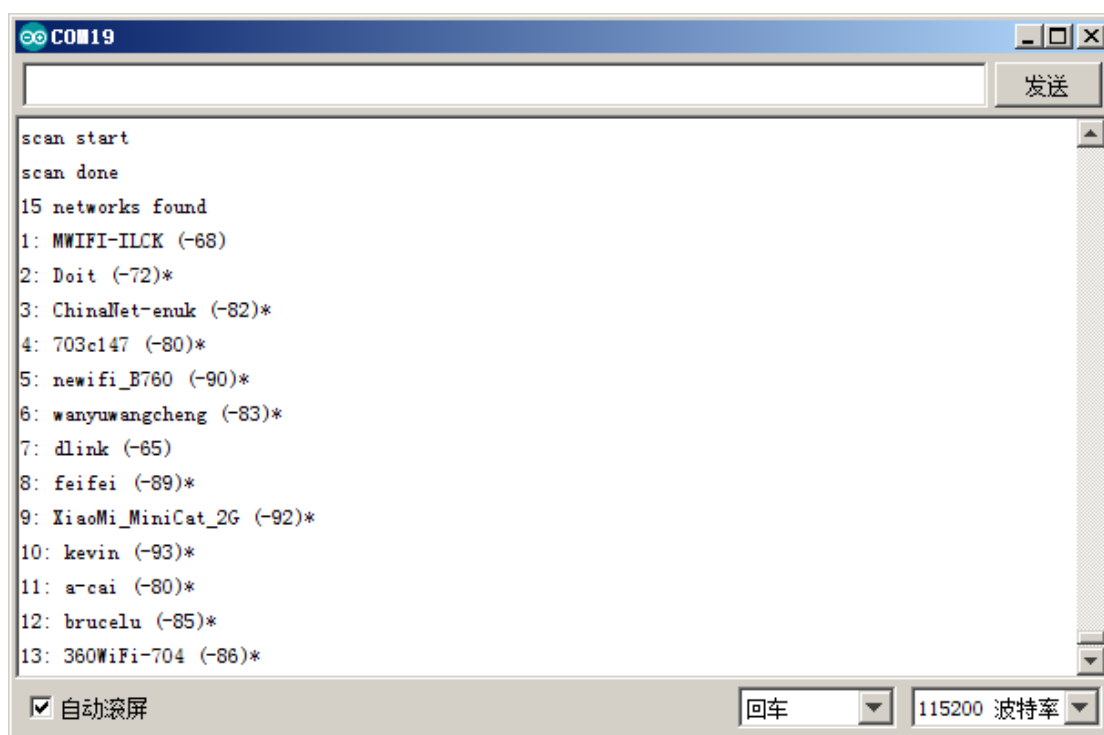


图 7-1 串口监视到多个 WiFi 热点

7.3 源码分析

```
#include "ESP8266WiFi.h"

void setup() {
    Serial.begin(115200);
    //设定 WiFi 为 STA 模式，如果先前已连接上 AP，则与之断开
    WiFi.mode(WIFI_STA);
    WiFi.disconnect();
    delay(100);
    Serial.println("Setup done");
}

void loop() {
    Serial.println("scan start");
    // WiFi.scanNetworks 可以返回扫描到的无线网络数目
    int n = WiFi.scanNetworks();
    Serial.println("scan done");
    if (n == 0)
        Serial.println("no networks found");
    else
    {
        Serial.print(n);
        Serial.println(" networks found");
        for (int i = 0; i < n; ++i)
        {
            /*在串口输出所发现的每一个无线网络的 SSID 和所接收的信号强度 RSSI */
            Serial.print(i + 1);
            Serial.print(": ");
            Serial.print(WiFi.SSID(i));
            Serial.print(" (");
            Serial.print(WiFi.RSSI(i));
            Serial.print(")");
            //判断 WiFi 网络是否加密，并输出结果
            Serial.println((WiFi.encryptionType(i) == ENC_TYPE_NONE)?" ":"*");
            delay(10);
        }
        Serial.println("");
        //连续扫描的时间间隔
        delay(5000);
    }
}
```

第 8 章 从 SmartConfig 到 AirKiss

SmartConfig 智能连接技术，是指 WiFi 芯片本身在不具有可视输入的情况下实现访问特定 SSID 以及通过安全密码校验的过程，实际上是通过第三方设备来配置 WiFi 模组的连接信息，达到加入 WiFi 网络的目的。SmartConfig 作为智能配置技术的统称，而 Airkiss 是微信对 SmartConfig 技术的一种具体实现。为了表述方便，我们将 AirKiss 归成 SmartConfig 的一种。

8.1 SmartConfig 的作用

8.1.1 为什么会有 SmartConfig

当拿到一个 WiFi 模块，如果要使模块上网，需要将 SSID 名称、密码设置到模块当中，如何才能设置呢？通常有如下方法：

1. 把模块通过串口接到一个输入设备上，在此设备上输入 SSID 名称和密码，然后通过串口设置（比如 AT 指令）。
2. 让设备先自己提供一个无线热点(Wireless AP)，然后更改手机（或是笔记本）的无线配置连接上设备的无线热点。此时设备与手机算是连上了，这时候再通过手机将家里 WiFi 的 SSID 及密码配置到设备上。接下来再把手机重新连接到家里的 WiFi。虽然只要初始配置这么一次，还是有些麻烦。
3. 通过 SmartConfig 技术直接配置，微信的 Airkiss、ESP8266 的 esptouch、EMW3165 的 easylink 都归为此类。

8.1.2 SmartConfig 工作流程：

举个例子，比如您买了一个 WiFi 摄像头，要使其上网，流程如下：

1. 摄像头插上电源；
2. 安装制造商提供的手机 app(应用无需任何特殊权限，只需要手机当前是接入 wifi 的)；
3. 在摄像头附近打开 app，输入你家 wifi 的密码，点击确认，稍等片刻，不出意外的话，摄像头已经接入你家 wifi 了。

支持 SmartConfig 的 WiFi 设备通常都是以上流程。

8.1.3 SmartConfig 工作原理

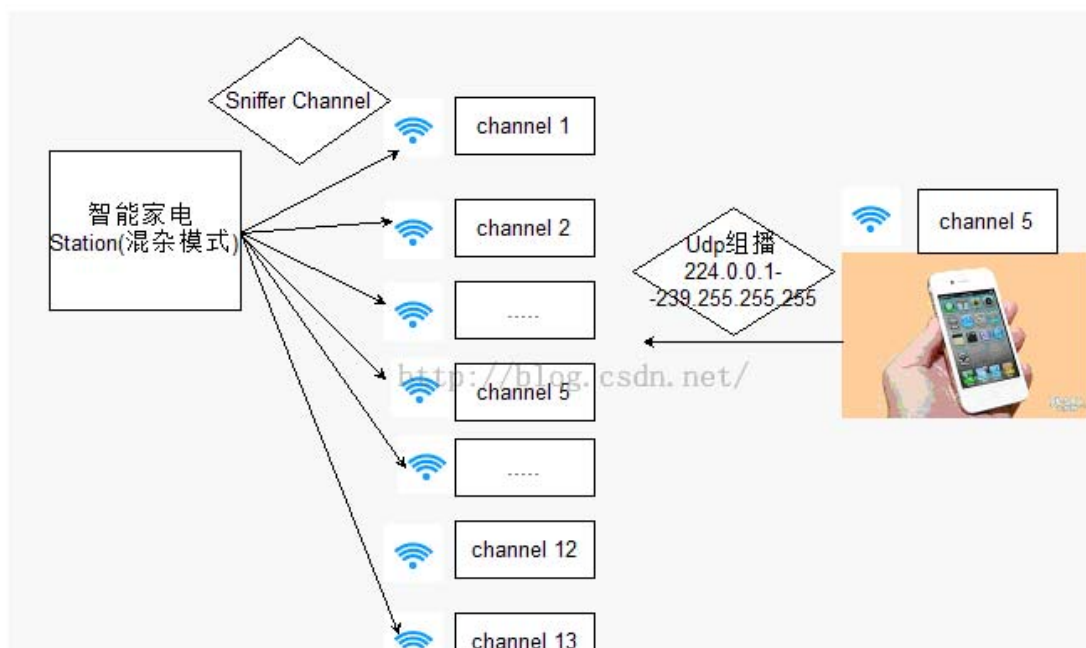


图 8-1 SmartConfig 原理

Smartcnfig 的技术本质是 UDP 广播,手机通过 UDP 广播,将 AP 的 SSID 名称和密码编码发出。WiFi 模块处于 monitor 模式下监听抓包,获取到 SSID 名称和密码之后,WiFi 模块便可以接入路由上网了,从原理上说支持混杂模式的 wifi 芯片都可以应用该技术

这里有一个难点是,如果 WLAN 都是不加密的话,那 UDP 直接把相关信息发出来就好,但可惜路由器的加密方式是不固定的,模块没法知道 UDP 数据是何种加密方式,因此没办法解析出 DATA 信息。如下图是 802.2 的封装格式:

DA	SA	Length	LLC	SNAP	DATA	FCS
6 Bytes	6 Bytes	2 Bytes	3 Bytes	5 Bytes	~	4Bytes

图 8-2 802.2 协议包的封装格式

所以,DATA 不行,聪明的人类打算从其他字段下手,大部分字段手机端无法获得权限,最适合的只剩下 Length(长度)字段:

DA	SA	Length	LLC	SNAP	DATA	FCS
---------------	---------------	--------	----------------	-----------------	-----------------	----------------

这里举个例子，比如要发送“1, 2, 3”，（即 1、2、3 的 ASCII 码值：0x31, 0x32, 0x33），那么应该发送如下数据：

DA	SA	0x31	LLC	SNAP	DATA	FCS
----	----	------	-----	------	------	-----

DA	SA	0x32	LLC	SNAP	DATA	FCS
----	----	------	-----	------	------	-----

DA	SA	0x33	LLC	SNAP	DATA	FCS
----	----	------	-----	------	------	-----

图 8-3 UDP 协议传输数据包的例子

8.1.4 实现细节

当然，原理虽是如此，实际还有许多可优化之处，下面以 ESP8266 为例说明：

1. 如果锁定 channel:

WiFi 通常工作的 channel 是从 1 到 13，如果不能确定工作在哪一个 channel 上，只能定时切换，比如在每个 channel 工作 100ms,这样的问题时，当切换到一个 channel 上时，udp 广播到其它 channel,比如模块在 channel 1 时监听，而手机在 7 上发包，这时配置成功就低很多了。

如何快速锁定 channel 呢? esp8266 先 scan 下 AP，得到 AP 的相关信息,如工作的 channel，然后配置 wifi 芯片工作在刚才 scan 到的 channel 上去接收 UDP 包，如果没有接收到，继续配置 ESP8266 工作在另外的 channel 上，如此循环，直到收到 UDP 包为止。

2. 为什么不通用

- 加密算法不同：为了确保通信安全，每家厂商都对编码进行了加密，比如微信的 airkiss 就和 esp8266 的 esptouch 不相同。
- 成功通知手段不同：当模块配置成功上网后，如何通知手机端呢，每一家厂商也不一样：Airkiss 通知 UDP 广播，Easylink 通过 http 通知。事实上这些硬件厂商的快连技术都是依托上游的芯片方案商，我这里把最近接触的一些厂商给列出来。他们的盲连接技术原理上都类似，但各家都分别给这项技术取了名字。如下表所示：

表 8-1 各类盲连接技术

厂商	技术名称
TI	SmartConfig

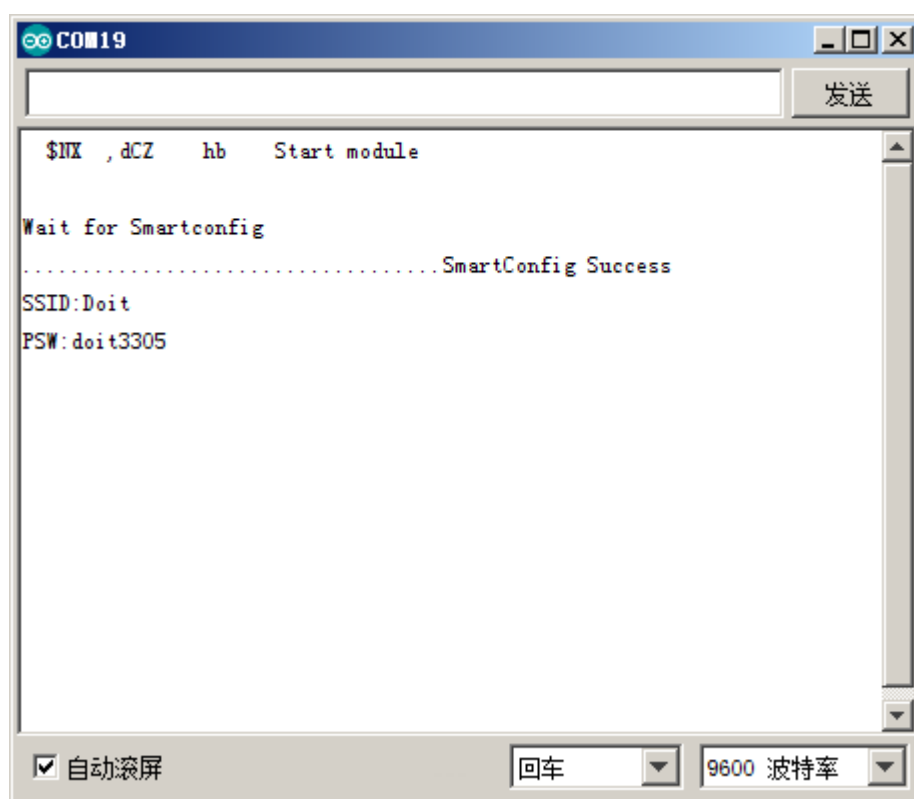
MTK	SmartConnection
Marvell	EasyConnect
Realtek	SimpleConfig
Espressif	SmartConfig
wechat	AirKiss

SmartConfig 是目前使用最多的称呼。

8.2 SmartConfig ESPDuino

8.2.1 工作流程

为了将带有 ESPDuino 的嵌入式设备连接上 WiFi 路由器，我们需要手机端的应用（专用 app 或者微信）作为连接信息输入的管道，同时把具有 SmartConfig 功能的 Arduino 程序上传固件，就可以在手机端进行 WiFi 网络的配置了。开发板加电后，进入 STA 模式，LED 灯会连续闪烁，等待相应手机端的配置信息送达，串口显示配置进度信息，配置成功后，串口打印出所连接上的 WiFi 名称和密码，配置结束。下图为串口监视窗口的运行状态，显示已成功连接上 Doit 网络。



8.2.2 源码分析

```
/*
  文件名称: smartconfig.ino
  功能: ESP8266 快速配置功能
  作者: www.doit.am
  日期: 2015-12-31
  版本: 1.0
*/
#include <ESP8266WiFi.h>
#define LED 16

void smartConfig()
{
  WiFi.mode(WIFI_STA);
  Serial.println("\r\nWait for Smartconfig");
  WiFi.beginSmartConfig();
  while (1)
  {
    Serial.print(".");
    digitalWrite(LED, 0);
    delay(500);
    digitalWrite(LED, 1);
    delay(500);
    if (WiFi.smartConfigDone())
    {
      Serial.println("SmartConfig Success");
      Serial.printf("SSID:%s\r\n", WiFi.SSID().c_str());
      Serial.printf("PSW:%s\r\n", WiFi.psk().c_str());
      break;
    }
  }
}

void setup()
{
  Serial.begin(9600);
  Serial.println("Start module");
  pinMode(LED, OUTPUT);
  digitalWrite(LED, 0);
  smartConfig();
}

void loop()
```

```
{  
  delay(100);  
}
```

代码首先设置 WiFi 工作模式 `WiFi.mode(WIFI_STA)`，然后进行 `WiFi.beginSmartConfig` 调用，当 SmartConfig 设置成功后，会默认存账号和密码到 Flash 中，当然你也可以根据返回自己存储。

8.3 手机设置

在把 WiFi 模块设置为 SmartConfig 之后，就需要第三方工具进行实际地配置 SSID 和密码信息，以便模块可以正确连入所需的无线网络，我们使用智能手机，设计了两类方式来实现输入 ESPDuino 联网参数（热点的 SSID 和密码）的功能，避免了一般 WiFi 模块没有直接输入界面的尴尬。当 SDK1.4 之后，Esp8266 能同时支持 ESPtouch 和 Airkiss 技术了，所以我们能用同一套代码，来应对不同的智能配置。

8.3.1 通过专用 app 设置

1. 首先从

<http://bbs.doit.am/forum.php?mod=attachment&aid=NjJ8NGU3MmFiYjJ8MTQ1MjYzMTE1NnwxfDQz> 下载专用 apk，安装并打开，界面显示显示如下，此时，没有通过网页注册的也可以在这里注册：



(a)

(b)

图 8-4 登陆和添加设备

2. 点击添加设备->一键配置（ESP8266）



(a)

(b)

图 8-5 配置及验证

3. 点击开始配置，界面显示的是手机已经登录的 WiFi 热点，Doit，此时，输入连接密码，确定。



图 8-6 成功

4. 观察 ESPDuino，指示灯亮，配置成功。

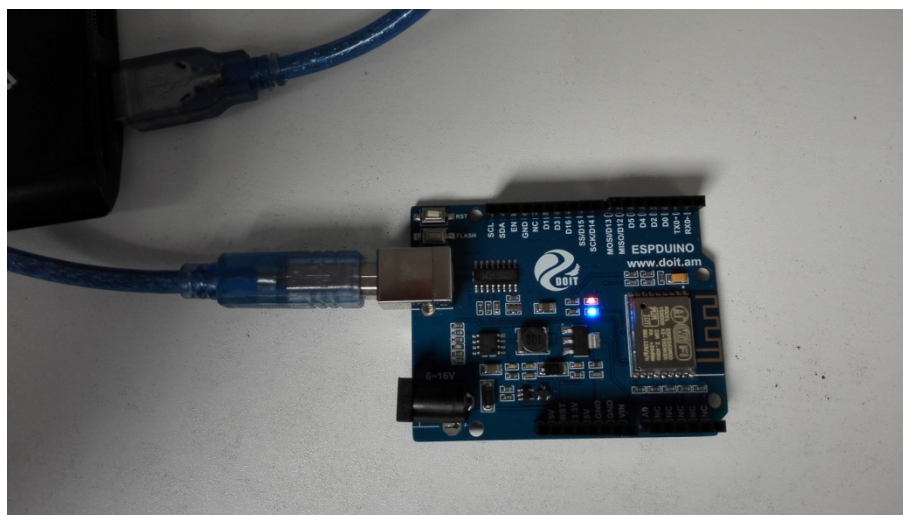


图 8-7 实测状态

温馨提示

在 iOS 版本上实现的 ESP8266 的盲连接专门应用可在此处下载：

<https://itunes.apple.com/us/app/esptouch/id1058702338?l=zh&ls=1&mt=8>

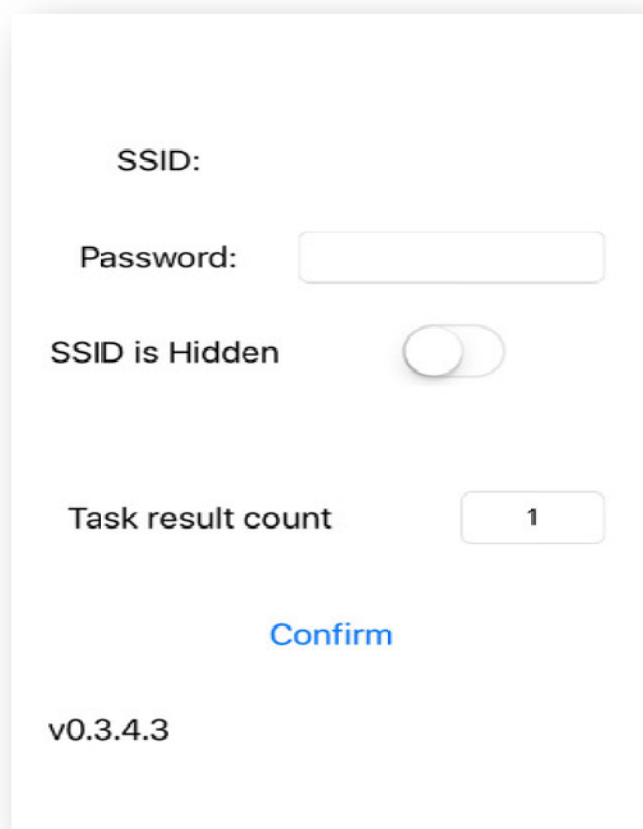


图 8-8 iOS 盲连接设置界面

8.3.2 通过微信设置

如果不想下载 App，也可以使用微信来实现 SmartConfig，当然，这就要求您的手机必须能接入互联网，步骤如下：

1. 先关注四博智联的公共服务号：

搜索“四博智联研发部”--->关注，或者直接扫描下图二维码关注：

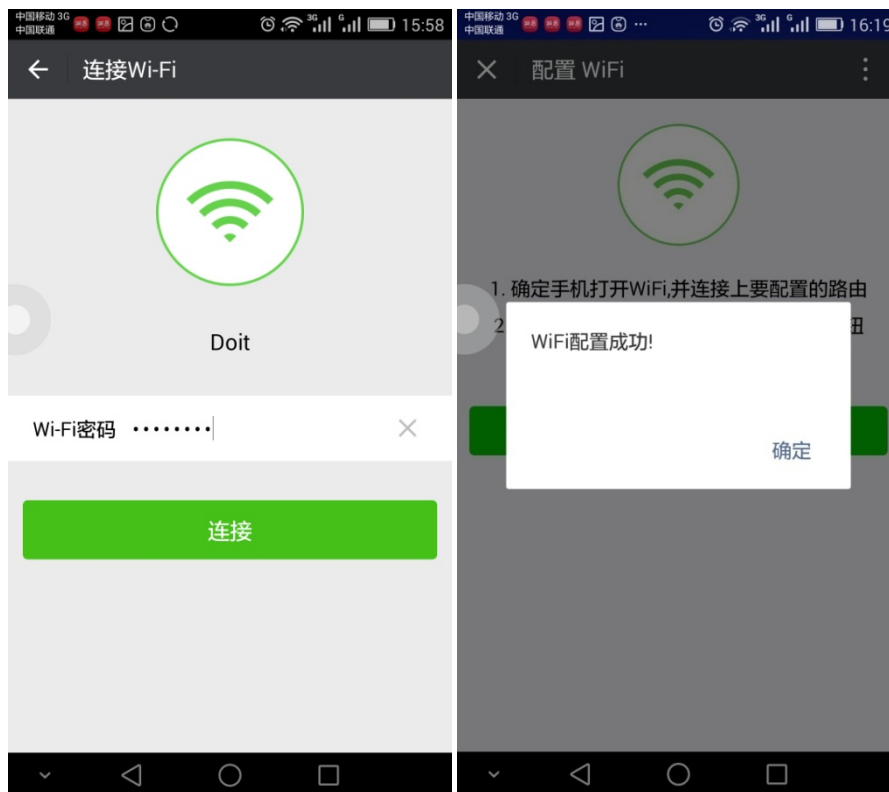


图 8-9 Doit 二维码

2. 点击 WiFi 配置->开始配置 (Doit)。由于手机当前连接的是 Doit 热点，因此后续 STA 所连的热点也会是 Doit。



3. 留意一下屏幕显示的注意事项，按照屏幕提示逐步操作即可完成。若中间因各种原因配置失败，重试即可，直到成功。



二、进阶篇

第 9 章 TCP Server/TCP Client

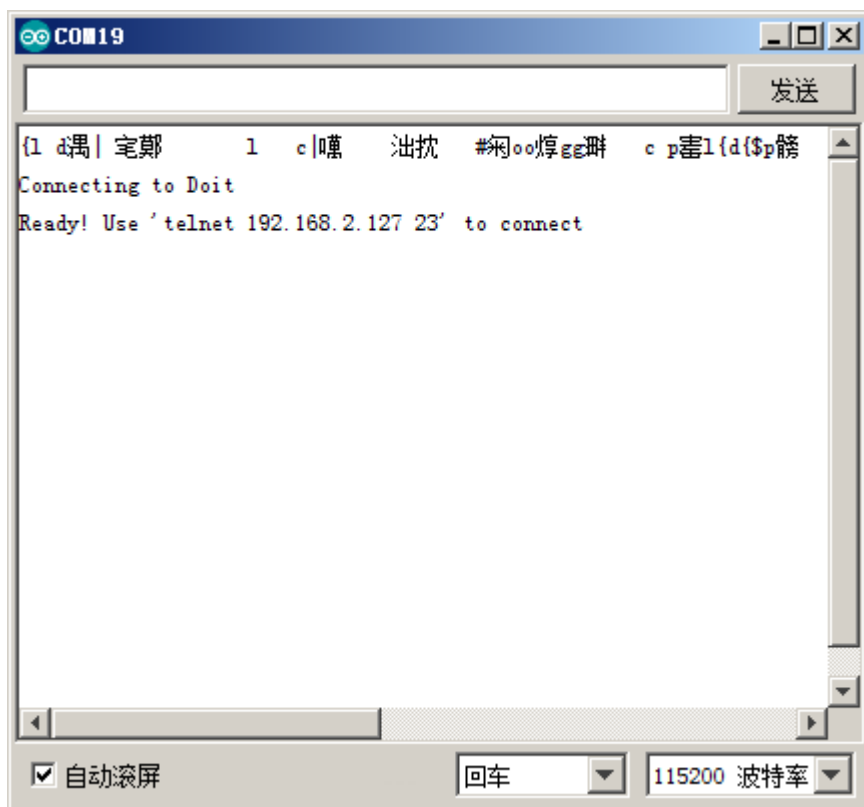
在当前的大部分应用中，通讯都是通过 TCP 协议进行的，幸运的是，通过编写和上传相应的固件程序，ESPduino 可以分别实现 TCP 服务器和客户端的功能。

9.1 TCP Server

Telnet 是使用 TCP 协议进行应用层通讯的最常用的工具之一。这里，我们使用它来测试被设置为 TCP 服务器的 ESPduino 工作是否正常。

9.1.1 实验步骤

1. 编写代码并上传固件，串口监视窗口显示如下内容，显示已经成功连接到 Doit 网络，并提示可以使用 telnet 来连接 ESPduino TCP Server 了，其中 Server 的 IP 地址为连接 Doit 网络时通过 DHCP 给它分配的 192.168.127.23。



2. 在 windows7 中，使用 cmd 命令打开命令行窗口，输入 telnet:



图 9-1 调用命令行

3. 进入 telnet 环境，使用 open 192.168.2.127 23 打开连接，

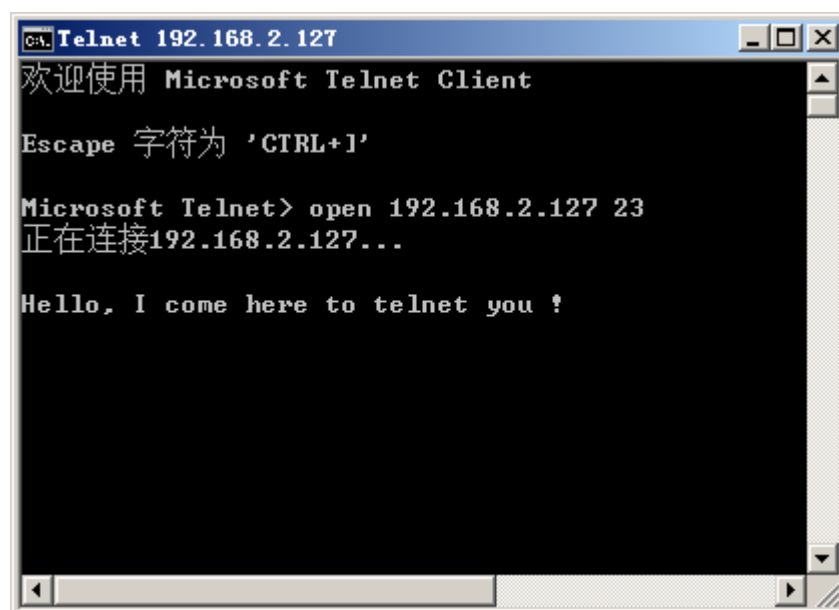
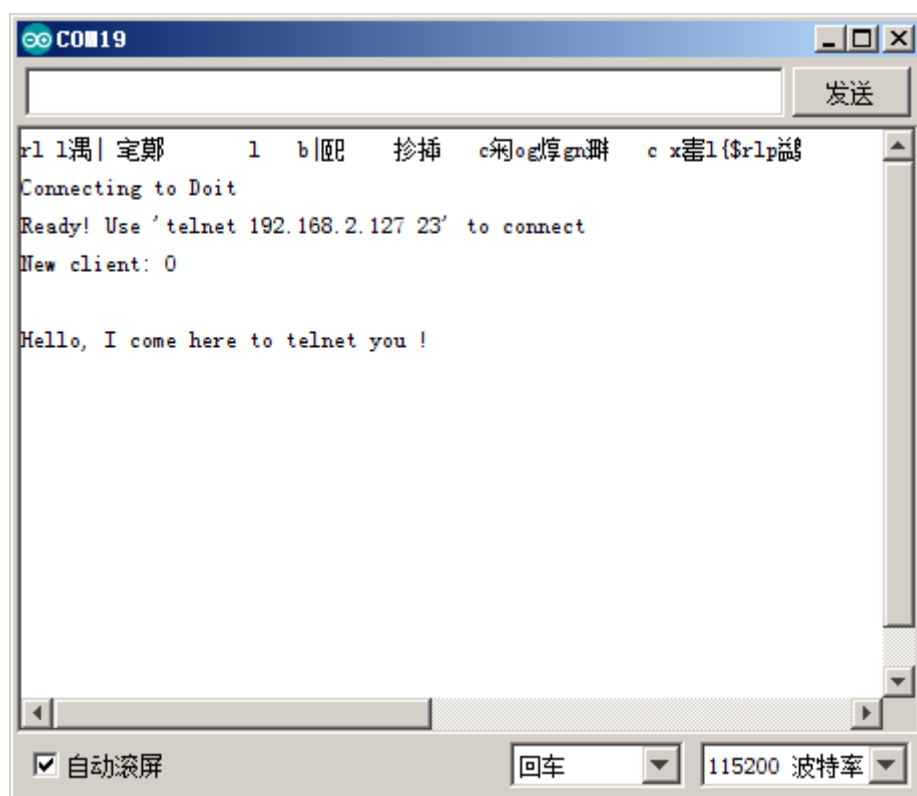


图 9-2 Telnet 连接窗口

4. 在客户端输入 “Hello, I come here to telnet you !” 后，观察串口监视窗，也显示了相同内容，说明 telnet 客户端和此 ESPDuino 服务器的 TCP 通讯正常。



9.1.2 源码分析

```
WiFiTelnetToSerial $
1 /* WiFiTelnetToSerial - 使用通用异步收发传输器 (Universal Asynchronous
2 Receiver/Transmitter, UART) 与ESP8266进行TCP透传的例子 */
3 #include <ESP8266WiFi.h>
4
5 //定义可连接的客户端数目最大值
6 #define MAX_SRV_CLIENTS 1
7 const char* ssid = "Doit";
8 const char* password = "doit3305";
9
10 WiFiServer server(23);
11 WiFiClient serverClients[MAX_SRV_CLIENTS];
12
13 void setup() {
14     Serial.begin(115200);
15     WiFi.begin(ssid, password);
16     Serial.print("\nConnecting to "); Serial.println(ssid);
17     uint8_t i = 0;
18     while (WiFi.status() != WL_CONNECTED && i++ < 20) delay(500);
19     if(i == 21){ //超时 (20x500=10000, 10秒钟), 提示连接失败
20         Serial.print("Could not connect to"); Serial.println(ssid);
21         while(1) delay(500);
22     }
23     //启动UART传输和服务端
24     server.begin();
25     server.setNoDelay(true);
26     |
27     Serial.print("Ready! Use 'telnet ");
28     Serial.print(WiFi.localIP()); //获得服务器本地IP
29     Serial.println(" 23' to connect");
30 }
31
32 void loop() {
33     uint8_t i;
34     //检测服务器端是否有活动的客户端连接
35     if (server.hasClient()){
36         for(i = 0; i < MAX_SRV_CLIENTS; i++){
37             //查找空闲或者断开连接的客户端, 并置为可用
38             if (!serverClients[i] || !serverClients[i].connected()){
39                 if(serverClients[i]) serverClients[i].stop();
40                 serverClients[i] = server.available();
41                 Serial.print("New client: "); Serial.println(i);
42                 continue;
43             }
44         }
45         //若没有可用客户端, 则停止连接
46         WiFiClient serverClient = server.available();
47         serverClient.stop();
48     }
}
```

```

49 //检查客户端的数据
50 for(i = 0; i < MAX_SRV_CLIENTS; i++){
51     if (serverClients[i] && serverClients[i].connected()){
52         if(serverClients[i].available()){
53             //从Telnet客户端获取数据，并推送到UART端口
54             while(serverClients[i].available()) Serial.write(serverClients[i].read());
55         }
56     }
57 }
58 //检查UART端口数据
59 if(Serial.available()){
60     size_t len = Serial.available();
61     uint8_t sbuf[len];
62     Serial.readBytes(sbuf, len);
63     //将UART端口数据推送到所有已连接的telnet客户端，实现双向通信
64     for(i = 0; i < MAX_SRV_CLIENTS; i++){
65         if (serverClients[i] && serverClients[i].connected()){
66             serverClients[i].write(sbuf, len);
67             delay(1);
68         }
69     }
70 }
71 }

```

保存完成。

9.2 TCP Client

在另外一些场合下，需要模块之间或是模块与其它终端（比如手机）通过服务器进行通讯，针对商业的产品一般都要专用的平台，对于多数玩家特别是初学者有，掌握原理更重要一些，基于此，Doit 开发了一个公网的 TCP 调试助手，使用 Web 方式，相对于其它的调试工具，它有如下特点：

- 超级简单：不需要注册和绑定设备，直接就可以使用。
- 超级稳定：自开放始，每天大约有几百玩家在使用，运行良好，受到一致好评。

利用此调试助手，可以实现 TCP 协议的点到点透传功能。

9.2.1 实验步骤

1. 登录 <http://tcp.doit.am> 获取要使用的 IP 和端口：

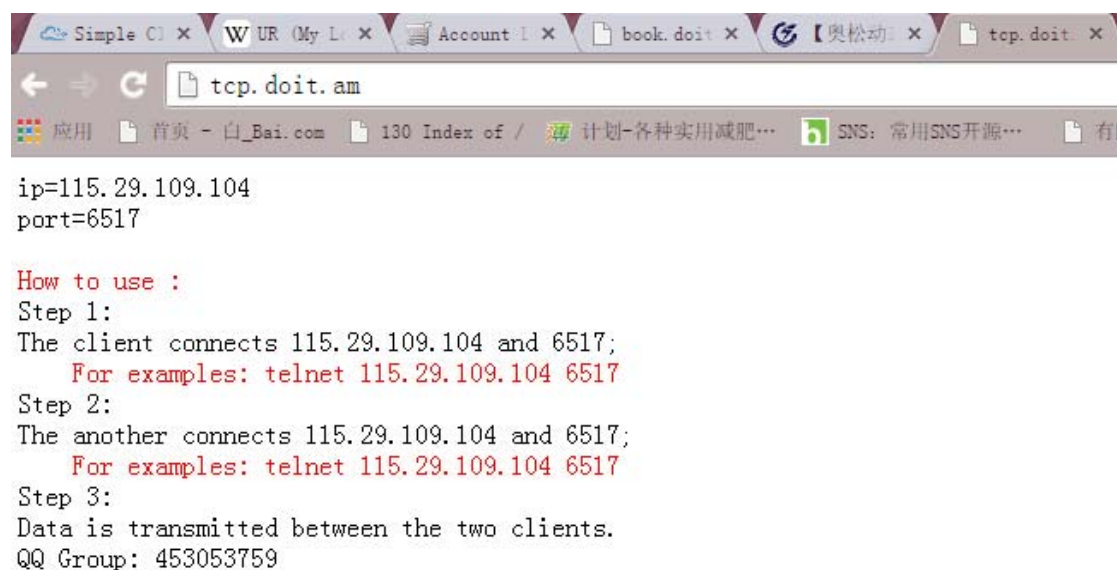


图 9-3 调试助手信息

2. 在 ESPDuino 的 Tcp Client 工程中，修改成步骤 1 的 IP 和端口（源码见后）
3. 下载并运行 TCP 调试助手，将步骤 1 的 IP 和端口填入。
4. 将 ESPDuino 例子程序烧录到板子运行。
5. 此时双方就可以愉快的通讯了。

9.2.2 具体方法

1. ESPDuino 启动后，连接热点，反馈成功信息如下：

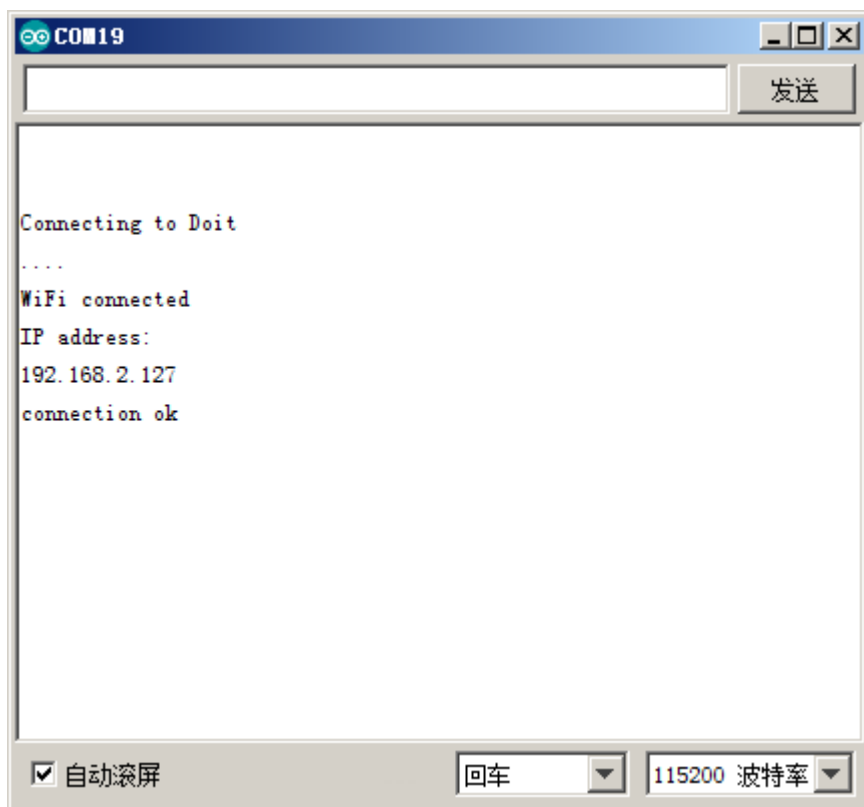


图 9-4 连接信息

2. 下载 TCP/UDP 测试工具，PC 版或手机版的都行，如
<http://bbs.doit.am/forum.php?mod=viewthread&tid=174&highlight=tcp>,
3. 安装后打开，在 Client Mode 下新建一个服务器连接，ip 和端口就是先前 9.2.1 中 1 所列出的服务器信息。连接成功后可以通过右上的调试窗口发送信息，右下显示接收到的信息。在本例中，输入 “can u hear me?” ,点击 “Send” ,显示出口回显相同信息。

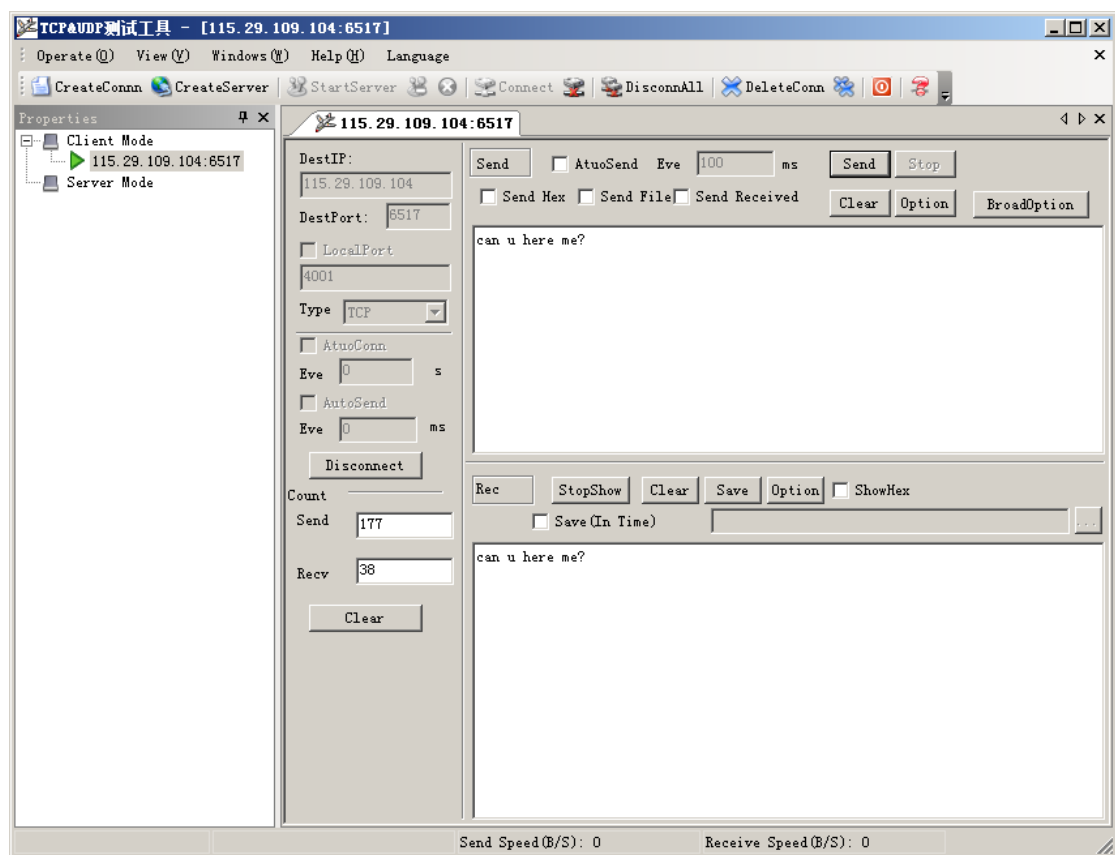


图 9-5 客户端的 TCP/UDP 调试工具

4. 此时观察串口监视器输出, 可以看到提示“Data is coming”, 并输出刚刚发送的“can u hear me?”

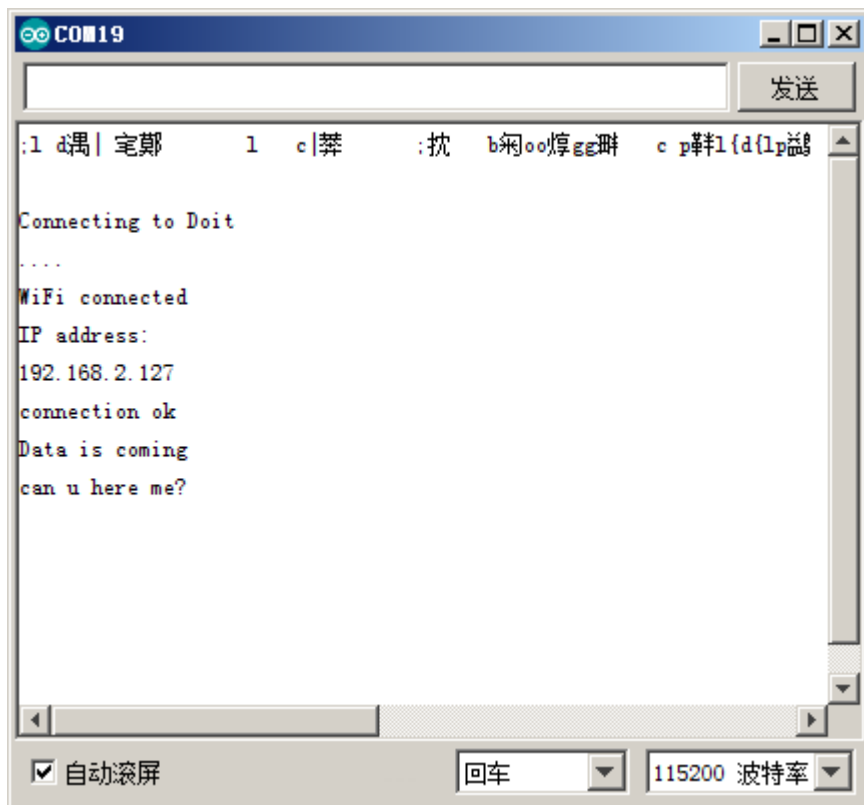


图 9-6 串口接收到输入

5. 3. 还可以添加多个终端，如在 windows 7 下打开 telnet，登陆到 TCP 调试助手服务器的 ip 上

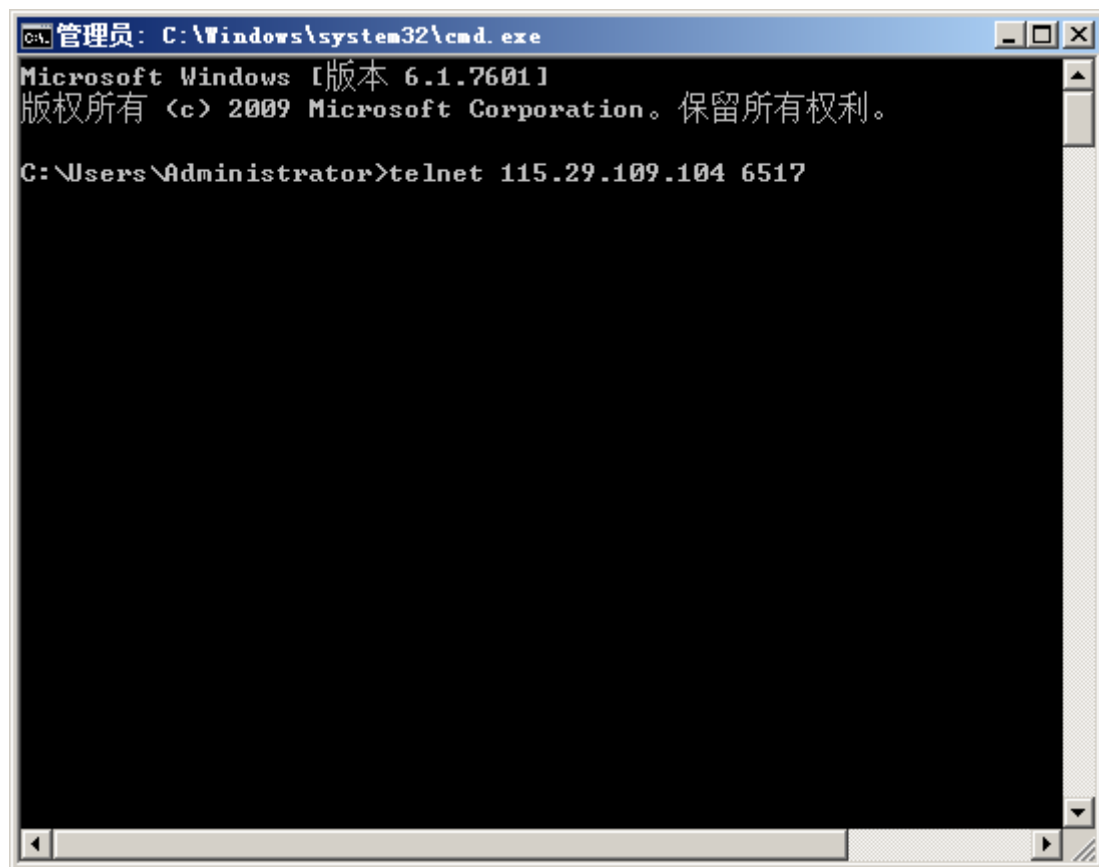


图 9-7 Telnet 到 TCP 调试助手服务器

6. 同时，在 TCP 测试工具中再添加一个到调试助手服务器的连接：

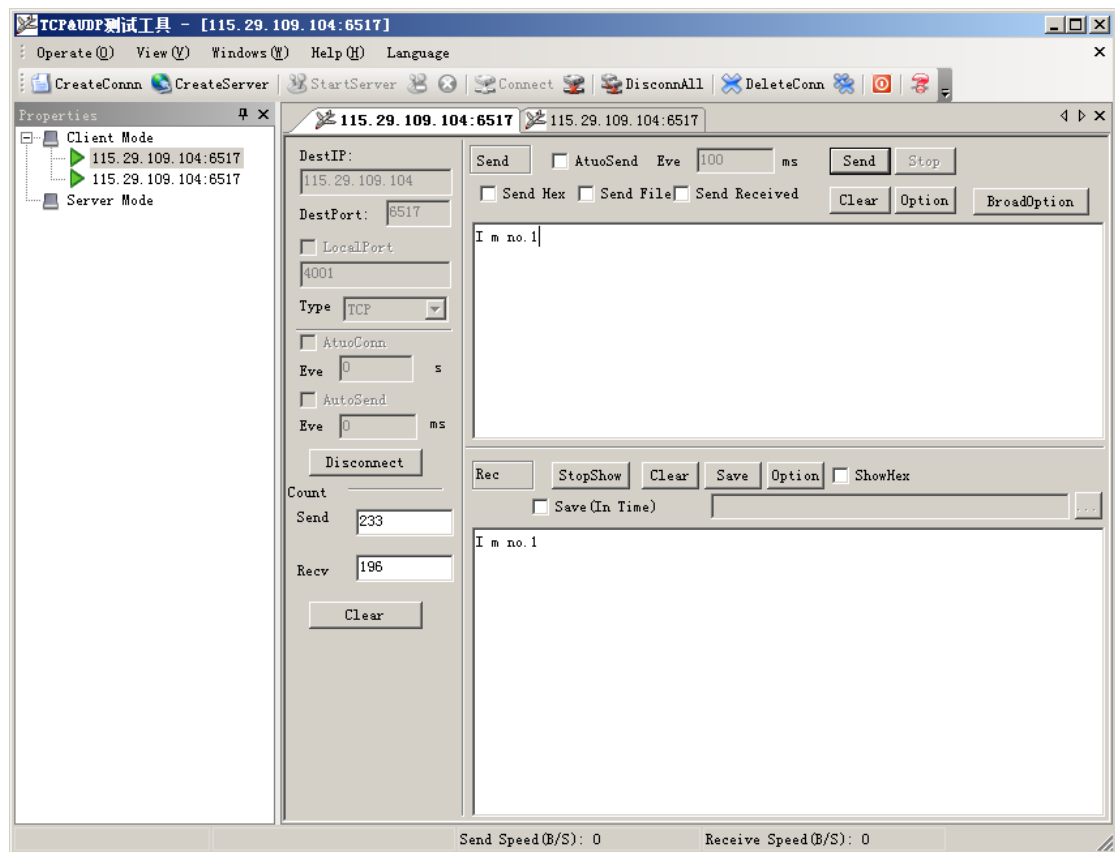


图 9-8 创建一个新的客户端连接

这样，测试环境就同时存在了三个终端，那这三个终端可以互相通信么？答案是肯定的：

7. 如上图所示，在测试工具的第一个终端发送窗口输入“l m no.1”，串口监视器和 telnet 客户端会分别收到同样的信息，如下图所示：

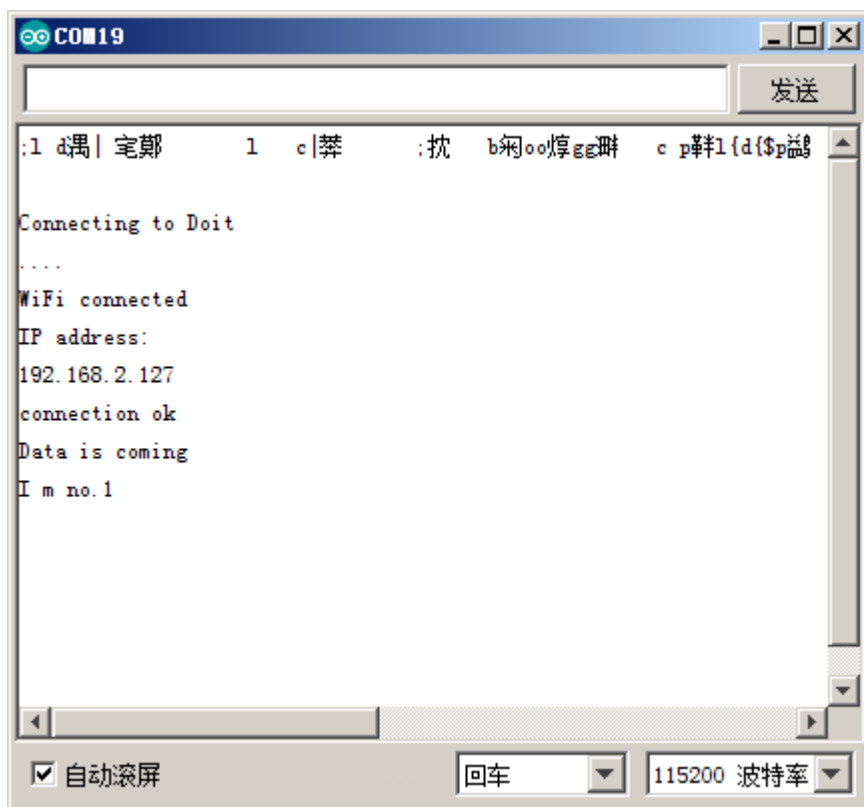


图 9-9 串口收到

Telnet 终端显示信息:

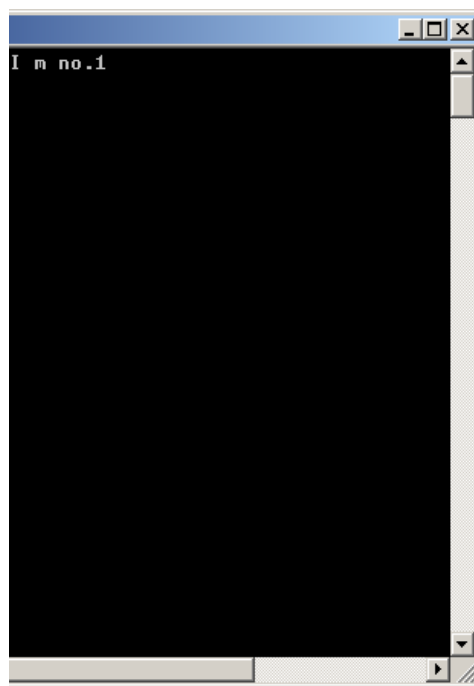


图 9-10 Telnet 终端收到

8. 测试工具中的第二个终端也可以进行同样的操作，比如发送“I m the 2 sender”，其他两

个终端均会收到响应信息，分别显示如下：

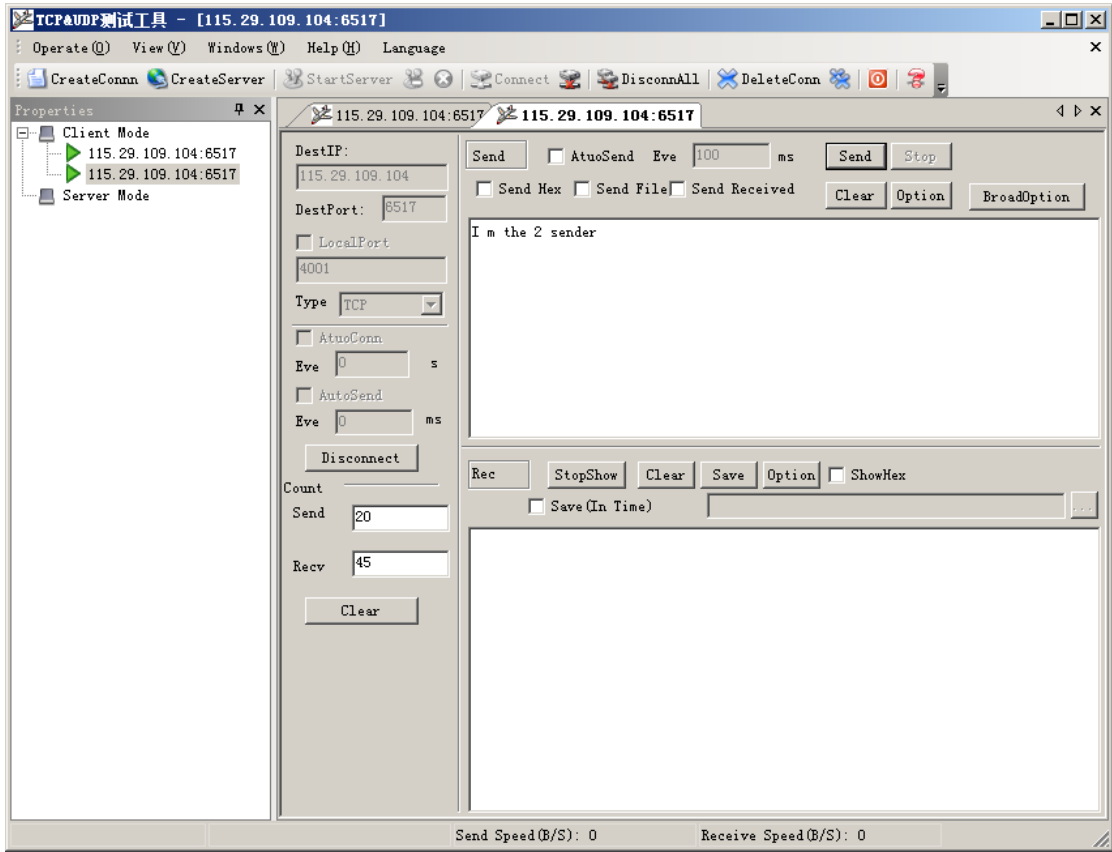


图 9-11 第二个终端发送信息

Telnet 终端显示：

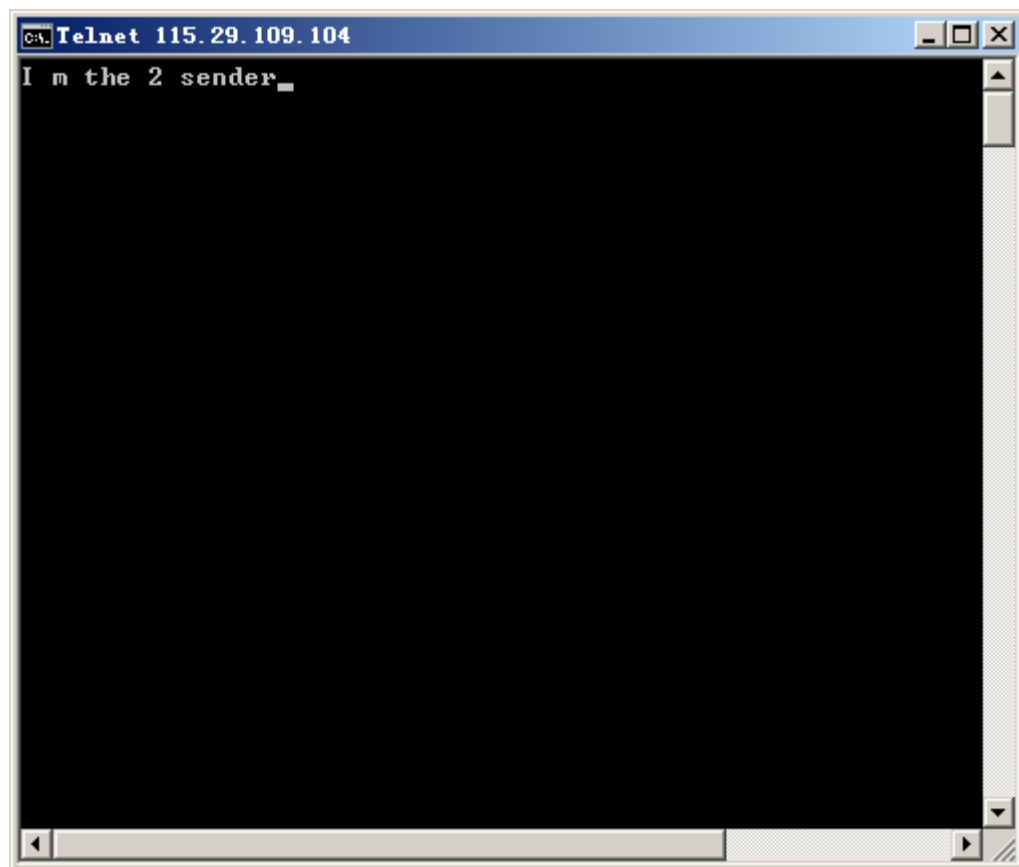


图 9-12 Telnet 终端收到

测试工具的第一个终端的信息接收窗口也显示同样信息：

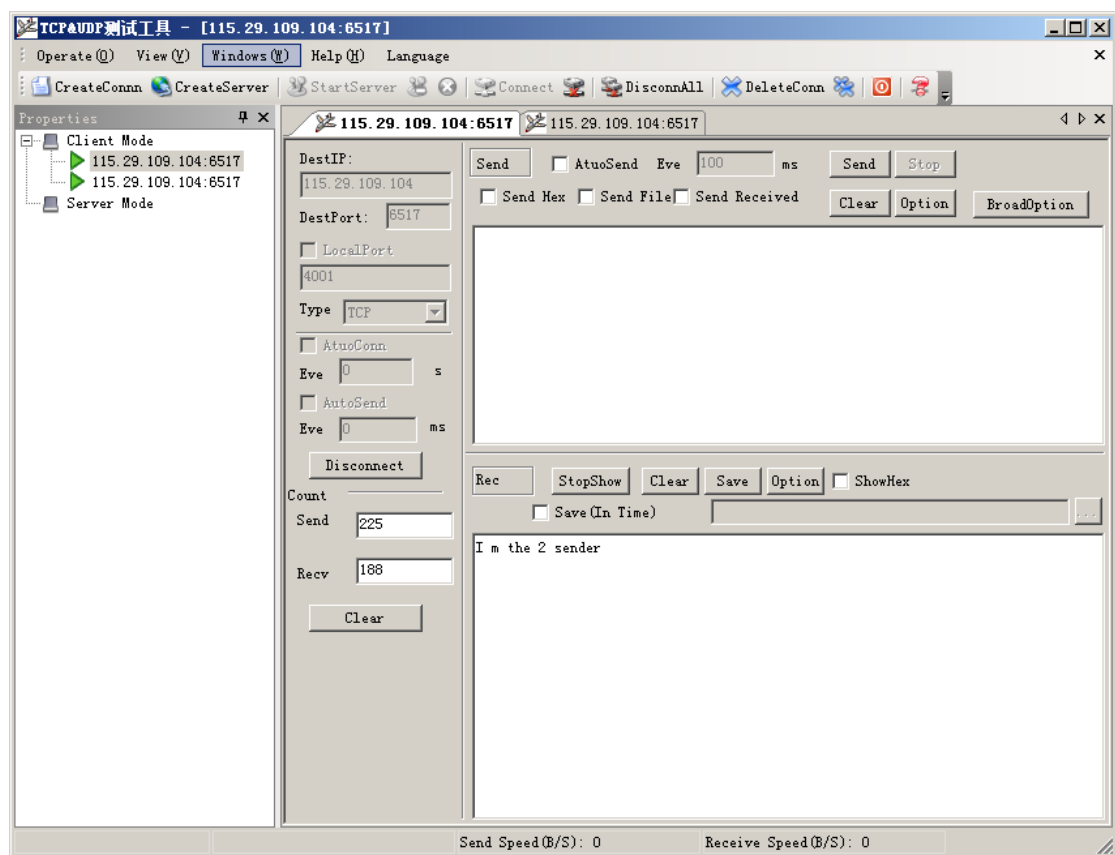


图 9-13 第一个终端收到

因此,可以判定,多个终端通过透传服务器 TCP 调试助手的中继,实现了点到点的信息透传。

9.2.3 源码分析

```
Doit_TCP_Trans_ESPduino $
1  /* Doit tcp test demos
2     version:1.0
3     2016-01-06
4     Copyright (c) 2015 www.doit.am All rights reserved.
5  */
6  /*How to
7     1, Visit "tcp.doit.am" in your brower to get an opened ip and port
8     2, Change "serverIP" and "serverPort" in this sktch, compile and run in ESPduino
9     3, Use a client such as ICPUDP Debugger to setup a tcp client using the ip and port
10    4, Transparent transmission is setup between ESPduino and the tcp client.
11  */
12  #include <ESP8266WiFi.h>
13
14  const char* ssid      = "Doit";
15  const char* password  = "doit3305";
16  const char* serverIP  = "115.29.109.104"; //tcp server的公网ip
17  int serverPort = 6517; //server的端口
18  WiFiClient client;
19  bool bConnected = false;
20  char buff[512];
21  int nm = 0;
22
23  void setup() {
24      Serial.begin(115200);
25      delay(10);
26      Serial.println();
27      Serial.println();
28      Serial.print("Connecting to ");
29      Serial.println(ssid);
30      WiFi.begin(ssid, password);
31      while (WiFi.status() != WL_CONNECTED) {
32          delay(500);
33          Serial.print(".");
34      }
35      Serial.println("");
36      Serial.println("WiFi connected");
37      Serial.println("IP address: ");
38      Serial.println(WiFi.localIP());
39  }
40
```

```

41 void loop() {
42     delay(1);
43     if (bConnected == false)
44     {
45         if (!client.connect(serverIP, serverPort))
46         {
47             Serial.println("connection failed");
48             delay(5000);
49             return;
50         }
51         bConnected = true;
52         Serial.println("connection ok");
53     }
54     else if (client.available())
55     {
56         Serial.println("Data is coming");
57         while (client.available())
58         {
59             buff[nm++] = client.read();
60             if (nm >= 511) break;
61             //缓冲区设为512个字节，若满则退出循环，重头开始写入
62         }
63         buff[nm] = 0x00;
64         nm=0;
65         Serial.print(buff);
66         client.print(buff);
67         client.flush();
68     }
69 }
70

```

TCP 是流式协议，在传输时源源不断地，分不清开始和结束的字节，每次的收发量不确定。TCP 协议在提交有效数给应用层时，是以字节形式上报给用户，几乎所有的 TCP 传输数据都会面临在接收 1 个大数据包时，可能会被拆分成多个数据包发送。多次 Send 底层也可能会合并成一次进行发送。所以用 TCP 作为通讯协议，应用层就需要 2 个操作来解决：

- 1.分包：Server 收到了多个数据包，需要拆分数据包
- 2.合包：Server 收到的数据只是包的一部分，需要缓存数据，合并成完整的包

针对流式这个问题，我们可以

- 1.每次发包时，在你的包前面加上长度，每次收发包时都处理这个长度。固定包头的协议非常通用，这种协议的特点是一个数据包总是由包头+包体 2 部分组成。包头由一个字段指定了包体或整个包的长度，长度一般是使用 2 字节整数来表示

2. EOF 协议处理的原理是每个数据包结尾加一串特殊字符表示包已结束，通常是 `\r\n` 等不常用字符。

第 10 章 UDP 服务

10.1 UDP Server

UDP 协议相比 TCP 协议，具有连接速度快，占用资源少的特点，因此在不太要求连接质量的场合应用也是很广泛的，它的实现也更为简单，因此本章就在明确区分 UDP Server 和 Client，一般地说，实现监听 UDP 包的就是 server，发送 UDP 包的就是 Client，在一个应用中，这二者往往同时具备。

10.2 实验步骤

1. 连接到 Doit 热点，启动 UDP 服务，UDP 端口为 8000，如下所示：

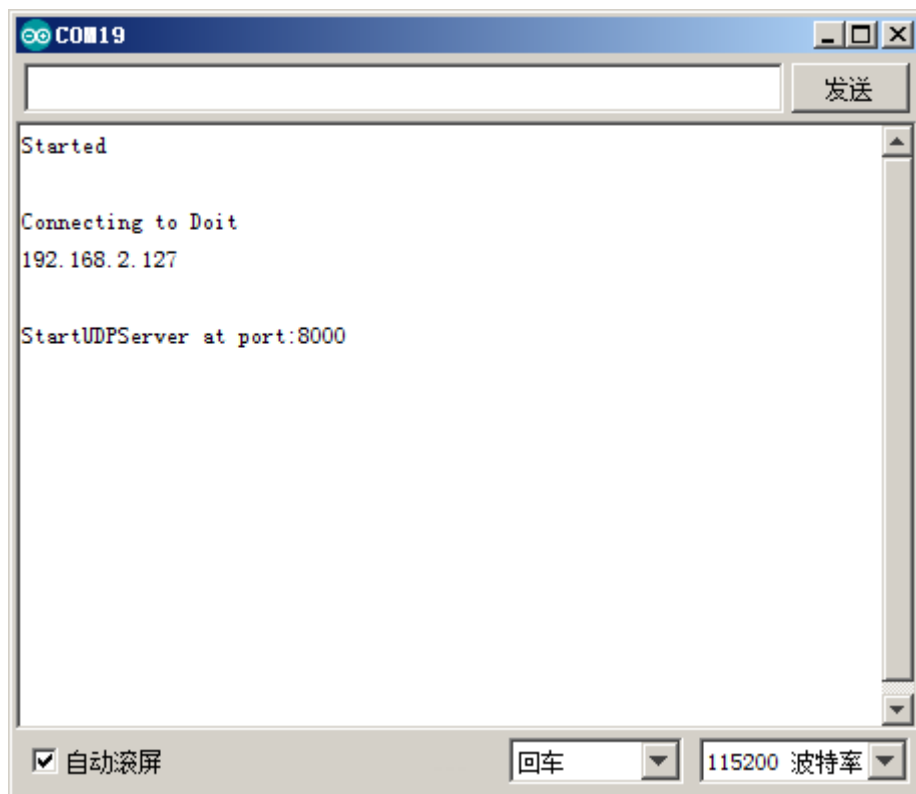


图 10-1 UDP 服务启动

2. 使用 TCP/UDP 测试工具，建立一个 UDP 的测试客户端：

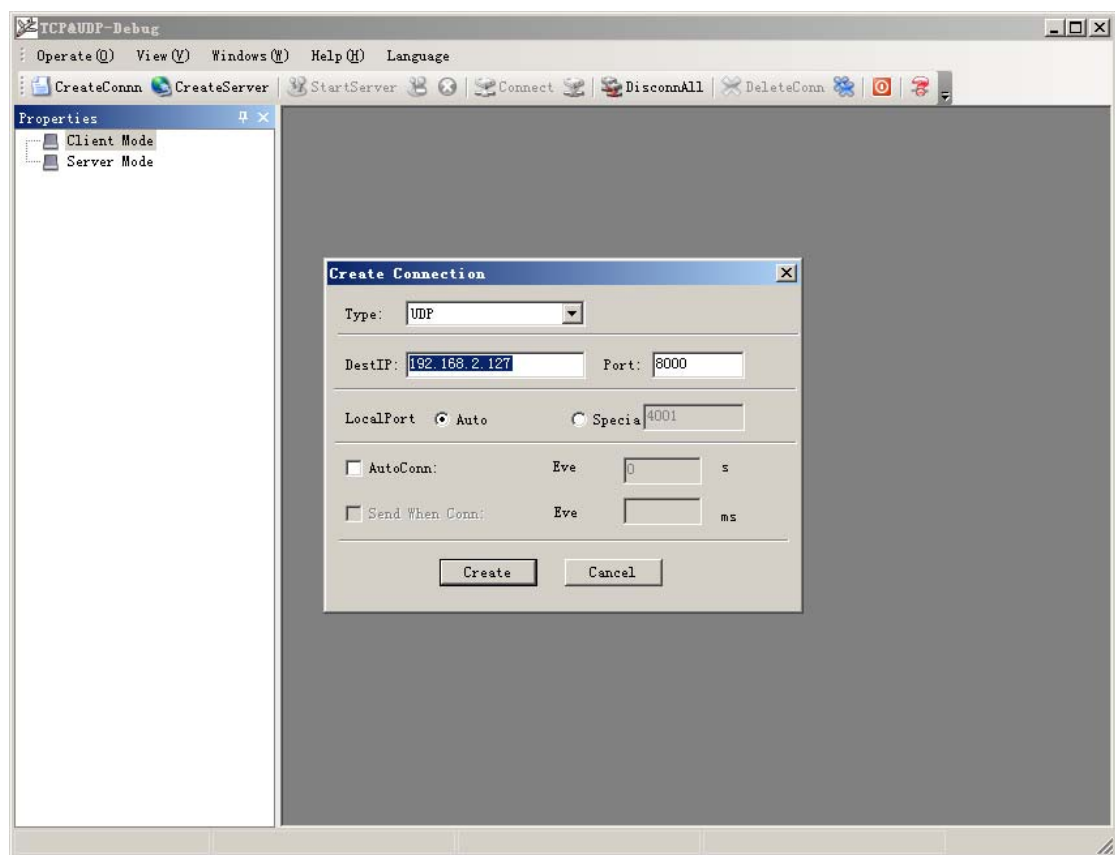


图 10-2 创建 UDP 测试客户端

3. 建立成功后如下图所示，其实和 TCP 测试几乎一样的：

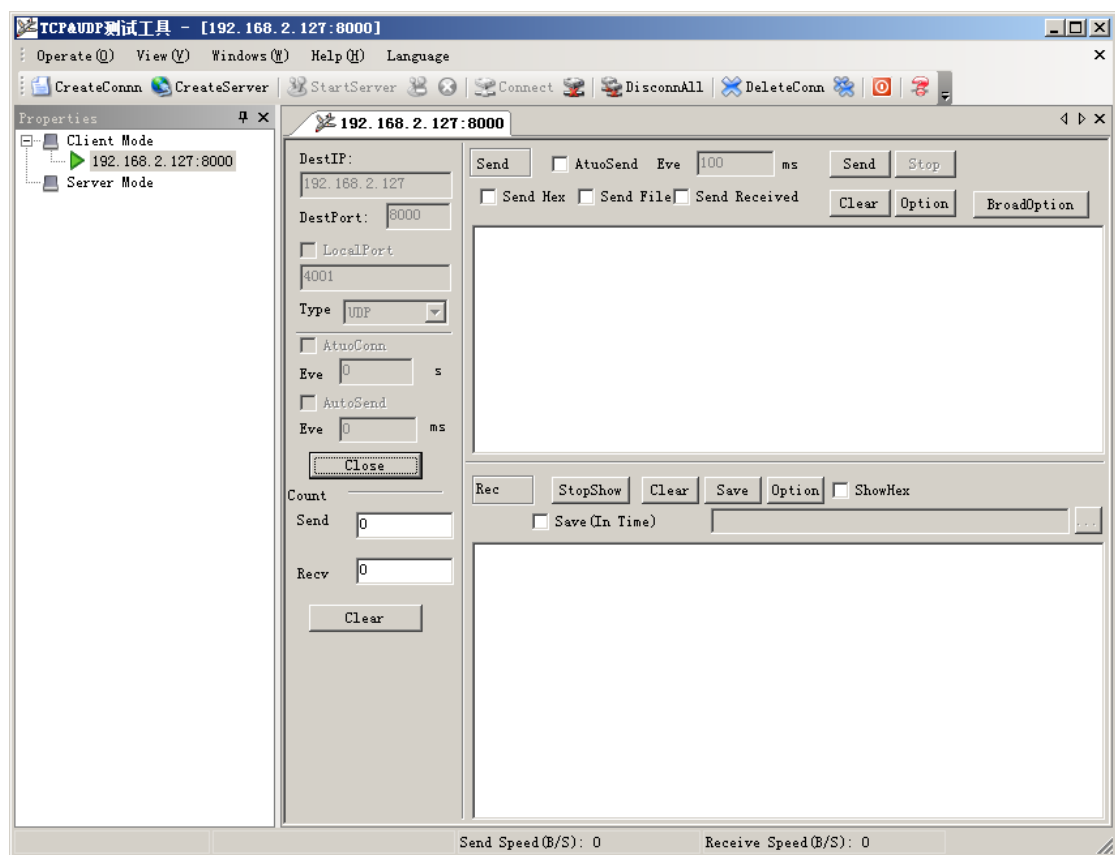


图 10-3 创建 UDP 连接

4. 在发送窗口输入“u can do it”，

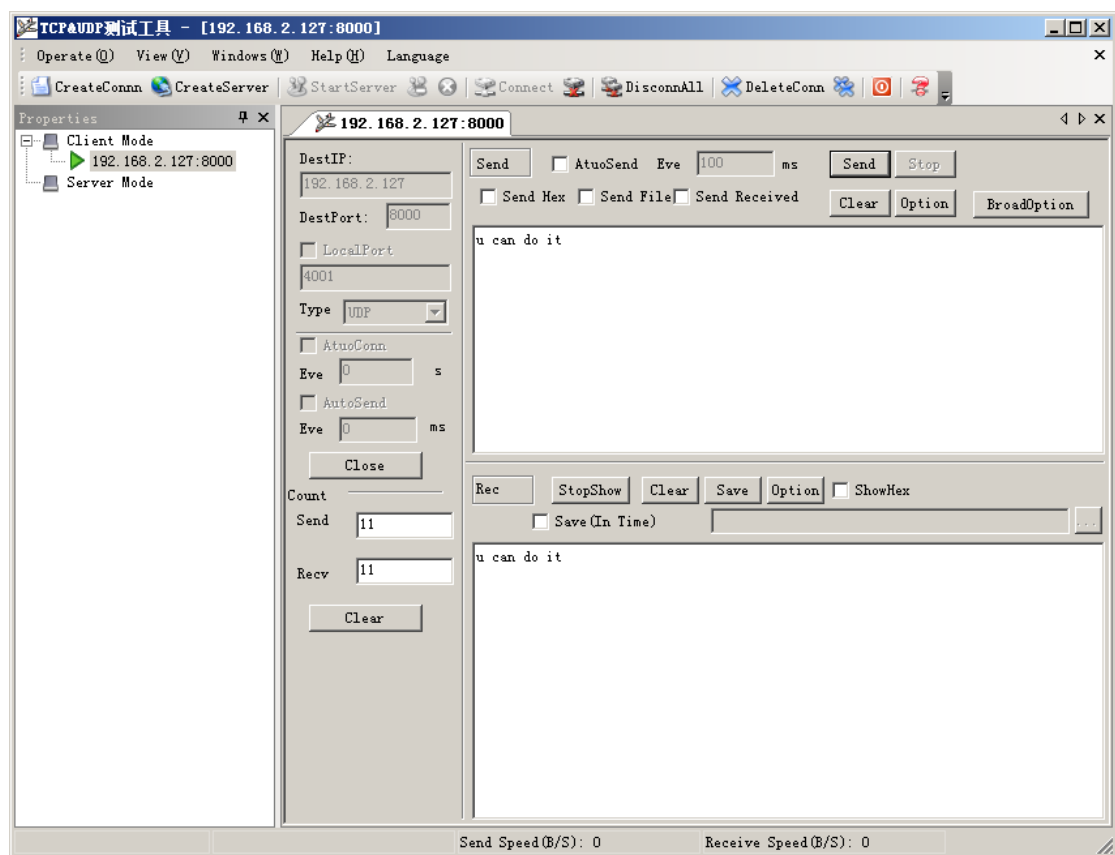
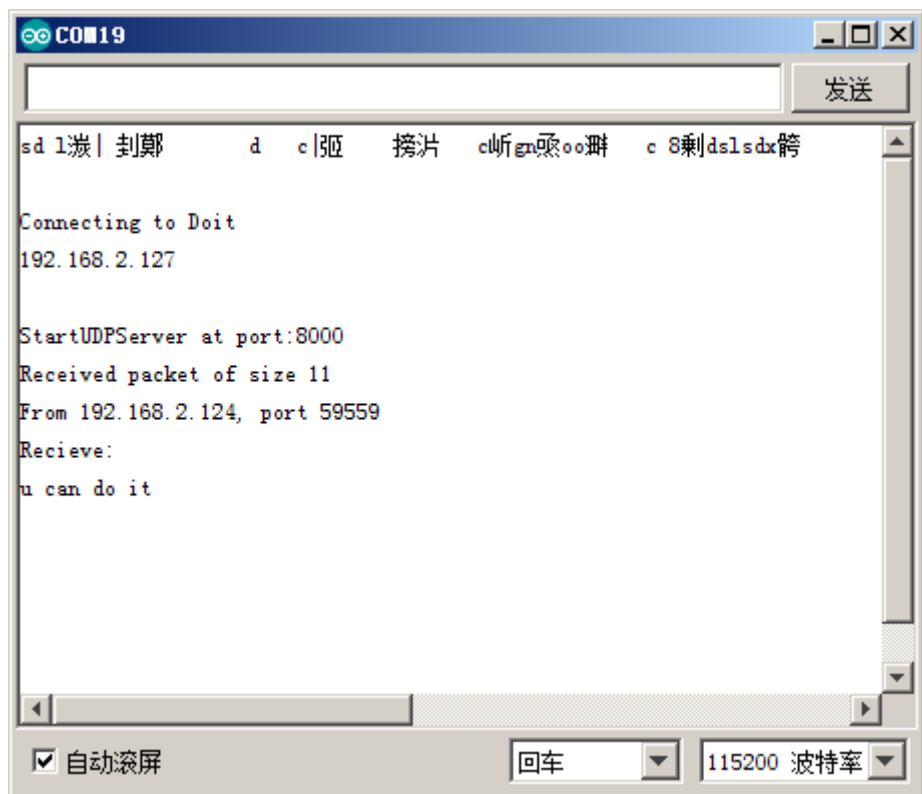


图 10-4

此时，串口监视器可以看到“u can do it”被接收了，说明 UDP 服务器工作正常。



10.3 源码分析

代码中，udp.begin()和 sendUDP()分别实现了 UDP 服务监听和 UDP 包发送的功能。

```
udp $
1  /* udp demo --udp server at port 8000
2   version:1.0
3   2016-01-05
4   Copyright (c) 2015 www.doit.am All rights reserved.
5   */
6
7   #include <ESP8266WiFi.h>
8   #include <WiFiUdp.h>
9
10  #define MAX_PACKETSIZE 512 //定义UDP包的最大值为512个字节
11  WiFiUDP udp; //udp服务器对象
12  const char* ssid = "Doit";
13  const char* password = "doit3305";
14  char buffUDP[MAX_PACKETSIZE]; //定义UDP缓冲区
15
16  void startUDPServer(int port)
17  {
18      Serial.print("\r\nStartUDPServer at port:");
19      Serial.println(port);
20      udp.begin(port);
21  }
22  void sendUDP(char *p)
23  {
24      udp.beginPacket(udp.remoteIP(), udp.remotePort());
25      udp.write(p);udp.endPacket();
26  }
27  void doUdpServerTick()
28  {
29      int packetSize = udp.parsePacket();
30      if(packetSize)
31      {
32          Serial.print("Received packet of size ");
33          Serial.println(packetSize);
34          Serial.print("From ");
35          IPAddress remote = udp.remoteIP();
36          for (int i = 0; i < 4; i++) { //允许最多4个客户端的udp连接
37              Serial.print(remote[i], DEC);
38              if (i < 3) Serial.print(".");
```

```

39     }
40     Serial.print(", port ");
41     Serial.println(udp.remotePort());
42     memset(buffUDP, 0x00, sizeof(buffUDP));
43     udp.read(buffUDP, MAX_PACKETSIZE-1);
44     udp.flush();
45     Serial.println("Recieve:");
46     Serial.println(buffUDP);
47     sendUDP(buffUDP); //回送
48 }
49 }
50
51 void setup() {
52     Serial.begin(115200);
53     Serial.println("Started");
54     WiFi.begin(ssid, password);
55     //WiFi.softAP("ESP8266_WIFI");
56     Serial.print("\nConnecting to "); Serial.println(ssid);
57     uint8_t i = 0;
58     while (WiFi.status() != WL_CONNECTED && i++ < 20) delay(500);
59     if(i == 21){
60         Serial.print("Could not connect to"); Serial.println(ssid);
61         while(1) delay(500);
62     }
63     Serial.println(WiFi.localIP());
64     startUDPServer(8000);
65 }
66
67 void loop() {
68     doUdpServerTick();
69     delay(1);
70 }

```

与第 9 章介绍的 TCP 不同的是，UDP 是基于报文的，在接收的时候，每次最多只能读取一个报文，报文和报文是不会合并的，如果缓冲区小于报文长度，则多出的部分会被丢弃。UDP 是无连接的协议，也就是说，只要知道接收端的 IP 和端口，且网络是可达的，任何主机都可以向接收端发送数据。UDP 数据报协议是面向无连接的，每个数据包都有完整的源、目的地址及分组编号，各自在网络中独立传输，传输中不管其顺序，数据到达收端后再进行排序组装，遇有丢失、差错和失序等情况，通过请求重发来解决。

第 11 章 WebServer，远程控制硬件

11.1 连接人和硬件的桥梁

在超视距环境下，人们总想对远程的硬件进行控制并传送应用数据。一个自然的想法就是将此硬件设置为一个服务器，通过浏览器这种较为便捷的方式实现对远程硬件的控制和数据推送。要满足这类需求往往需要设定一些特别的协议来标识，或称 URI（Uniform Resource Identifier，统一资源标识符），而服务器应具有解析这种 URI 的能力。按如下方式设计，ESPduino 就可以作为一个 Web Server，来响应远程的控制指令，并推送相应数据。

11.2 实验步骤

1. 开发版加电，上传固件代码，串口监视显示连网成功，Web 服务器启动：

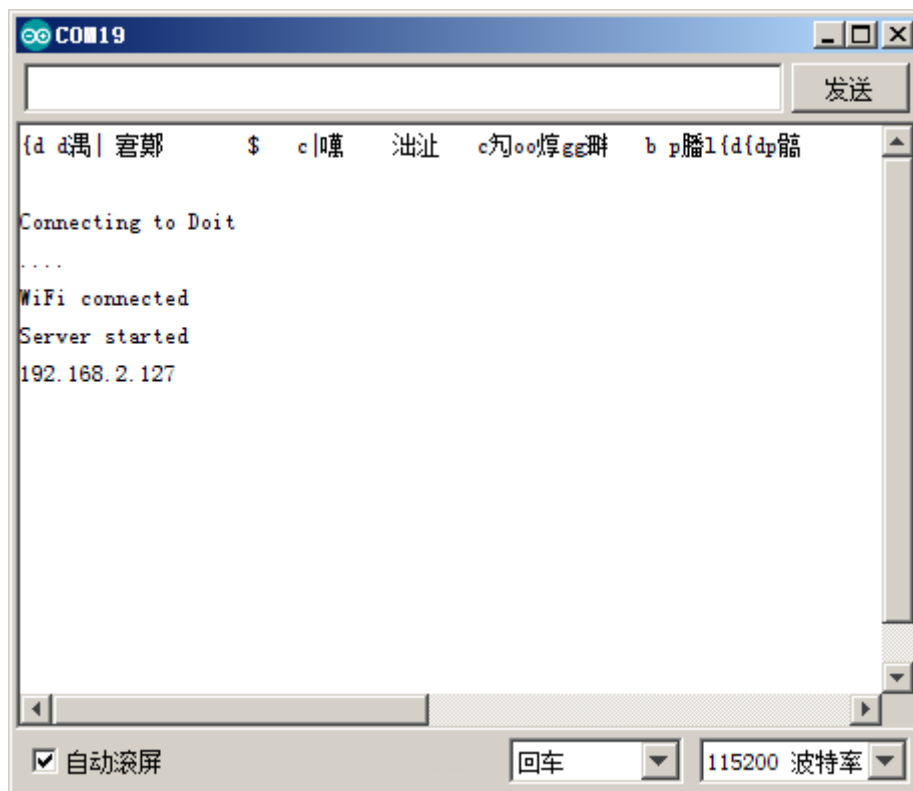


图 11-1 Web 服务器启动

2. 在浏览器中输入 ip 地址:

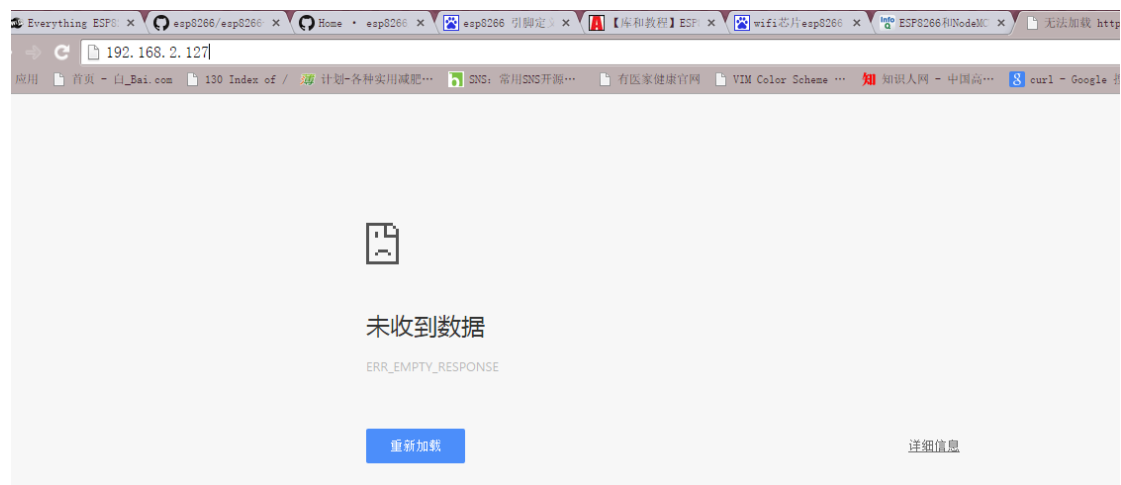


图 11-2

提示未收到数据，观察串口监视器，发现客户端是已经成功连接了——new client，但 HTTP 的 GET 请求是无效的——invalid request:

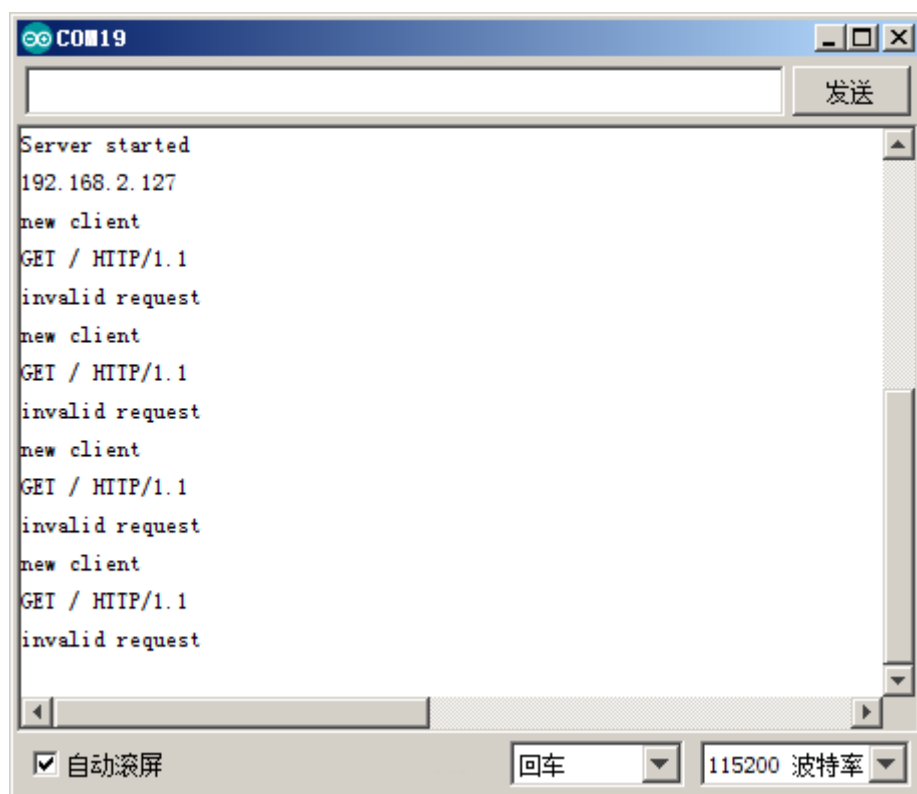
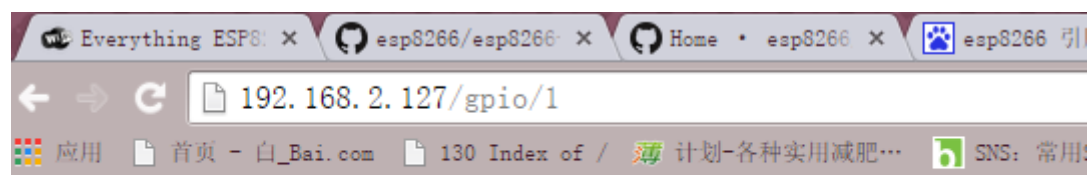


图 11-3 无效请求信息

显然，这是由于 HTTP 地址没有写对，需要按照源码规定的协议格式进行输入。

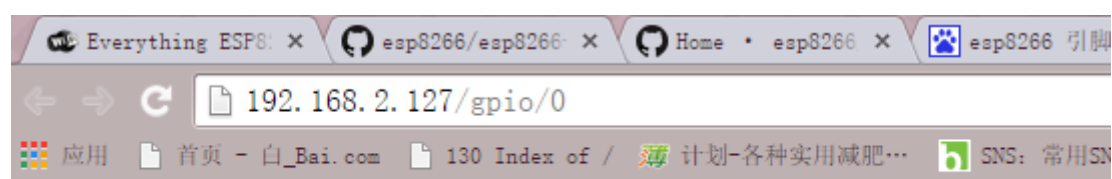
3. 我们重新在浏览器中填写 HTTP 地址如下所示:



GPIO is now high

图 11-4 使用正确协议格式访问服务器

GPIO (General Purpose Input Output, 通用输入输出) 是嵌入式系统最简单也是最常用的资源, 是各种控制指令/响应的硬件接口, 通过高低电平实现具体控制。本例中, /gpio/1 表示把 GPIO 接口置为高电平, 这是可以观察到, LED 灯灭了;



GPIO is now low

图 11-5 远程改变电平

类似的, 可以通过/gpio/0 将 LED 灯点亮。

11.3 源码分析

```
webServer$
1  /*
2   * This sketch demonstrates how to set up a simple HTTP-like server.
3   * The server will set a GPIO pin depending on the request
4   *   http://server_ip/gpio/0 will set the GPIO2 low,
5   *   http://server_ip/gpio/1 will set the GPIO2 high
6   * server_ip is the IP address of the ESP8266 module, will be
7   * printed to Serial when the module is connected.
8   */
9  #define PIN_LED 16
10 #define KEY_FLASH 0
11
12 #include <ESP8266WiFi.h>
13
14 const char* ssid = "Doit";
15 const char* password = "doit3305";
16
17 //创建一个服务器实例，并定义监听端口号80作为参数
18 WiFiServer server(80);
19
20 void setup() {
21     Serial.begin(115200);
22     delay(10);
23     //准备GPIO2
24     pinMode(PIN_LED, OUTPUT);
25     digitalWrite(PIN_LED, LOW);
26
27     // 连接到 WiFi网络
28     Serial.println();
29     Serial.println();
30     Serial.print("Connecting to ");
31     Serial.println(ssid);
32
33     WiFi.begin(ssid, password);
34
35     while (WiFi.status() != WL_CONNECTED) {
36         delay(500);
37         Serial.print(".");
38     }
39     Serial.println("");
40     Serial.println("WiFi connected");
```

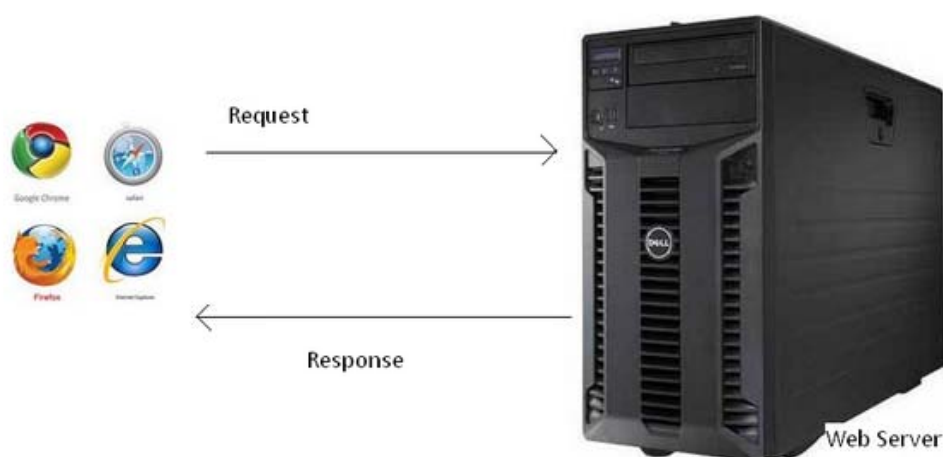
```

41
42 //开始提供web服务
43 server.begin();
44 Serial.println("Server started");
45
46 //输出IP address
47 Serial.println(WiFi.localIP());
48 }
49
50 void loop() {
51 // 检测是否有客户端连上
52 WiFiClient client = server.available();
53 if (!client) {
54     return;
55 }
56 //等待客户端发送数据
57 Serial.println("new client");
58 while(!client.available()){
59     delay(1);
60 }
61
62 // 读取请求的第一行
63 String req = client.readStringUntil('\r');
64 Serial.println(req);
65 client.flush();
66
67 //匹配不同的请求类型
68 int val;
69 if (req.indexOf("/gpio/0") != -1)
70     val = 0;
71 else if (req.indexOf("/gpio/1") != -1)
72     val = 1;
73 else {
74     Serial.println("invalid request");
75     client.stop();
76     return;
77 }
78
79 // 根据请求内容设置GPIO2的具体值
80 digitalWrite(PIN_LED, val);
81
82 client.flush();
83
84 // 准备响应信息
85 String s = "HTTP/1.1 200 OK\r\nContent-Type: text/html\r\n\r\n<!DOCTYPE HTML>\r\n<html>\r\nGPIO is
86 s += (val)? "high": "low";
87 s += "</html>\n";
88
89 // 将响应信息发送给客户端
90 client.print(s);
91 delay(1);
92 Serial.println("Client disconnected");
93
94 //当函数返回并且client对象被撤销时，客户端真正端口连接
95 // when the function returns and 'client' object is destroyed
96 }
97

```

下面就 HTTP 协议进行简单的讲解，协议是指计算机通信网络中两台计算机之间进行通

信所必须共同遵守的规定或规则，超文本传输协议(HTTP)是一种通信协议，它允许将超文本标记语言(HTML)文档从 Web 服务器传送到客户端的浏览器，目前我们使用的是 HTTP/1.1 版本。当我们打开浏览器，在地址栏中输入 URL，然后我们就看到了网页。原理是怎样的呢？实际上我们输入 URL 后，我们的浏览器给 Web 服务器发送了一个 Request, Web 服务器接到 Request 后进行处理，生成相应的 Response，然后发送给浏览器，浏览器解析 Response 中的 HTML,这样我们就看到了网页，过程如下图所示【3】。



HTTP 是基于 TCP 协议的应用层协议用\r\n\r\n来分割消息头和消息体，HTTP 请求的消息头中有 Content-Length 来告知消息体有多大，如果没有该字段就表示无消息体，GET 请求大多是这样。HTTP 响应的消息头中，或者有 Content-Length，或者有 Transfer-Encoding: chunked 告知以 chunk 模式分析消息体。一个典型的请求如下所示：

HTTP 请求

```
GET / HTTP/1.1\r\nAccept: image/gif, ..., */*\r\nAccept-Language: zh-cn\r\nUA-CPU: x86\r\nAccept-Encoding: gzip, deflate\r\nUser-Agent: Mozilla/4.0 (compatible; Think\r\nHost: www.soso.com\r\nConnection: Keep-Alive\r\nCookie: suid=748391392; pgv_pvid=8070695194\r\n\r\n
```

HTTP 响应 1: Content-Length

```
HTTP/1.1 200 OK
Server: SWS/1.0
Date: Thu, 29 Jul 2010 07:38:51 GMT
Content-Type: image/gif
Content-Length: 43
Last-Modified: Tue, 28 Oct 2008 07:00:22 GMT
Connection: keep-alive
Expires: Thu, 29 Jul 2010 07:38:51 GMT
Cache-Control: max-age=0
Accept-Ranges: bytes

GIF89a 二进制数据
```

HTTP 是基于 TCP 的，采用 `\r\n\r\n` 来分割消息头和消息体，这种用特定字符/字符串来分割或分包的方式，还有不少协议用到。例如 FTP/SMTP/POP3 都是用 `\n` 来作为一个命令结束的标志。这种消息分包的方式，需要应用层去扫描已 `recv` 到的数据，性能上还不够高效，代码不严谨的还容易被攻击。在需要自定义协议的项目中，不少选择用二进制协议，解析高效，安全性更好些。这种分隔方式在第 9 章末尾进行了详细的说明，有不清楚的可以翻阅。

常见问题：

1. GPIO 的例子测试失败，访问 <http://172.27.35.12/gpio/1>，串口打印出

```
GET /favicon.ico HTTP/1.1
```

```
invalid request
```

原因：浏览器每次发起请求，都会同时请求一次 `favicon.ico`（如果浏览器没有缓存 `favicon.ico`），对图标的请求是由浏览器发起的，需要在处理程序中进行处理。在书的例子中，为了集中阐述 GPIO，没有对这个情况进行处理。

第 12 章 通过 Web OTA 升级固件

12.1 什么是 web OTA

利用 OTA (Over the Air)进行更新是指在对 ESP 模块固件更新时使用 WiFi 连接而非串口连接进行。由于和模块没有物理接触，避免了很多局限性，因此是一个十分有用的功能。下图展示了 OTA 的过程：

1. 始时，当前程序（current sketch，绿色所示）和数据区（SPI Flash File System,SPIFFS 用于嵌入式系统地一种闪存文件，蓝色所示）位于闪存空间的不同区域，中间空白的是空闲空间；
2. 在更新时，新的程序（new sketch，黄色所示）空闲区的最大可用地址开始，将新程序写入空闲空间，此时闪存中同时保留了这三种对象；
3. 在模块启动过程中，新的程序再从原来当前程序的开始地址处写入，替换旧程序，以达到固件更新的目的。



图 12-1 升级原理

12.2 实验步骤

12.2.1 设置固件环境参数

1. 这一步要注意选择闪存空间为 4M，数据区需要预留 3M 的菜单选项。

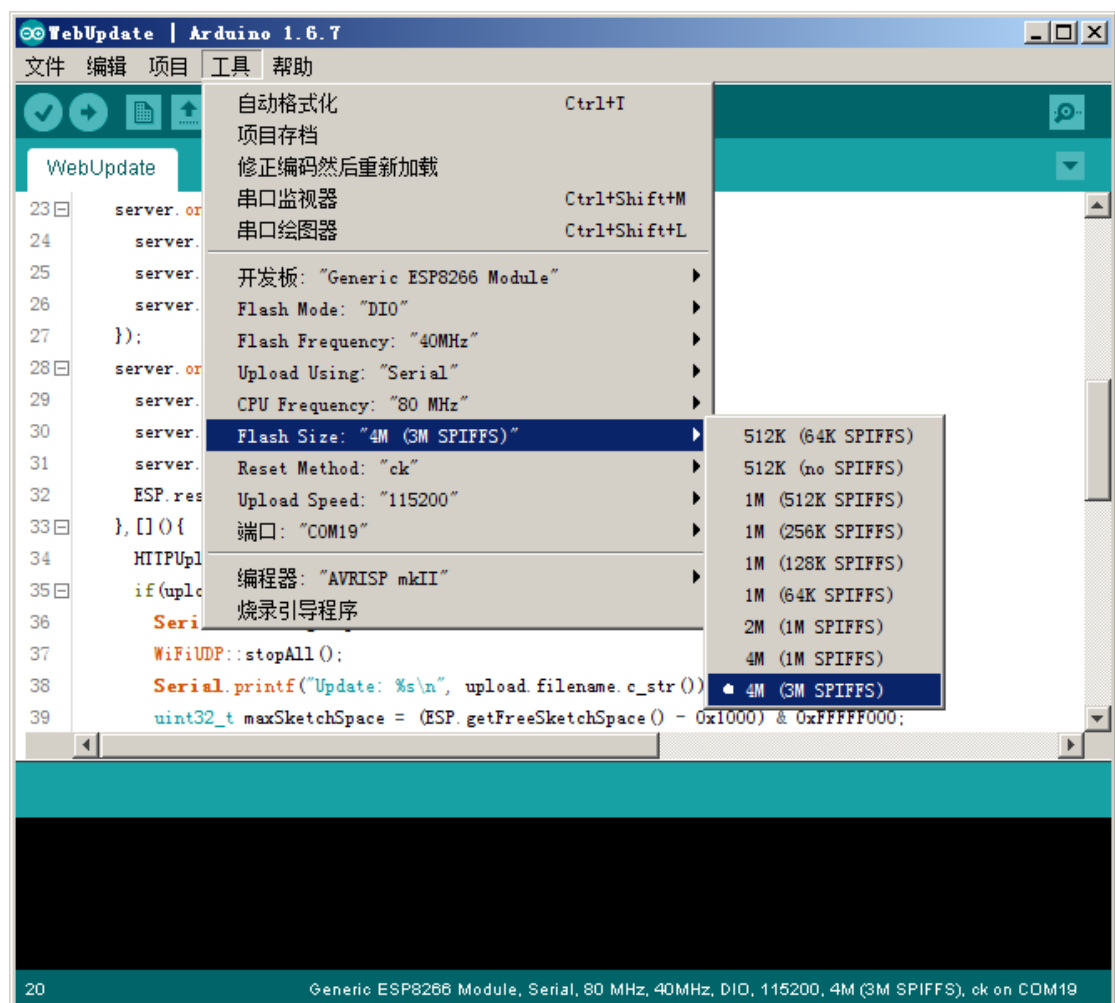
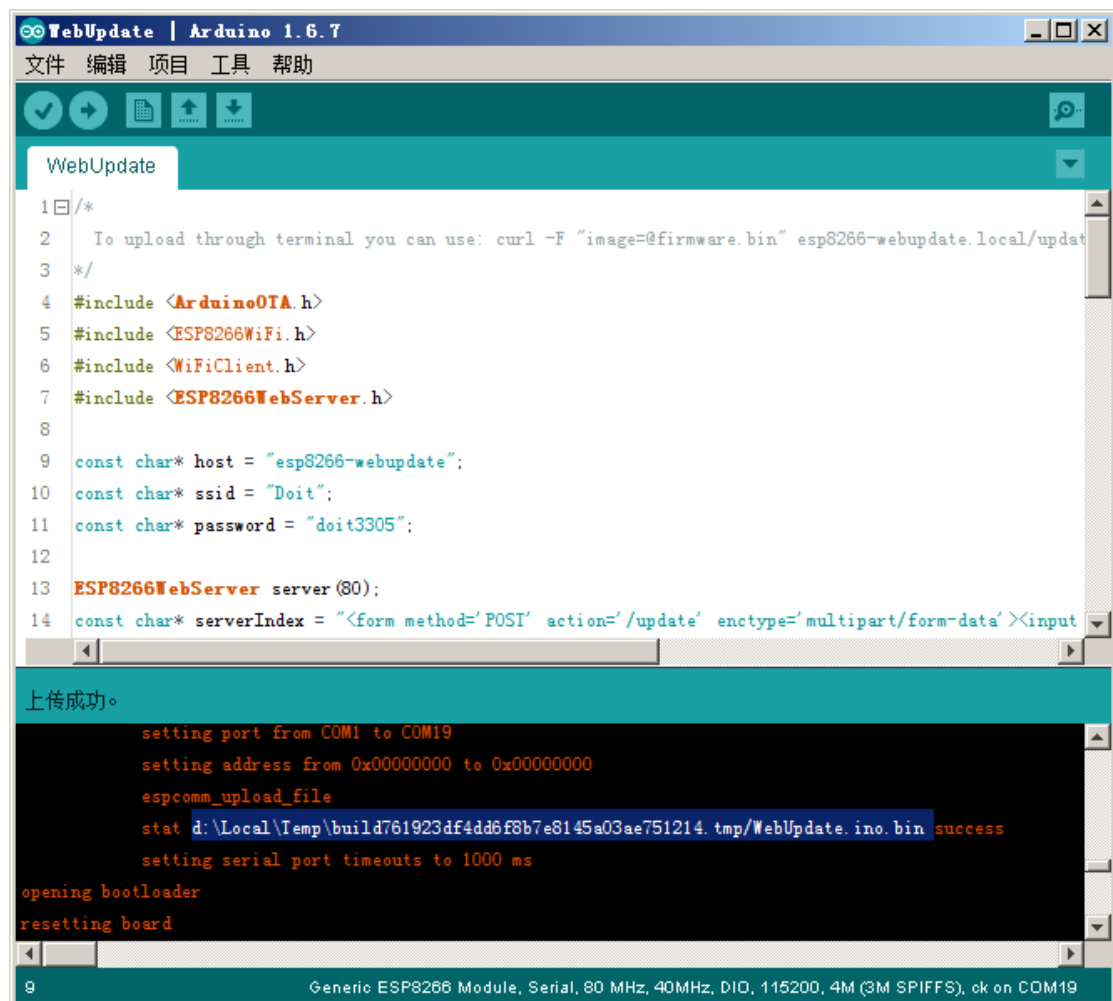


图 12-2 调整首选项

2. 在编译时要注意编译窗口返回的编译信息，里面包含了编译好的最新版固件程序（.bin 文件）所在本地的位置，如下图标蓝所示，找到此文件另存在易于查找的目录下，后面要用。



3. 观察串口监视窗，在加电启动时，如果 WiFi 不可用或更新服务器不可达，显示如下：

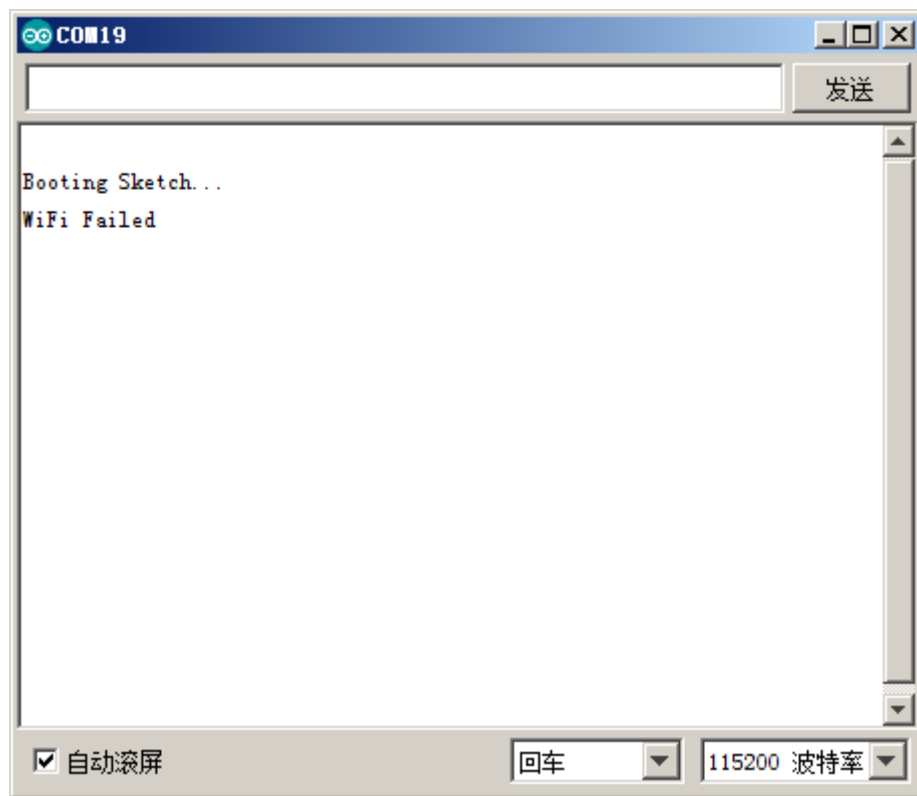
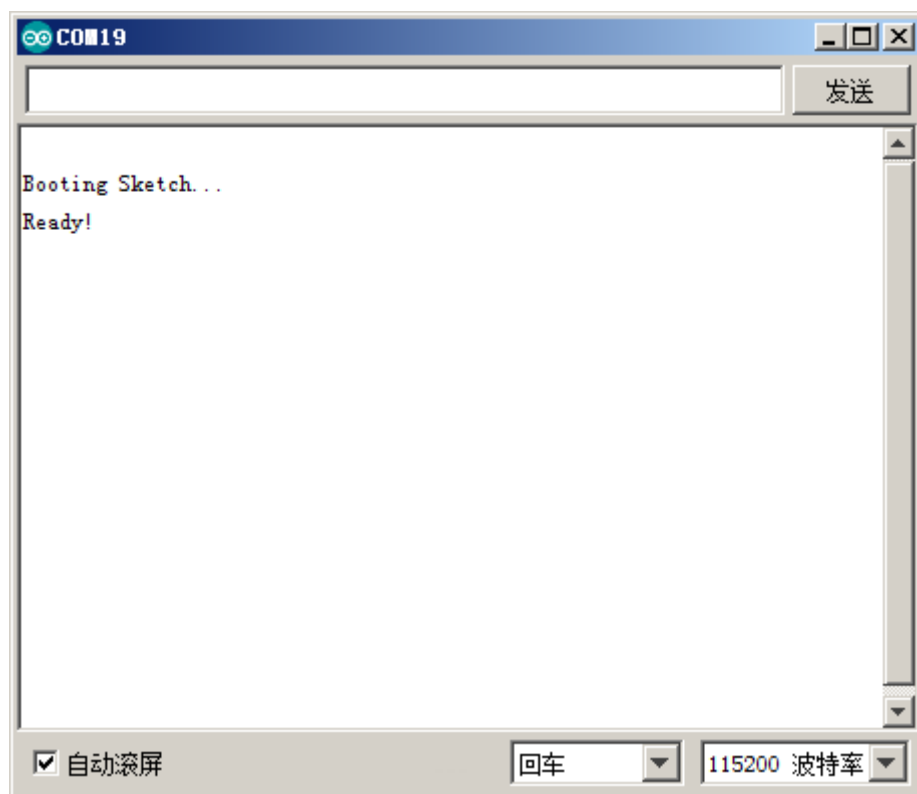


图 12-3 WiFi 无法连接

反之，表示一切就绪，可以进行 OTA 升级。



12.2.2 使用 STA 模式

1. 在上一节中已经把 ESP 模块置为 Web OTA 模式准备升级，此时模块是 AP+STA 的混杂模式，我们可以在浏览器中输入 STA 的 IP 地址，显示如下

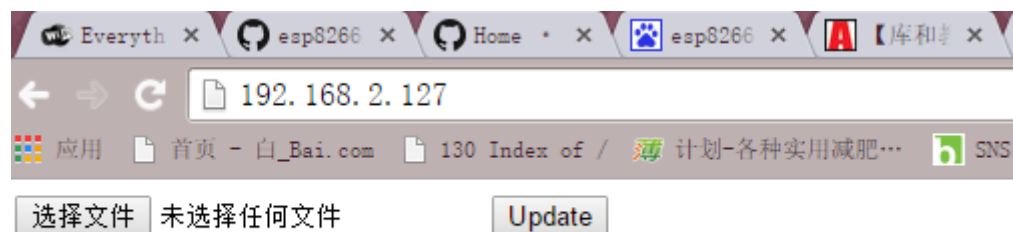


图 12-4 STA 方式的访问

2. 点击选择文件，从打开窗口中找到 12.2.1 的第 2 步所保存的更新文件位置，并选择：

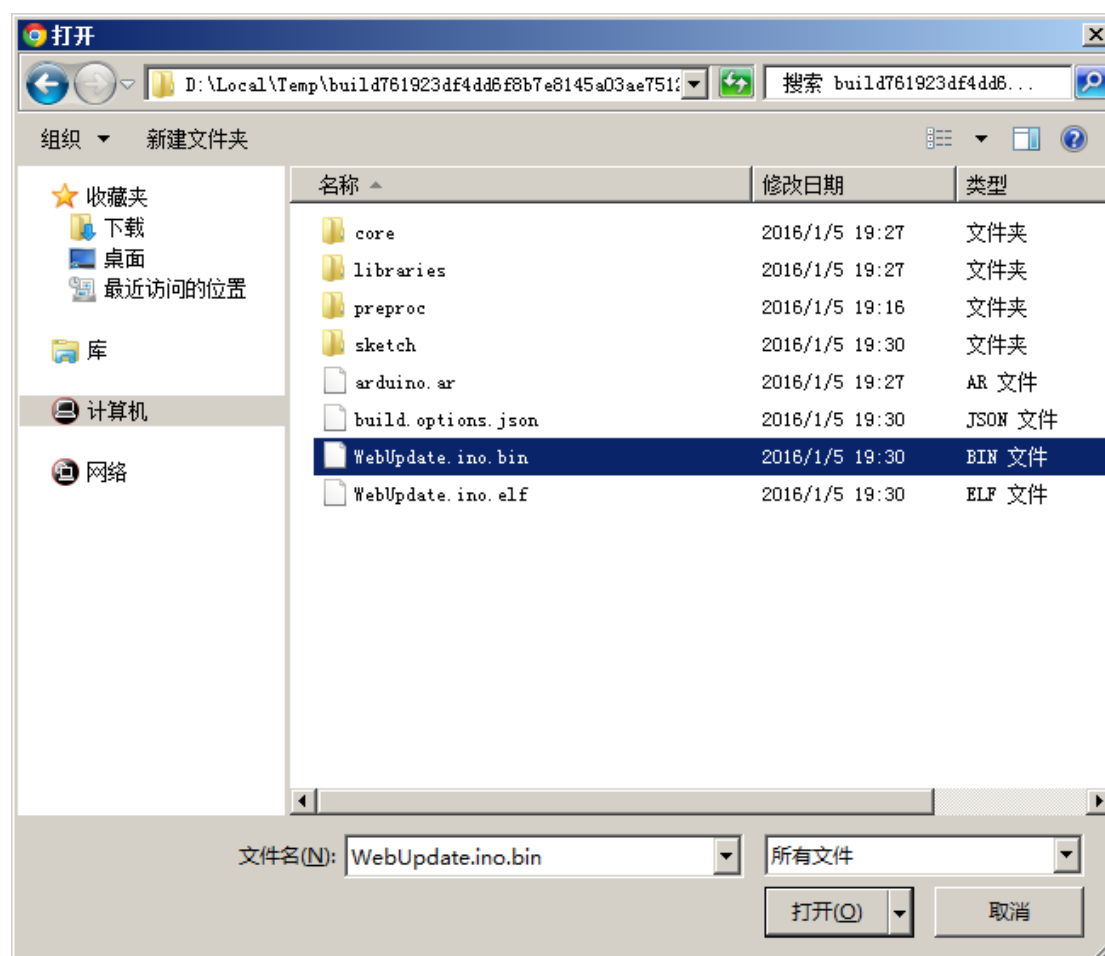


图 12-5 打开更新文件位置

显示如下，标明更新文件已准备好：

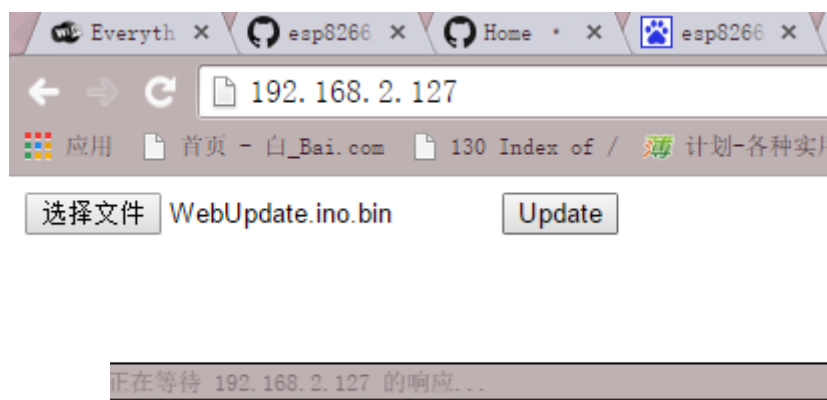


图 12-6 上传更新文件界面

3. 点击上图的 Update, 此 ESP 模块响应后, 更新文件就可以 HTTP Post 的方式上传到芯片的闪存空间, 返回更新状态 “ok”, 完成更新过程, 如下图所示:

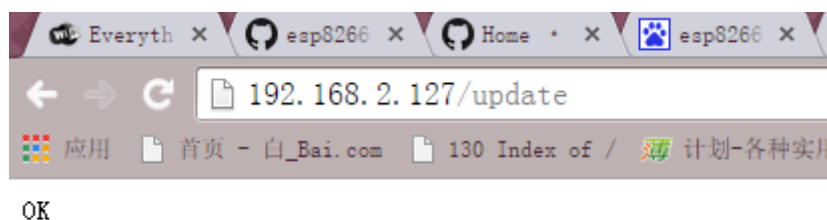
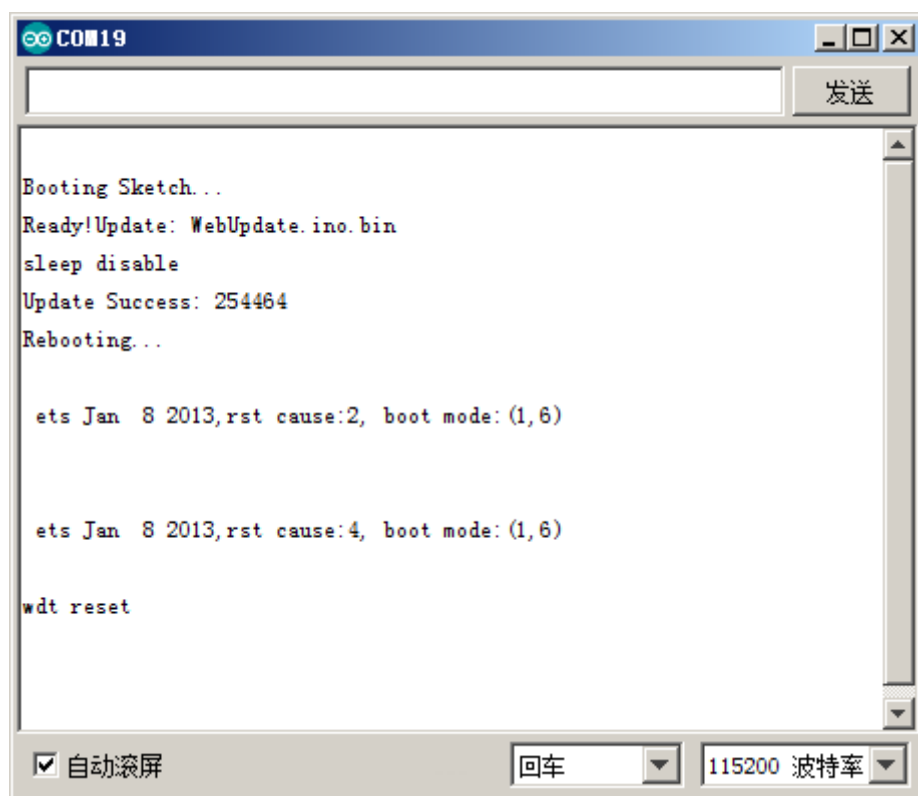


图 12-7 更新完毕

在此过程中, 我们可以观察到串口监视窗的输出内容如下:



其包含了更新文件名、更新状态、文件大小等详细信息。

12.2.3 使用 AP 模式

1. 除 STA 模式之外，如果使用 ESP 模块本身的 AP 地址，也可以实现 Web OTA，此时需要下位机选择此 AP 的 SSID 作为无线网络连接的标识，如下图所示：



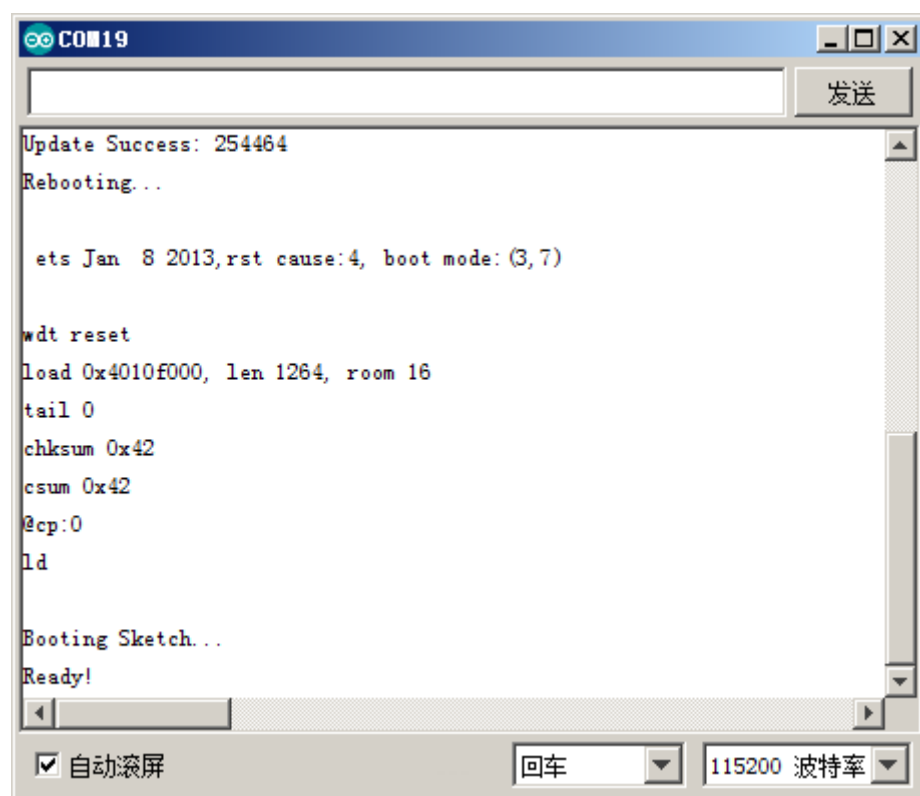
图 12-8 连接 ESP 的 AP 模式

2. 选择文件后的上传过程，后续操作基本一致，注意浏览器 IP 地址的变化：



图 12-9 使用 AP 本身地址升级

3. 串口监视器中的内容，包括加载地址和校验和等更多细节：



12.3 源码分析

```
WebUpdate $
1 /*
2  可以通过终端，使用如下方式： curl -F "image=@firmware.bin" esp8266-webupdate.local/update 进行
3  ESP8266的固件升级
4  */
5  #include <ArduinoOTA.h>
6  #include <ESP8266WiFi.h>
7  #include <WiFiClient.h>
8  #include <ESP8266WebServer.h>
9
10 const char* host = "esp8266-webupdate";
11 const char* ssid = "Doit";
12 const char* password = "doit3305";
13
14 ESP8266WebServer server(80);
15 const char* serverIndex = "<form method='POST' action='/update' enctype='multipart/form-data'>
16   <input type='file' name='update'><input type='submit' value='Update'></form>";
17 //设定服务器的HTML升级页面
18
19 void setup(void){
20   Serial.begin(115200);
21   Serial.println();
22   Serial.println("Booting Sketch...");
23   WiFi.mode(WIFI_AP_STA);
24   WiFi.begin(ssid, password);
25   if(WiFi.waitForConnectResult() == WL_CONNECTED){
26     server.on("/", HTTP_GET, [](){
27       server.sendHeader("Connection", "close");
28       server.sendHeader("Access-Control-Allow-Origin", "*");
29       server.send(200, "text/html", serverIndex);
30     });
31     server.on("/update", HTTP_POST, [](){
32       server.sendHeader("Connection", "close");
33       server.sendHeader("Access-Control-Allow-Origin", "*");
34       server.send(200, "text/plain", (Update.hasError())?"FAIL":"OK");
35       ESP.restart();
36     }, [](){
37       HTTPUpload& upload = server.upload();
38       if(upload.status == UPLOAD_FILE_START){
39         Serial.setDebugOutput(true);
40         WiFiUDP::stopAll();
41         Serial.printf("Update: %s\n", upload.filename.c_str());
42         uint32_t maxSketchSpace = (ESP.getFreeSketchSpace() - 0x1000) & 0xFFFFF000;
43         if(!Update.begin(maxSketchSpace)){//从最大可用地址开始更新
44           Update.printError(Serial);
45         }
46       } else if(upload.status == UPLOAD_FILE_WRITE){
47         if(Update.write(upload.buf, upload.currentSize) != upload.currentSize){
48           Update.printError(Serial);
```

```
49     }
50 } else if(upload.status == UPLOAD_FILE_END){
51     if(Update.end(true)){ //当前更新过程结束
52         Serial.printf("Update Success: %u\nRebooting...\n", upload.totalSize);
53     } else {
54         Update.printError(Serial);
55     }
56     Serial.setDebugOutput(false);
57 }
58     yield();
59 });
60     server.begin();
61     Serial.printf("Ready!", host);
62 } else {
63     Serial.println("WiFi Failed");
64 }
65 }
66
67 void loop(void){
68     server.handleClient();
69     delay(1);
70 }
```

第 13 章 一个准产品的例子

13.1 产品概述

迄今为止,我们已经可以使用前面介绍过的技术来构造一个将设备和互联网紧密联系起来的方案,这种联系可以称为物联网的设备发现,设备之间实现通信,也就是通信行业里大名鼎鼎的 M2M (Machine to Machine, 有时也叫做 Machine to Man) 功能,这也是整个物联网生态的基础。概括来讲,就是希望嵌入了 ESPDuino 的硬件模块/设备,利用前述的若干原理和技术,通过 Web 服务器,实现设备发现和设备间通信。

13.2 功能描述

本节的例子介于玩家和商业产品之间,其功能已经可以满足大多数业务需求。实现功能如下:

1. 注册
2. 登录
3. 获取时间
4. 数据转发
5. 注销

采用 TCP 连接(IP:book.doit.am 端口: 5588), 数据格式如下:

#Type#Device_id#DATA\r\n

服务器通过“#”来进行字符分隔,因此在 User 和 DATA 字段应当避免包含“#”,如果包含,需要自行使用转移字符处理。

表 13-1 协议说明

Type	备注	分隔符	Device_id	备注	分隔符	Data	备注
0	新用户注册	#	*****	需要注册的设备名	#	*****	注册设备

1	登陆	#	*****	登录的设备名	#	*****	密码
2	发送实时数据，目标不在线，直接抛弃数据。	#	*****	目标用户名	#	*****	数据
3	注销用户，必须登录，才能注销登录用户。	#			#		
4	获取系统时间	#			#		
5	保持心跳	#					

下面就来演示一下具体的做法。

13.3 实验步骤

1. 我们访问以 STA 模式工作的 ESPDuino 的 Web 服务，Reset 重启后，首先显示 ESPDuino 的配置信息如下，可以自定义输入 Device ID，如下所示：

ESPduino Setting

WiFi Status:

STA IP: 192.168.2.127

STA MAC: 18-FE-34-D6-F2-D1

Device ID&Key:

Device ID: eee_demo

Device Key: 123456

STA Setting:

STA SSID: Doit

AP List: Doit

Scan: Refresh

STA Password: doit3305

Submit

[Update Firmware](#)

Visit bbs.doit.am for more help

Copyright © www.doit.am 2016(V1.0), All Rights Reserved

图 13-1 STA 联网信息配置

2. 点击 AP List 下拉列表显示了当前所能扫描到的所有活动 WiFi 热点的 SSID, 选择 Doit, 提交。

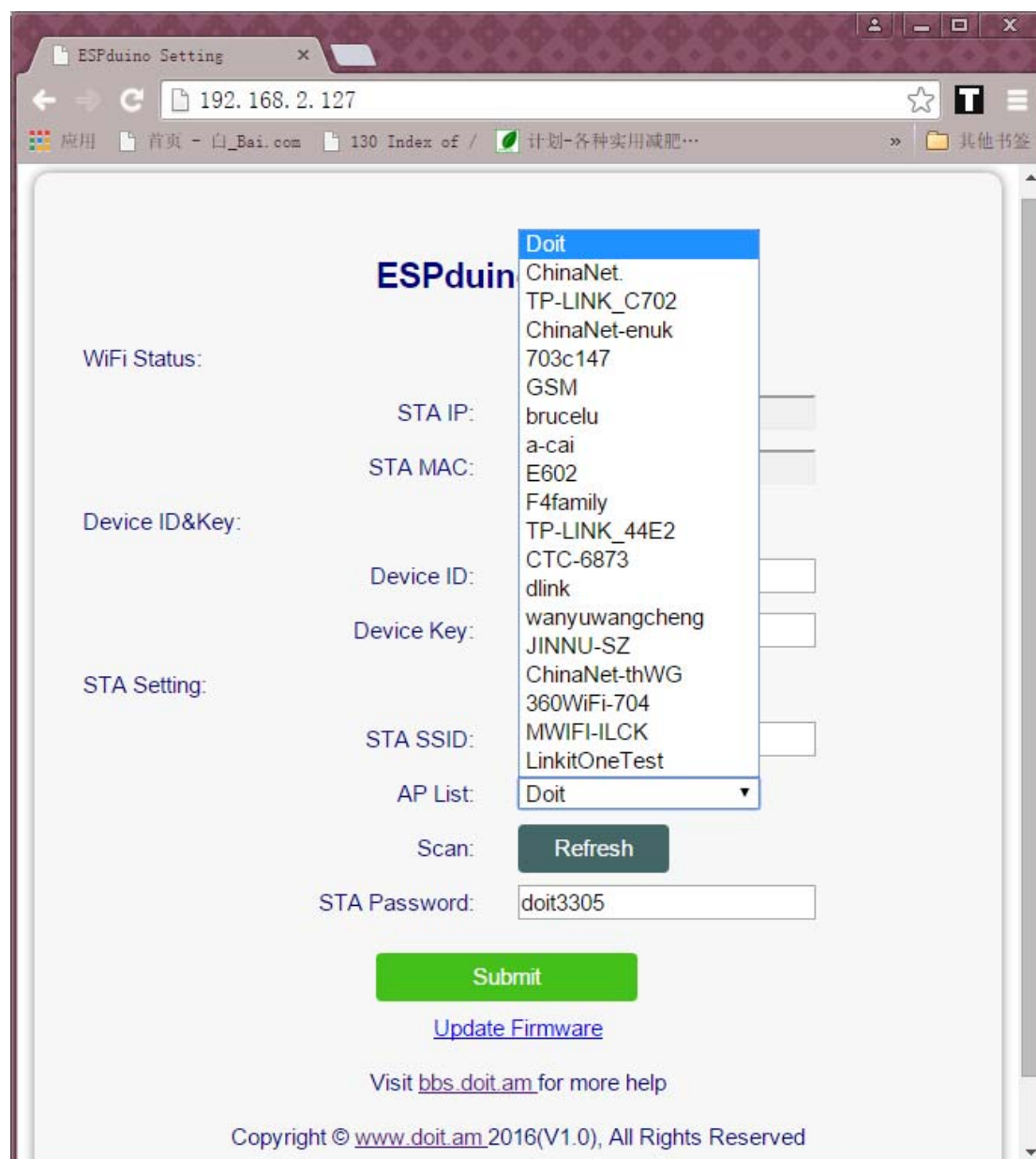


图 13-2 选择网络连接

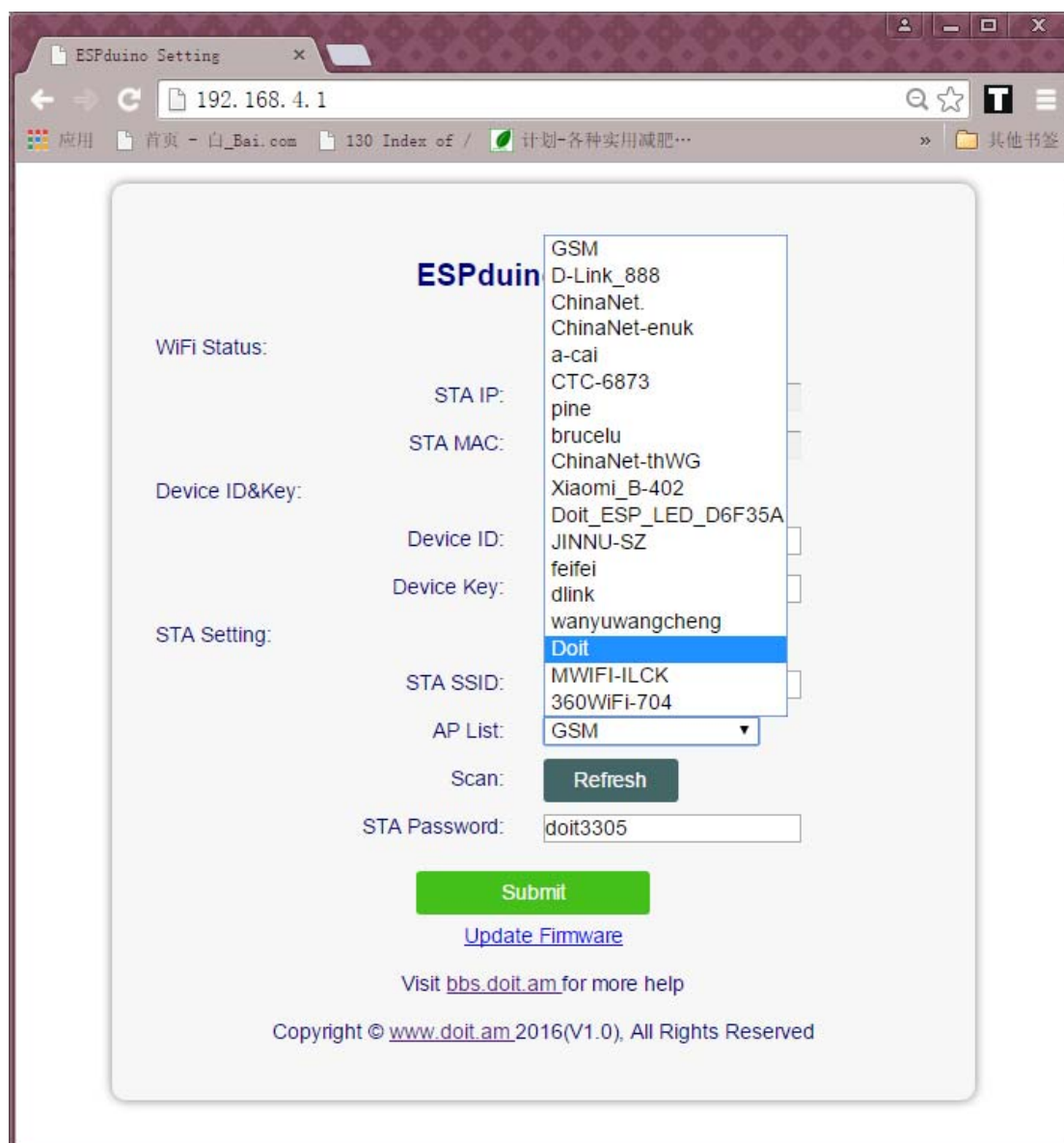


图 13-3 AP 方式挑选可用 WiFi

3. 观察串口监视窗显示的初始信息如下图所示。包含了固件版本信息，STA 模式下所连接的热点信息、设备 ID 和访问密钥，STA 的 MAC 地址，还启动了 UDP 服务器、TCP 客户端，以及一个串口输入方式的配置菜单等等。

```
COM19
Start ESP8266
Firmware Version:V 1.0
SDK Version:1.3.0
-----Read config-----
stassid:Doit
stapsw:doit3305
id:eee_demo
key:123456
-----
[initParseData]
Short press key: smartconfig mode
long press key:factory restore mode
.....
MAC address:18-FE-34-D6-F2-D1
Upload server started at port 8080

StartUDPServer at port:8089

-----Doit Product demo-----
'h' : print this help
'r' : regist product.name:eee_demo_xxxxxx,pass:123456,response:#0#1:regist ok; #0#0:exist yet.
'l' : login.name:eee_demo_xxxxxx,pass:123456;Response:#1#1:ok; #1#0:Password Error;#1#2:usernmae not exist.
'e' : logout.
'd' : send data.send ESPduino timetick to server
't' : get server time.
'k' : send a keepalive package.
Visit:http://book.doit.am/online.php to check
-----
Heap size:30632

Get IP Address: 192.168.2.127

startICPClient
Connected to server:book.doit.am
Keep alive:#5#
```

发送

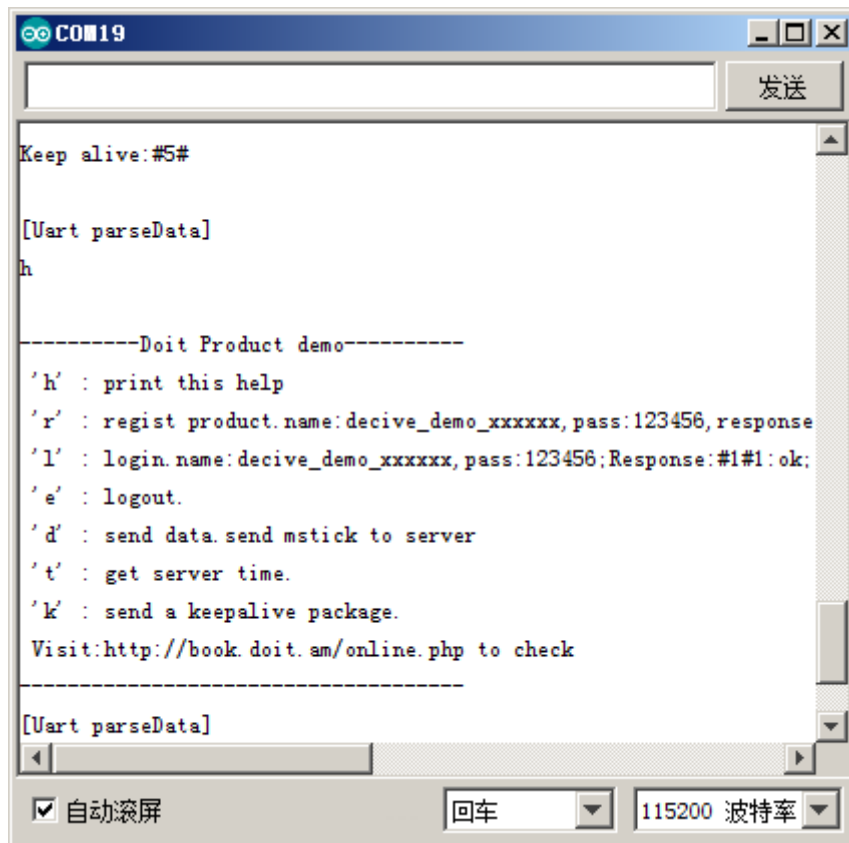
☒ 自动滚屏

没有结束符 115200 波特率

图 13-4 串口启动信息

注意[：initParseData]段，已经告我们，在模块启动过程中（蓝色灯频繁闪烁的阶段），短按键就使之进入了 SmartConfig 模式，这时就可以用专用 app 或微信来配置它登录上网；长按键将会把模块恢复原是的出厂设置。

4. 在串口监视器输入 h，显示指令菜单如下：



5.

图 13-5 串口菜单

其它指令代码含义和示例如下（假设设备名称为 Example,密码为：1234）：

- ‘r’: 设备注册:

#0#Example#1234\r\n 注册设备名为: Example 密码为 1234 的用户

如果注册成功，服务器返回：

#0#1\r\n 代表恭喜您注册成功！已登录！

如果已经存在，则返回：

#0#0\r\n 代表注册失败，设备已存在

- ‘l’: 设备登录

#1#Example#1234\r\n

登录成功返回：

#1#1\r\n

密码错误：

#1#0\r\n

如果用户名称不存在:

#1#2\r\n

- ‘d’: 数据转发:

#2#Example#1234\r\n 向名为 Example 的设备发送数据 1234

如果用户在线直接转发

如果用户不在线直接抛弃

转发出来的数据依然为#2#发出设备名#1234\r\n (是不是应该转发成 1234)

- ‘e’: 注销用户

#3#\r\n 注销当前在线的用户

- ‘t’. 获取服务器时间

#4#\r\n 获取服务器时间

当服务器收到该指令是向发送方发送当前的服务器系统时间

格式为: #4#2016-01-13#15: 34: 26\r\n

- ‘k’.保持心跳(每 60 秒发送一次, 保持链接)

#5#\r\n

6. 在烧过代码后, 注意首先长按 flash 键 3 秒, 恢复出厂设置, 这样之前通过网页之类配置的 id 信息才能够被清空, 当前应用的最新 id 设置信息才可以成功保存到开发板。

下图表示自定义 DeviceID 和密码, 提交后显示配置保存成功, 模块重启。

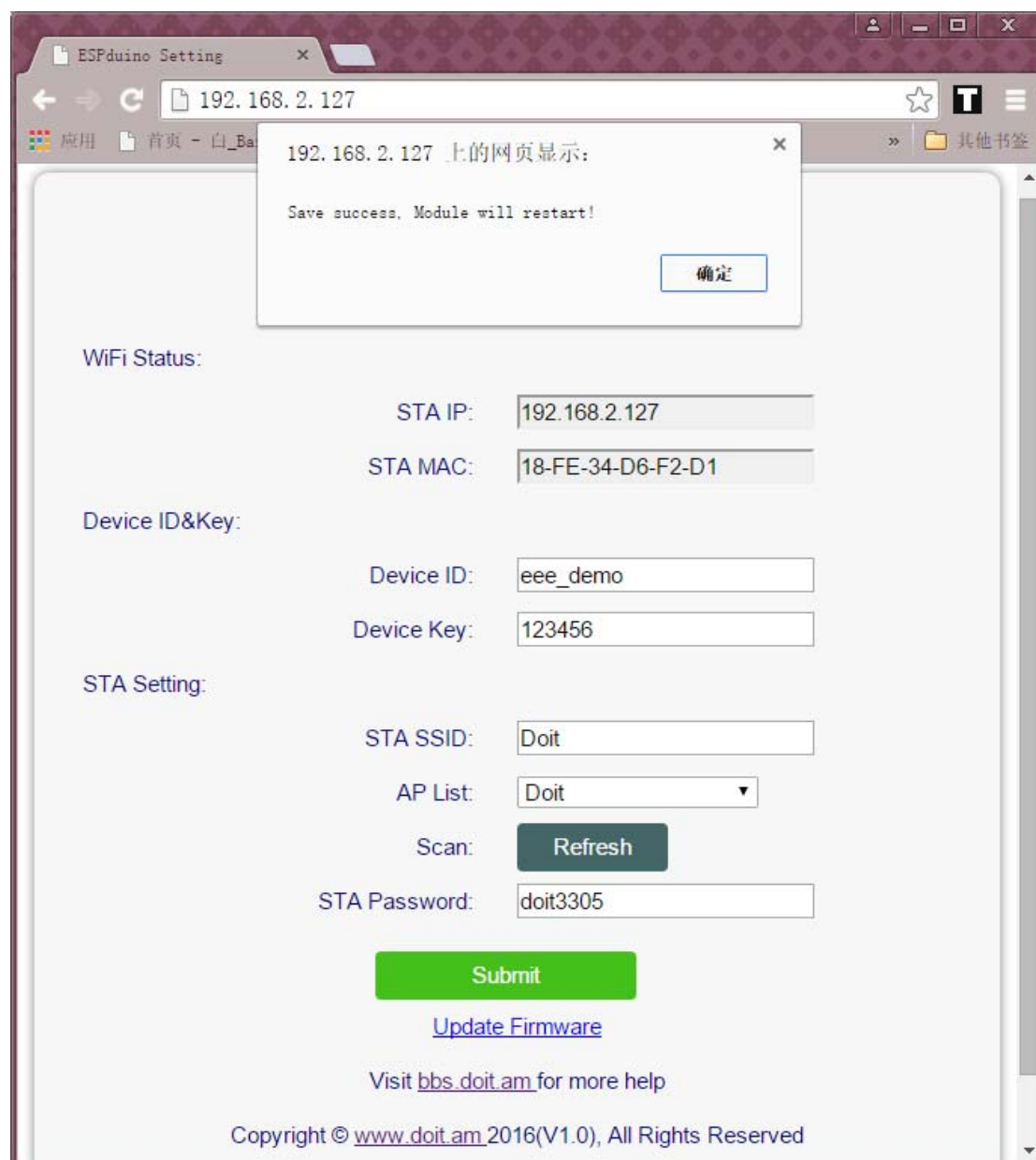


图 13-6 OTA 方式更新成功

做以上操作时，串口监视器显示如下状态序列

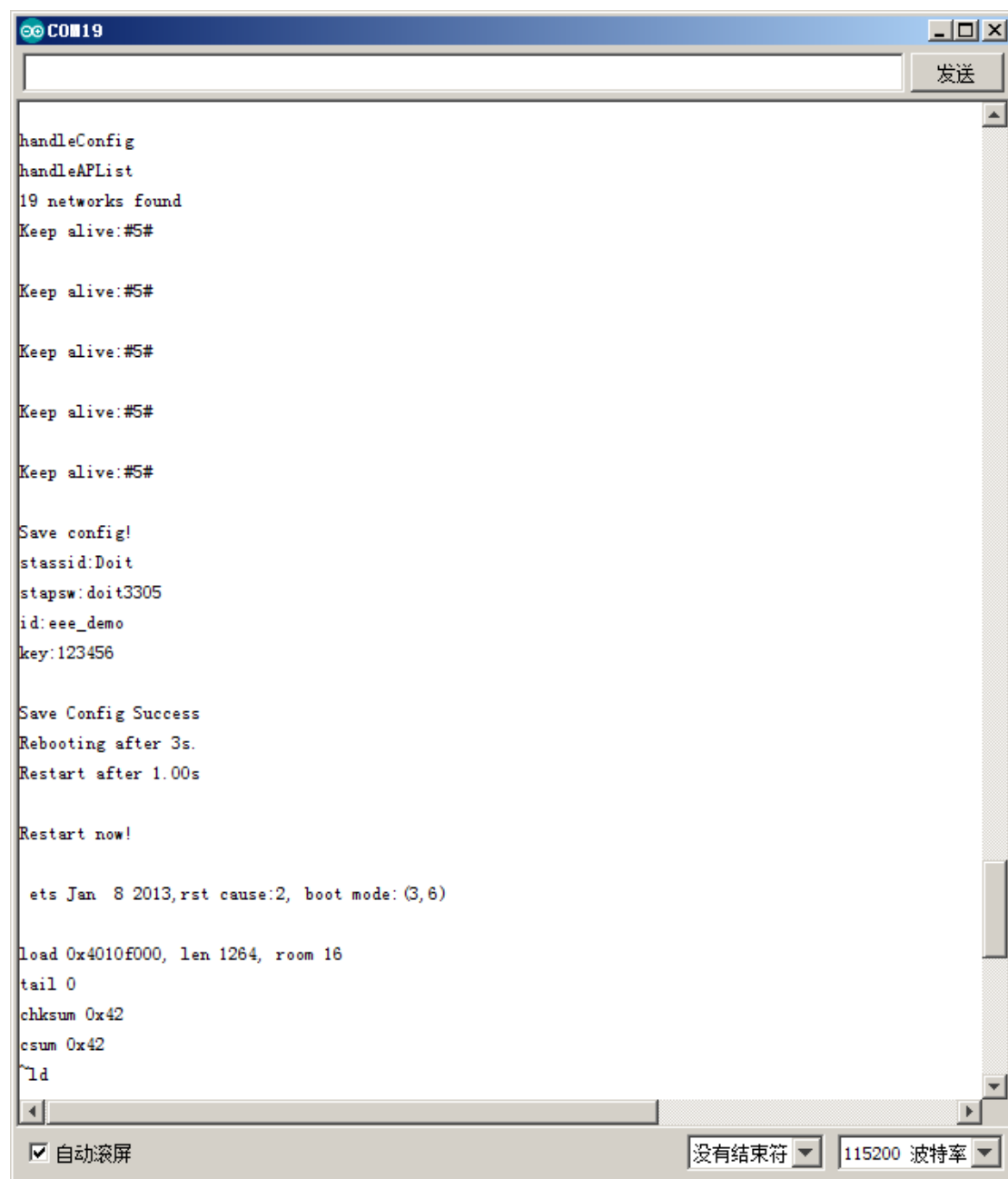


图 13-7 串口反馈的执行状态序列

此时，灯常亮，表示已连接上 Doit 无线网络,串口状态如下：

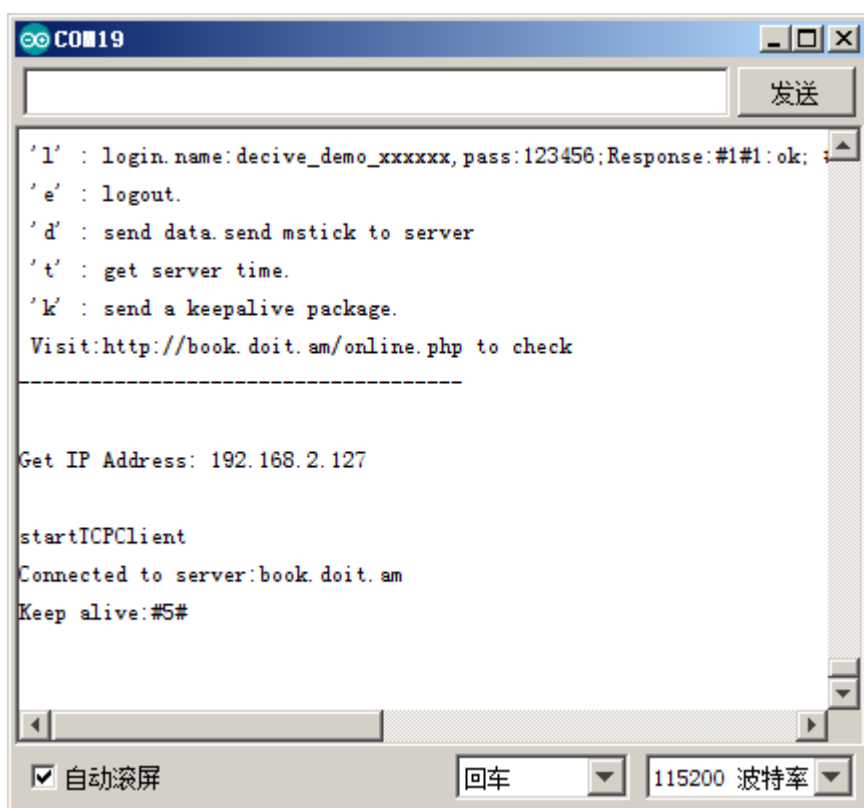


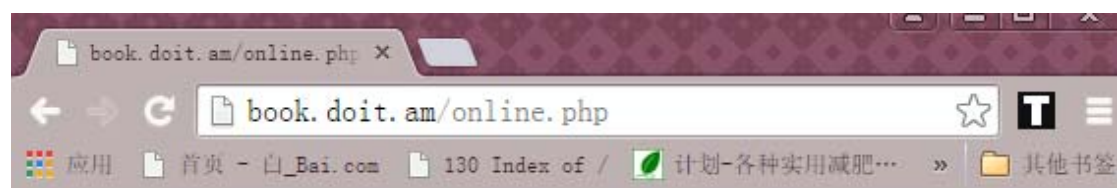
图 13-8 连接状态

7. 输入 r,l,分别进行注册、登陆，通过网址：http://book.doit.am/online.php 进行设备的实时在线查看，下图表示已经有一个自定义名为 eee_demo_D6F2D1 的设备登陆系统



图 13-9 一个已登录的自定义设备

如果恢复出厂设置，设备 ID 默认为 Device_demo_D6F2D1，如下图所示：



设备ID	登录时间	模拟一个设备测试
decive_demo_D6F2D1	2016-01-07#10:58:42	虚拟设备

图 13-10 一个已登录的默认设备

串口显示#1#1，表示设备登录成功。之所以还出现了一个 Invalid cmd,是由于在串口发送指令时选择了回车结束符（\r）作为命令结束的，此时模块把\r'也当作可执行命令了，但在系统里并未定义，所以，此时可以选择“没有结束符”，就能避免这种多余的错误。

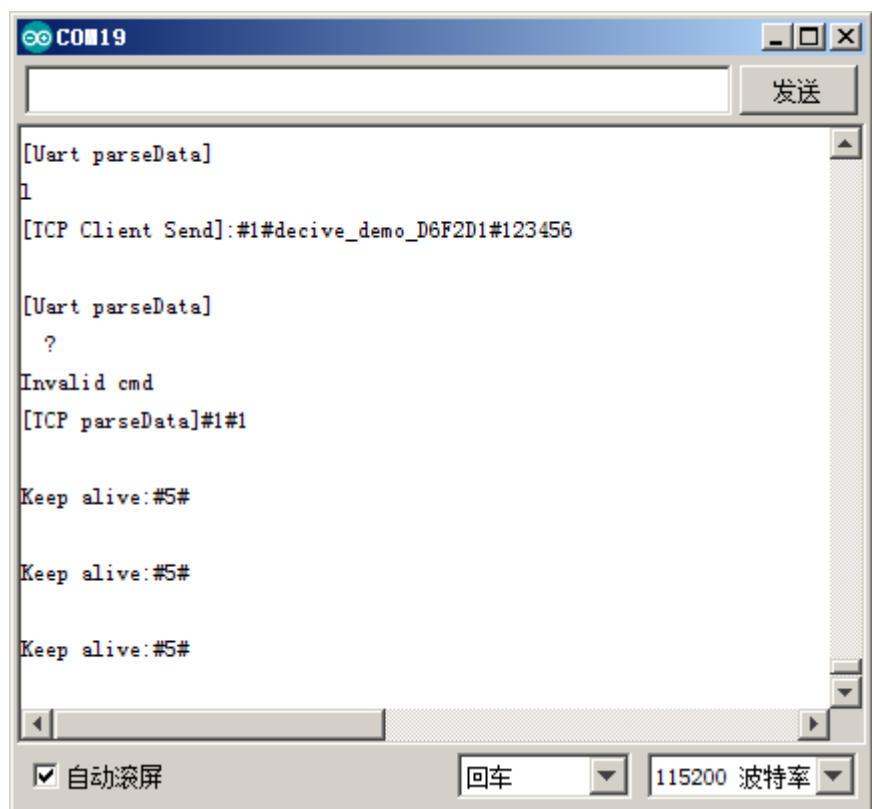


图 13-11 串口反馈

8. 多个客户端登陆后可以看到有以下设备列表：

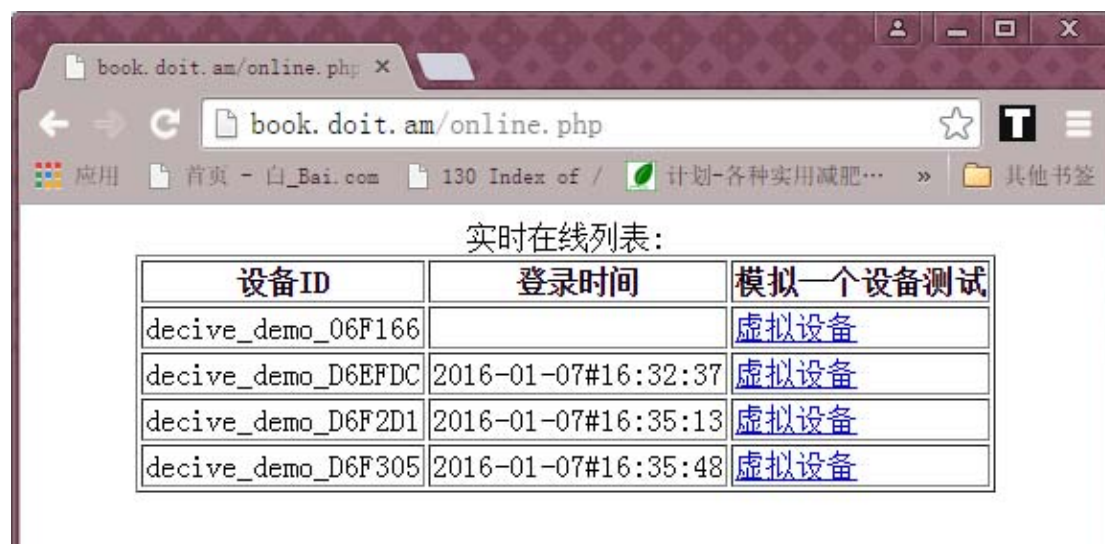


图 13-12 多个登录设备

9. 为了方便用户调试，特别开发了“虚拟设备”，用 web 的方式模拟串口设备的输入输出。

如上图所示，点开后的界面如下：

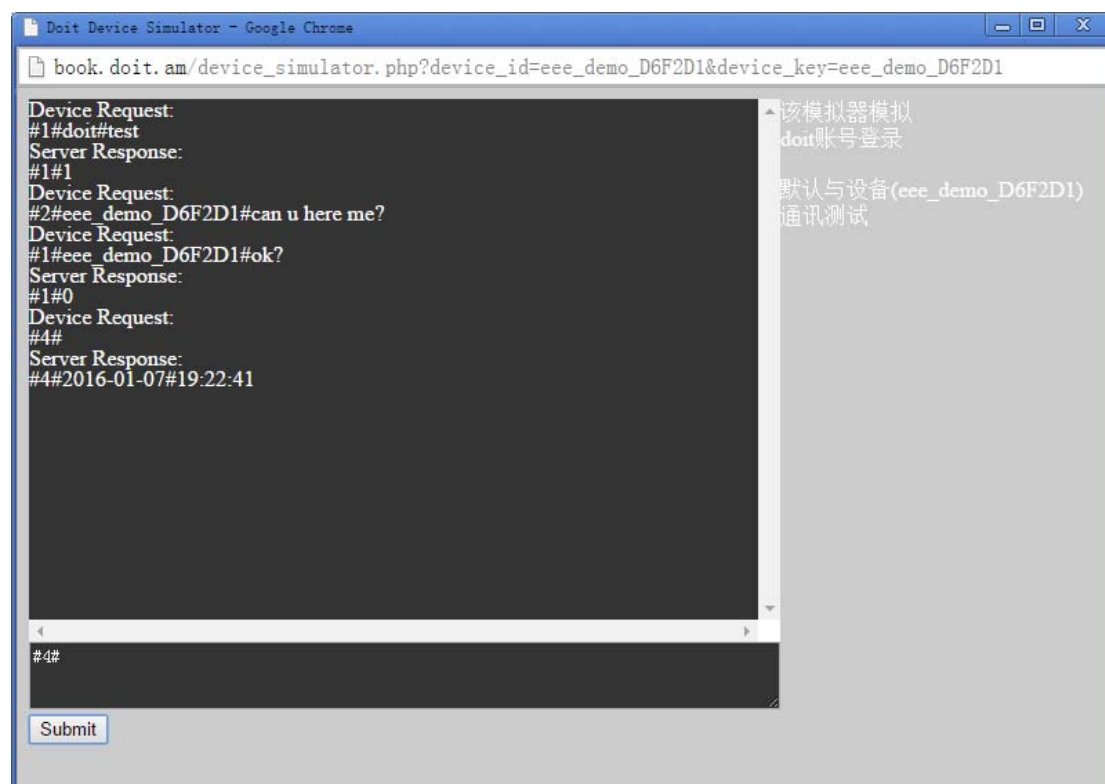


图 13-13 虚拟设备界面

在前面例子中执行的那些指令和返回的信息都可以在这个虚拟设备中完美复现。

13.4 项目结构分析

这个准产品的项目结构由以下几部分组成：



名称	修改日期	类型	大小
httpserver.ino	2016/1/7 1...	Arduino file	7 KB
netTask.ino	2016/1/7 1...	Arduino file	3 KB
parseData.ino	2016/1/7 1...	Arduino file	4 KB
project.h	2016/1/7 1...	H 文件	2 KB
project.ino	2016/1/7 1...	Arduino file	8 KB
webPage.h	2016/1/7 1...	H 文件	11 KB

图 13-14 项目内容

13.4.1 主程序分析

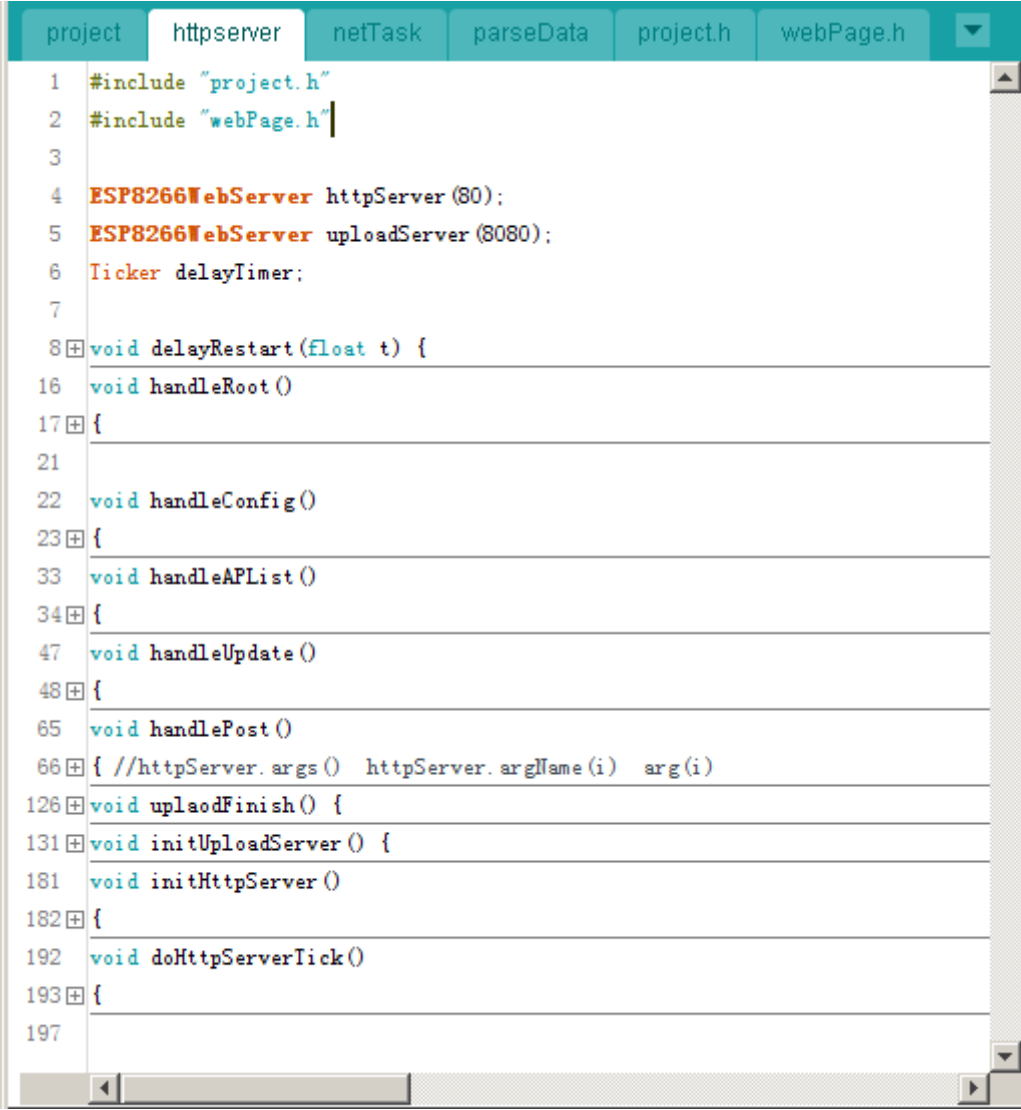
project.ino 文件含有 setup(), 是整个项目的入口, 其功能如下:

```
project  httpserver  netTask  parseData  project.h  webPage.h
1  /*
2   0, 上电: Doit微信开发板上电, 前5秒等待配置阶段LED灯0.1s亮, 0.1s灭: 5秒后1秒亮1秒灭尝试连接到sta
3   1, smartconfig配置
4   前5s内, LED灯快闪(0.1s亮, 0.1s灭), 此时短按Flash或者key键进入smartconfig配置模式, led灯0.3秒亮0.3秒灭
5   在这种情况下可使用微信“四博智联研发部”公众号, wifi配置功能可实现wifi配置上网
6   或者使用ESP-touch app软件进行配置。wifi配置功能可实现wifi配置上网。
7   如果超过3分钟没有配置成功, 模块重启
8   2, Webserver配置: Doit微信开发板上电5s后发出ap信号, 启动httpserver
9   在httpserver启动后, 开发板发出wifi, ssid为: “Doit_ESP_Car_XXXXXX”, 其中XXXXXX为芯片mac地址后六位
10  使用笔记本或者手机连接该ssid, 密码为空, 然后使用浏览器设置, 地址为192.168.4.1
11  3, Doit微信开发板上LED灯常亮表示开发板已经连接上无线路由器
12  4, OTA升级固件
13  使用webserver可以对开发板的固件进行OTA升级。地址为: http://xxx.xxx.xxx.xxx:8080, 或者在web配置页面连接过去。
14  5, 恢复出厂设置
15  启动后, 长按Flash或者Key键超过3秒钟, 恢复出厂设置
16  */
17
18  #include <ESP8266WiFi.h>
19  #include <WiFiClient.h>
20  #include <ESP8266WebServer.h>
21  #include <ESP8266DNS.h>
22  #include <EEPROM.h>
23  #include <ArduinoOTA.h>
24  #include <ESP8266HTTPUpdateServer.h>
25  #include <Ticker.h>
26  #include "project.h"
27
28  #define MAGIC_NUMBER 0xAA
29
30  /*
31   * 保存参数到EEPROM
32   */
33  void saveConfig()
34  {
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54  /*
55   * 从EEPROM加载参数
56   */
57  void loadConfig()
```

```
58 {
88
89 /*
90  * 开启smartconfig，等待配置完成，若超时，延时重启
91  */
92 void doSmartconfig()
93 {
94     Serial.println("[doSmartconfig]");
95     WiFi.mode(WIFI_STA);
96     WiFi.stopSmartConfig();
97     WiFi.beginSmartConfig();
98     int cnt = 0;
99     bool flag_ok = false;
100 while (true) {
123 }
124 /*
125  * 恢复出厂设置
126  */
127 void restoreFactory()
128 {
138
139 /*
140  * 启动前5秒检查用户是否按键，如果有按键判断是短按或者长按，分别进入smartconfig模式或者恢复出厂
141  */
142 void waitKey()
143 {
185
186 /*
187  *初始化WiFi
188  */
189 void initWiFi()
190 {
215
216 /*
217  *WiFiTick
218  *检查是否需要初始化WiFi
219  *检查WiFi是否连接上，若连接成功启动TCP Client
220  *控制指示灯
221  */
222 void wifiTick()
223 {
256 /*
257  *检查串口是否有数据
258  */
259 void doUartTick()
260 {
268 /*
269  *初始化
270  */
271 void setup() {
284 /*
285  *循环
286  */
287 void loop() {
```

13.4.2 HTTP 服务器配置

Httpserver.ino 子单元构建了一个较为完整的 HTTP 服务器。



```
1 #include "project.h"
2 #include "webPage.h"
3
4 ESP8266WebServer httpServer(80);
5 ESP8266WebServer uploadServer(8080);
6 Ticker delayTimer;
7
8 void delayRestart(float t) {
16 void handleRoot()
17 {
21
22 void handleConfig()
23 {
33 void handleAPlist()
34 {
47 void handleUpdate()
48 {
65 void handlePost()
66 { //httpServer.args() httpServer.argName(i) arg(i)
126 void uplaodFinish() {
131 void initUploadServer() {
181 void initHttpServer()
182 {
192 void doHttpServerTick()
193 {
197
```

13.4.3 网络任务处理

netTask.ino 子单元处理 TCP、UDP 数据报的发送和接收。

```
project  httpserver  netTask  parseData  project.h  webPage.h
1  #include "project.h"
2
3  #define MAX_PACKETSIZE 512
4  WiFiClient client;
5  WiFiUDP udp;
6  extern config_type config;
7  extern const char serverAddr[];
8  extern const uint16_t serverPort;
9  char buffTCP[MAX_PACKETSIZE];
10 unsigned int buffTCPIndex=0;
11 unsigned long preTCPTick=0;
12 unsigned long preHeartTick=0;
13 unsigned long preTCPStartTick=0;
14 bool preTCPConnected=false;
15 char buffUDP[MAX_PACKETSIZE];
16 unsigned long preUDPTick=0;
17
18 void startTCPClient()
19 {
35 void sendTCP(char *p)
36 {
46 void doTCPClientTick()
47 { //检查数据，发送心跳
90 void startUDPServer(int port)
91 {
96 void doUdpServerTick()
97 {
106 void sendUDP(char *p)
107 {
111
```

13.4.4 协议解析

parseData.ino 用来进行数据报和协议的解析与执行



```
1
2 #include "project.h"
3
4 #define LEDR 5
5 #define LEDB 4
6 #define LEDG 2
7
8
9 void initParseData()
10 //初始化在这里
13
14 void parseUDPPackage(char *p)
15 {
50 void device_regist(void)
51 {
58 void device_login(void)
59 {
66 void device_logout(void)
67 {
72 void device_dataTrans(void)
73 //发送错误信息到服务器，服务器会送回
80 void device_getTime(void)
81 {
86 void device_keep(void)
87 {
92 void printHelp(void)
93 {
105 void parseICPPackage(char *p)
106 {
110
111 void parseUartPackage(char *p)
112 {
125
```

13.4.5 头文件处理

本项目主要包含了两个头文件: `project.h` 和 `webPage.h`, 前者是用来定义一些项目常量, 后者是用来给 HTTP server 提供 web 功能 UI, 这里只给出 `project.h` 的程序结构, 本书中所有源码均可以 <https://github.com/SmartArduino/ESPduino/tree/master/Book>。

project	httpserver	netTask	parseData	project.h	webPage.h
---------	------------	---------	-----------	-----------	-----------

```
1  #ifndef _MAIN_H_
2  #define _MAIN_H_
3
4
5  extern "C" {
6
7
8  struct config_type
9  {
10
11
12
13
14
15
16
17
18  void startICPClient();
19  void doICPClientTick();
20  void startUDPServer(int);
21  void doUdpServerTick();
22  void sendICP(char *);
23  void sendUDP(char *);
24  void initParseData();
25  void parseICPPackage(char*);
26  void parseUDPPackage(char*);
27  void parseUartPackage(char*);
28
29  void initHttpServer();
30  void doHttpServerTick();
31  void delayRestart(float);
32  void printHelp();
33
34  //常量
35  #define VER "V 1.0"
36  #define DEFAULT_APSSID "Doit_Product_Demo"
37  #define DEFAULT_STASSID ""
38  #define DEFAULT_STAPSW ""
39  #define PINLED 16
40  #define PINKEY 0
41  #define HOST_NAME "DoitWiFi_Device"
42
43  //产品相关
44  #define DEFAULT_ID "device_demo"
45  #define DEFAULT_KEY "123456"
46
47  const char apssid[]=DEFAULT_APSSID;
48  const char serverAddr[] = "book.doit.am";
49  const uint16_t serverPort = 5588;
50  const uint16_t udpPort = 8089;
51
52  unsigned long lastWiFiCheckTick = 0;
53  bool ledState = 0;
54
55  config_type config;
56
57  #endif
```

13.5 所用技术小结

回顾一下，为了完成一个贴近产品的项目，我们综合了前面各章的基本技术，主要包括开发环境的安装、烧写固件的注意事项和技巧、硬件初始化方法、数字信号控制模拟信号的输出、AP 和 STA 的不同设定、扫描 WiFi 热点、“盲”连接 SmartConfig，支持 TCP、UDP 的服务器和客户端的开发，Web Server 的开发，TCP 透传的实现，Web OTA 固件升级技巧，初步 M2M 系统搭建以及一些常用调试工具的使用等。在掌握这些技术的基础上，后两章将带领大家做出几个真正产品级的项目，在这个看脸的世界，颜值颇高呢。

三、产品篇

第 14 章 DoIT 智能控制三色灯

14.1 产品概述

在基础篇第三章我们就已经介绍了关于灯控的内容，那里只是使用板载的 LED 指示灯来演示 ESPDuino 固件控制的效果。本章将介绍一种产品级的灯控系统——DoIT 智能控制三色灯，它是基于 ESP8266 开发，提供微信公众号、Android 和 iOS APP 等三种接入方式，可使用微信或者手机对其进行远程控制（包括在同一局域网内进行控制），是理想的智能灯开发平台。DoIT 团队已开源了大多数的软硬件资源，可以保证开发者快速入门，按自己创意进行自由开发。

Doit 微信智能灯的十大特点：

- （1）基于 ESP8266 SOC 控制芯片，价廉物美；
- （2）提供 Android 和 iOS 手机测试 APP；
- （3）提供基于 Arduino 的控制源代码；
- （4）提供手机 APP 源代码；
- （5）内置 Web Server；
- （6）手机 APP 和微信两种配置上网功能；
- （7）局域网发现功能；
- （8）固件 OTA 升级；
- （9）局域网控制和手机 APP 远程控制；
- （10）微信远程控制。

演示视频：

http://v.youku.com/v_show/id_XMTQzOTQ1NzlwMA==.html?from=s1.8-1-1.2

第 15 章 微信控制智能小车

DoitWiFi 智能小车基于 ESP8266 开发，配合 Doit 团队独家研发的驱动板，可实现两轮或者四轮 WiFi 小车控制。同时提供微信公众号、Arduino APP 和 iosAPP，可完成微信或者手机远程控制、手机局域网控制等功能，是理想的智能小车开发平台。



图 15-1 智能小车

Doit 团队开源大部分的软硬件资源，保证开发者拿到就能快速入门，实现自己创意。智能小车的十大特点：

- (1) 基于 ESP8266 SOC 以及 L298DD 驱动；
- (2) 提供 Arduino 和 ios 手机测试 APP；
- (3) 提供基于 Lua 和 Arduino 的控制源代码；
- (4) 提供手机 APP 源代码；
- (5) 内置 Webserver；
- (6) 手机 APP 和微信配置上网功能；
- (7) 局域网发现功能；
- (8) 固件 OTA 升级；
- (9) 局域网控制和手机 APP 远程控制；
- (10) 微信远程控制。

效果视频：

http://v.youku.com/v_show/id_XMTQzODk0MjkxMg==.html

http://v.youku.com/v_show/id_XMTMxMzkyMTYxNg==.html?from=s1.8-1-1.2

本章内容仅在纸质书中提供。

电子书:

<https://item.taobao.com/item.htm?id=526035950989&qq-pf-to=pcqq.c2c>

纸质书:

<https://item.taobao.com/item.htm?id=526043053791&qq-pf-to=pcqq.c2c>

第 16 章 ESPDuino MP3

基于 ESPDuino 制作 MP3 播放器，实现 WiFi、微信控制 MP3 播放功能。主要特点：

-
- (1) 硬件包括：ESPduino、MP3 硬件解码芯片、TF 卡、喇叭
 - (2) OTA 固件升级
 - (3) 通过 WiFi 实现 SD 卡内 MP3 文件管理：文件查看、上传、下载、改名、删除等
 - (4) 通过 ESPduino 本地/远程/微信控制 MP3 解码芯片播放音频文件

本章内容仅在纸质书中提供。

电子书：

<https://item.taobao.com/item.htm?id=526035950989&qq-pf-to=pcqq.c2c>

纸质书：

<https://item.taobao.com/item.htm?id=526043053791&qq-pf-to=pcqq.c2c>

第 17 章 ESPduino WiFi 屏智能信息站

基于 ESPduino，实现在 TFT 屏上显示文字、图形和图片。主要特点：

-
- (1) 硬件包括：ESPduino、TFT 屏
 - (2) OTA 固件升级
 - (3) 通过 WiFi 实现 TFT 屏的文字显示、图形显示、图片展示等功能。文字内容包括：微信留言、天气预报、重要事件提醒等；图形和图片功能根据需要展示。
 - (4) 通过 ESPduino 控制屏幕显示内容，实现本地局域网控制、远程控制、微信控制等。



图 17-1 带显示屏的 ESPduino 模组

本章内容仅在纸质书中提供。

电子书：

<https://item.taobao.com/item.htm?id=526035950989&qq-pf-to=pcqq.c2c>

纸质书：

<https://item.taobao.com/item.htm?id=526043053791&qq-pf-to=pcqq.c2c>

第 18 章 ESPduino 机械臂

基于 ESPduino 和 doit 2 路电机 16 路舵机驱动板，实现 WiFi 控制机械臂运动。开源机械臂的控制算法。

主要特点：

- (1) 六自由度机械臂，完全模仿工业机器人。
- (2) 基于 ESPDuino 实现 WiFi 控制 6 路舵机，控制和驱动高度集成。
- (3) 机械臂的运动学建模教程、算法。
- (4) 开源控制算法。
- (5) 基本的控制方法案例：手臂末端画直线，末端画圆，手臂写字等。

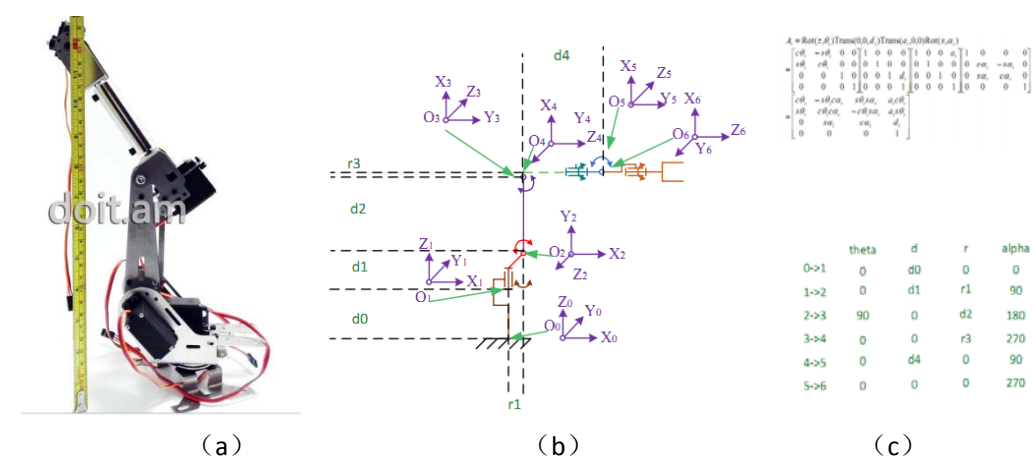


图 18-1 机械臂的构造

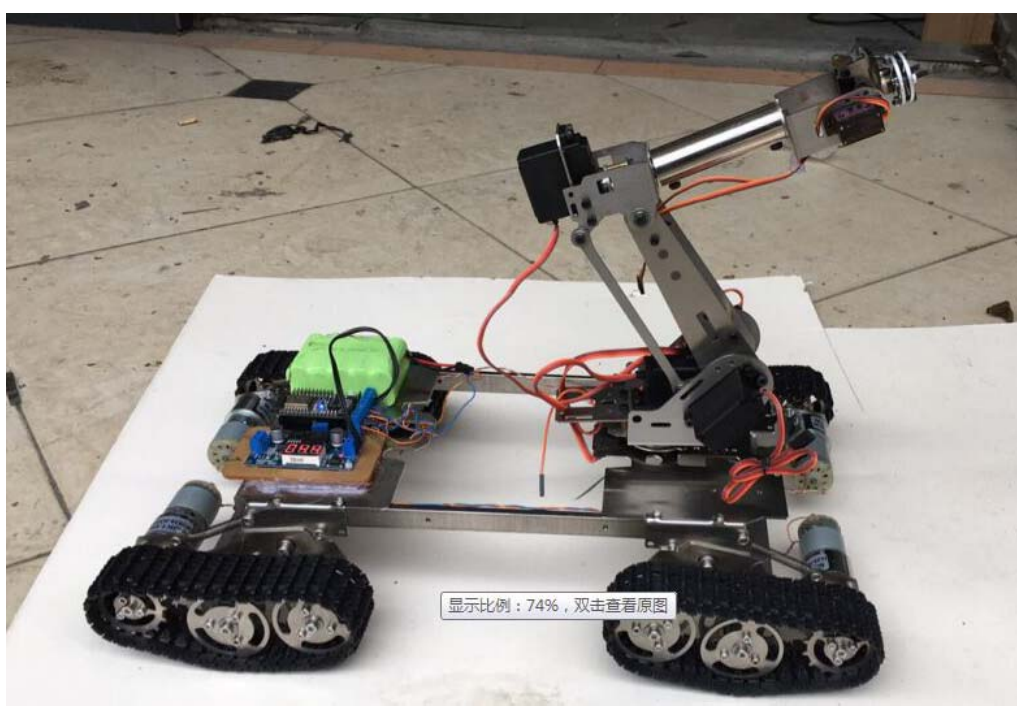


图 18-2 机械臂产品样机

附录-ESP8266 Arduino 核心说明

1. 数字 IO（Digital IO）

Arduino 中的 Pin（引脚）号直接与 ESP8266 GPIO 的引脚号对应通信。
pinMode, digitalRead, 以及 digitalWrite 函数不变，所以要读取 GPIO2, 可调用 digitalRead(2)。Digital pins 0—15 可以是 INPUT, OUTPUT, 或者 INPUT_PULLUP. 第 16 引脚（Pin 16）可以是 INPUT, OUTPUT 或 INPUT_PULLDOWN. 在初始启动时，引脚被配置为 INPUT。Pins 还可以完成其他功能，诸如 Serial, I2C, SPI。这些功能通常由相应的函数库完成。下图展示了同流行的 ESP-12 模块引脚的对应关系。Digital pins 6—11 并没有显示在上图中，这是因为在大多数模块中，这些引脚是用来连接闪存的，试图使用这些引脚作为 IO 将会导致程序崩溃。

注意某些开发板和模块(ESP-12ED, NodeMCU 1.0) 仍将使用第 9 和第 11 引脚如果闪存芯片工作在 DIO 模式(与默认的 QIO 模式相反), 它们可用于 IO。

引脚的中断功能是通过 `attachInterrupt`, `detachInterrupt` 函数所支持的。除了 GPIO16, 中断可以绑定到任意 GPIO 的引脚上。所支持的标准 Arduino 中断类型有: CHANGE, RISING, FALLING。

2.模拟输入 (Analog input)

ESP8266 具有一个用户可用的单通道 ADC。它可以被用来读取 ADC 引脚的电压值, 也可以读取模块的供电电压 (VCC)。要读取 ADC 引脚的外部电压, 使用 `analogRead(A0)`。输入电压范围 在 0 — 1.0V 之间。要读取 VCC 电压, ADC 引脚必须保持开路状态。此外, 以下划线的命令应该加入工程中。

```
ADC_MODE(ADC_VCC);
```

这一命令应在任何函数体外出现, 例如紧跟在工程的 `#include` 语句后面。

3.模拟输出 (Analog output)

`analogWrite(pin, value)` 可以在给定的引脚上实行软 PWM 的功能。PWM 一般使用 0 到 18 号引脚。调用 `analogWrite(pin, 0)` 来禁止对应引脚的 PWM 功能。`value` 一般在 0 到 PWM_RANGE 范围, 默认值是 1023。PWM 的调控范围可以通过调用 `analogWriteRange(new_range)` 来改变。

PWM 默认频率是 1kHz。改变默认频率的函数是 `analogWriteFreq(new_frequency)`。

4.计时和延时 (Timing and delays)

`millis()` 和 `micros()` 分别返回重启 (reset) 后所经过的毫秒数和微秒数。

`delay(ms)` 可以使工程暂停一个给定的毫秒数的时间间隔，以便允许 **WiFi** 和 **TCP/IP** 的任务运行。`delayMicroseconds(us)` 可以使工程暂停一个给定的微秒数的时间间隔。

当 **WiFi** 连接上时，记住除了工程本身，还有很多代码要在芯片上运行。在每一次 `loop()` 函数结束或者调用 `delay` 时，**WiFi** 和 **TCP/IP** 库才有机会处理任何待处理的事件。如果在你的工程某处有一个循环很耗时(>50ms)且又没有调用 `delay`，就应该考虑添加一个 `delay` 函数的调用来保证 **WiFi** 协议栈工作的平滑性。

与 `delay(0)` 等价的是 `yield()` 函数。另外，`delayMicroseconds` 函数会阻止其他任务的运行，因此,在延时 20 毫秒以上时，不建议使用此函数。

5.串口 (Serial)

`Serial` 对象的工作方式与传统的 **Arduino** 设备完全一样。除了硬件 **FIFO** (128 字节用于发送, **TX** 和接收, **RX**) 硬件串口还有额外的 256-字节的 **TX** 和 **RX** 缓存。发送和接收都由中断驱动。当 **FIFO**/缓存分别满/空时，`Write` 和 `read` 函数只阻塞工程的异常。

`Serial` 使用 **UART0**, 对应引脚是 **GPIO1 (TX)**和 **GPIO3 (RX)**。在 `Serial.begin` 执行之后，调用 `Serial.swap()`，可以将 `Serial` 重新映射到 **GPIO15 (TX)** 和 **GPIO13 (RX)**。再次调用 `swap` 将 **UART0** 重新映射回 **GPIO1** 和 **GPIO3**。

`Serial1` 使用 **UART1**, **TX** 引脚是 **GPIO2**。由于 **RX** 引脚被用来连接闪存芯片，因此 **UART1** 不能被用来接收数据。要使用 `Serial1`，调用 `Serial1.begin(baudrate)`。

默认情况下，调用 `Serial.begin` 时，来自 **WiFi** 库的诊断输出是不可用的。使能调试输出的模式，可调用 `Serial.setDebugOutput(true)`。要把调试输出重定向到 `Serial1`，调用 `Serial1.setDebugOutput(true)`。

在 `printf()` 函数中，还可用使用 `Serial.setDebugOutput(true)` 来调试输出。

`Serial` 和 `Serial1` 对象支持 5, 6, 7, 8 数据位，奇校验(**O**)，偶校验 (**E**)，以及无奇偶校验 (**N**)，1 或 2 个停止位。要想设置成所希望的模式，调用 `Serial.begin(baudrate, SERIAL_8N1)`, `Serial.begin(baudrate, SERIAL_6E2)`，等等。

6.编程 (Program)

内存编程的特性与传统 **Arduino** 的工作方式完全一样；将只读数据和字符串放置于只读存储器，并为你的应用释放堆栈空间。重要区别在于 **ESP8266** 上的字符串并不 `the literal strings are not pooled`. 这意味着代码中在 `F("")`和/或 `PSTR("")`

内定义的相同的字符串将为每一个实例占据空间。因此，你需要自己处理字符串的复制。

7.WiFi 库(ESP8266WiFi library)

与 WiFi 支持库（shield library）极为相似，区别在于：

- `WiFi.mode(m)`：设定 **WiFi** 模式为 `WIFI_AP`, `WIFI_STA`, 或 `WIFI_AP_STA`
- 调用 `WiFi.softAP(ssid)` 设置一个开放网络
- 调用 `WiFi.softAP(ssid, password)` 设置一个 **WPA2-PSK** 的网络 (密码至少要 8 个字符)
- `WiFi.macAddress(mac)` 用于 **STA**, `WiFi.softAPmacAddress(mac)` 用于 **AP**
- `WiFi.localIP()` 用于 **STA**, `WiFi.softAPIP()` 用于 **AP**.
- `WiFi.RSSI()` 无效
- `WiFi.printDiag(Serial)` 可以打印出一些诊断信息
- `WiFiUDP` 类支持在 **STA** 接口发送和接收广播包。当发送一个广播包时，使用 `udp.beginPacketMulticast(addr, port, WiFi.localIP())` 替换 `udp.beginPacket(addr, port)`。当监听广播包时，使用 `udp.beginMulticast(WiFi.localIP(), multicast_ip_addr, port)` 替换 `udp.begin(port)`。可使用 `udp.destinationIP()` 来识别收到的包是发送到广播地址还是单播地址。还要注意，在 **softAP** 接口广播无效。

`WiFiServer`, `WiFiClient`, 和 `WiFiUDP` 的功能和使用 **WiFi** 基本库非常相似。在下面的地址里提供了使用这个库的四个例子，能发现更多命令：

<http://www.arduino.cc/en/Reference/WiFi>

8.计时器（Ticker）

这个库用于周期性地重复调用函数，包含两个例子。

目前不建议使用 **Ticker** 回调函数来阻塞 **IO** 操作（网络、串口、文件）。相反，在 **Ticker** 回调函数中设置一个标记，并在 **loop** 函数中检测。

电子可擦除可编程只读存储器（**EEPROM**）

同标准的 **EEPROM** 类有略微不同，在开始读写之前，需要调用 `EEPROM.begin(size)`, **size** 代表需要使用的空间大小. **Size** 范围在 4 和 4096 字节之间。

`EEPROM.write` 不会直接向闪存写入，相反，必须调用 `EEPROM.commit()`，在任何时候，想要将更新保存到闪存时，需使用 `EEPROM.end()`，并会在 **RAM** 中释放 **EEPROM** 内容的副本。

EEPROM 库使用位于 0x7b000 扇区的闪存作为存储空间。库中包含三个例子。

9. I2C (Wire library)

Wire 库目前最高支持大约 450KHz 的主模式。在使用 I2C 之前，要调用 `Wire.begin(int sda, int scl)` 对 SDA 和 SCL 设置引脚，也即在 ESP-01 上是 `Wire.begin(0, 2)`，否则，会默认为引脚 4(SDA)和引脚 5(SCL)。

10.SPI

SPI 库支持包含事务的全部 SPI API，包括设置阶段 (CPHA)。并不支持设置时钟率 (CPOL) is not supported (SPI_MODE2 和 SPI_MODE3 均无效)。

11.ESP-specific APIs

仅在 Alpha 版本中，ESP 对象中有关深度睡眠和看门狗定时器的 APIs 才可用。

`ESP.deepSleep(microseconds, mode)` 将使芯片进入深度睡眠模式。Mode 取值为 `WAKE_RF_DEFAULT`, `WAKE_RFCAL`, `WAKE_NO_RFCAL` 或 `WAKE_RF_DISABLED`。(GPIO16 需要与 RST 绑定来将芯片从深度睡眠模式唤醒)

`ESP.restart()` 重启 CPU。

`ESP.getFreeHeap()` 返回自由堆栈空间大小。

`ESP.getChipId()` 返回 32 位整数的 ESP8266 芯片标识 ID。

以下是几个可用来获取闪存芯片信息的 APIs:

`ESP.getFlashChipId()` 返回 32 位整数的闪存芯片标识 ID。

`ESP.getFlashChipSize()` 返回闪存大小，字节为单位，可参加 SDK (往往小于实际大小)。

`ESP.getFlashChipSpeed(void)` 返回以 Hz 为单位的闪存芯片频率。

`ESP.getCycleCount()` 返回无符号 32 位整数标识的 CPU 从启动开始的指令周期数目。在需要精确时间，诸如位处理的短事件中十分有用。

`ESP.getVcc()` 可用来度量供电电压。为了使这一特性有效，ESP 需要在启动时重新配置 ADC。要使用 `getVcc`，将下列语句添加到工程的首部:

```
ADC_MODE(ADC_VCC);
```

此模式下，TOUT 引脚需断开连接。

注意，默认情况下，通过使用 `analogRead(A0)` 和 `ESP.getVCC()` 读取 TOUT 引脚状态来配置 ADC 是不可用的。

12.OneWire

将寄存器定义包含在 `OneWire.h` 文件中，`OneWire` 库即可配合 ESP8266 工作。注意，如果在你的 `Arduino/libraries` 文件夹下已经有了此库，将会取代这一函数包。

13.mDNS 和 DNS-SD 响应器(ESP8266mDNS 库)

允许工程对诸如“foo.local”域名解析响应 DNS 的广播请求，还包括 DNS-SD (服务发现) 请求。当前，此库只工作在 STA 接口，AP 接口并不支持。 详见相关例子。

14.SSDP 响应器(ESP8266SSDP)

SSDP 是另外一种服务发现协议，在 Windows 中支持，详见相关例子。

15.DNS 服务(DNSServer 库)

实现一个简单的 DNS 服务器，可以在 STA 和 AP 模式中都能使用。目前，DNS 服务器只能支持单域名(对于其他域名将响应 NXDOMAIN 或自定义的状态码)。利用它，客户端可以使用域名打开一个在 ESP8266 运行的 web 服务器，而不用使用 IP 地址。详见相关例子。

16.伺服电机（Servo）

这个库可以实现控制 RC (hobby) 伺服电机的能力。在任意可用的输出引脚上，最多支持 24 个伺服电机。在默认情况下，前 12 个伺服电机将使用 `Timer0`，目前也不会和其他冲突。超过 12 个的伺服电机将使用 `Timer1` 并起作用。一些 RC 伺服电机可接受 ESP8266 的 3.3V 引脚的 IO 数据，大多数在 3.3V 以下不能运行，而需要其他的电源供应，以符合规范要求。要确保在 ESP8266 和伺服电机的电源供应端保持接地。

17.其他的库 (没有包含在 IDE 中)

不依赖于对 AVR 寄存器底层访问方式的库依旧可以运行良好，以下是一些经过验证有效的函数库：

- [arduinoWebSockets](#) – 兼容 ESP8266 (RFC6455)的 WebSocket 服务器和客户端。
- [aREST](#) REST API 句柄库。
- [Blynk](#) – 提供给制造商的简易 IoT 框架(试试 [Kickstarter 页面](#)).
- [DallasTemperature](#)
- [DHT11](#) – 下载最新的 v1.1.0 库，无需更新。旧版本将使用 DHT dht (DHTPIN, DHTTYPE, 15) 对 DHT 进行初始化。
- [NeoPixel](#) - Adafruit's NeoPixel 库，现在支持 ESP8266 (在 Arduino 的库管理器中使用 1.0.2 或更高版本)。
- [NeoPixelBus](#) - Arduino NeoPixel 库，兼容 ESP8266。使用 ESP8266 的 "NeoPixelAnimator" 分支 来获取更多的 HSL 颜色支持。
- [PubSubClient](#) 由@Imroy 开发的 MQTT 库。
- [RTC](#) – 兼容 ESP8266，为 Ds1307 & Ds3231 开发的 Arduino 库。
- [Souliss, Smart Home](#) – 基于 Arduino, Android 以及 openHAB 的智能家庭框架。
- [ST7735](#) - Adafruit's ST7735 库，被修改用来兼容 ESP8266。由于仍旧是 AVR 规范，注意在样例中要修改引脚定义。

致谢

在本书成文过程中，很多内容参考了开源项目，当然也包括已出版的文献资料、工作报告、技术说明，还有更多互联网资源，包括但不限于：

www.doit.am

www.smartarduino.com

baike.baidu.com

www.arduino.cc

github.com/esp8266/Arduino

blog.csdn.net

www.espressif.com

.....

在此一并致谢！

我们相信：

物联使世界更美好！

参考文献

- 1.ESP8266EX 芯片规格书
- 2.艾拉斯岱尔·艾伦 博客
- 3.<http://www.cnblogs.com/TankXiao/archive/2012/02/13/2342672.html>
